

NOVA ROMA

DOKUMENTACJA PROJEKTU



Logika, zasady, zależności modułów i decyzje projektowe gry strategiczno-ekonomicznej w jednym pliku HTML.

Cztery nacje: Frankowie, Saraceni, Wikingowie, Słowianie.

Wersja dokumentu: 1.0 · **Zbudowano:** 2026-10-06

Stan kodu: commit cf37876 z 2026-10-06

Stan projektu: fazy 0–9 ukończone (teren v2, ścieżki i drogi, zasady zabudowy, wyrównywanie, logistyka, fauna i polowanie, menu RTS z Piaskownicą); następna: faza 9B (ożywienie ludzi — fizyczna logistyka)

Opracowano na podstawie: kodu *Nova_Roma.html* i narzędzi *tools/*; dokumentu „Ekonomia „Nova Roma” — wersja finalna”; dokumentu „Nova Roma — specyfikacja ekonomii do wdrożenia” (v3.2 / v3.6); planu faz i decyzji projektowych podjętych w trakcie prac.

Spis treści

1. Wprowadzenie	5
1.1 Czym jest Nova Roma	5
1.2 Zakres dokumentu i hierarchia źródeł	5
1.3 Stan projektu	6
1.4 Struktura repozytorium	6
1.5 Uruchamianie i parametry adresu	6
2. Architektura programu	8
2.1 Jeden plik, wiele modułów	8
2.2 Zależności między modułami	9
2.3 Stan gry	11
2.4 Pętla gry i kolejność ticku	12
2.5 Losowość i determinizm	13
2.6 Cień symulacji	14
2.7 Przełączniki scenariusza	14
2.8 Wejście do gry: UI.startGame	14
3. Ekonomia przepływowa	15
3.1 Zasady nadrzędne	15
3.2 Jednostki i czas	15
3.3 Zasoby i magazyny	15
3.4 Obsada pracowników	16
3.5 Algorytm produkcji budynku	17
3.6 Narzędzia i zawory bezpieczeństwa	18
3.7 Drewno i las	19
3.8 Woda	19
3.9 Złóża	19
3.10 Złoto	20
4. Ludność, jedzenie i popularność	21
4.1 Jedzenie	21
4.2 Głód	22
4.3 Mieszkania i imigracja	22
4.4 Popularność	23
4.5 Dobrobyt	23
5. Budowa i budowniczowie	25
5.1 Otwarcie placu	25
5.2 Budowniczowie	25
5.3 Etapy	26
5.4 Złoto, rezerwa, anulowanie	27
5.5 Odbudowa po pożarze	27
6. Katalog budynków	28
6.1 Jak czytać tabele	28
6.2 Budynki wspólne	28

6.3	Dwór	29
6.4	Frankowie	29
6.5	Saraceni	30
6.6	Wikingowie	30
6.7	Słowianie	31
7.	Nacje	33
7.1	Frankowie	33
7.2	Saraceni	34
7.3	Wikingowie	36
7.4	Słowianie	37
7.5	Przepisy botów	39
8.	Haracz seniora	40
8.1	Poziomy i osobominuty	40
8.2	Presety i terminy	41
8.3	Frankowie: żołnierze	42
8.4	Saraceni: złoto i rynek	42
8.5	Wikingowie: wyprawy	43
8.6	Słowianie: danina koszykowa	43
8.7	Pojemność przepisu	44
8.8	Doradca a haracz	44
9.	Zdarzenia i kryzysy	45
9.1	Napad na karawanę	45
9.2	Kryzysy wg harmonogramu	45
10.	Teren, klimaty i mapy	47
10.1	Mapa i kafle	74
10.2	Klimaty i typy map	47
10.3	Potok generatora	48
10.4	Walidacja	49
10.5	Rodzaje pól: koszty i przechodność	50
10.6	Statystyki map domyślnych	50
11.	Zasady zabudowy	51
12.	Ruch, drogi i logistyka	52
12.1	Ścieżki: moduł Path	52
12.2	Widoczni piesi: moduł Walkers	52
12.3	Drogi: moduł Roads	53
12.4	Sprawność logistyczna: moduł Logistics	53
12.5	Co z tego wynika dla gracza	54
13.	Fauna i polowanie	55
14.	Interfejs, menu i Doradca	56
14.1	Wejście do gry: menu główne i Piaskownica	56
14.2	Pasek stanu (HUD)	57
14.3	Pasek budowy	58

14.4	Panele informacyjne	58
14.5	Doradca	58
14.6	Co jeszcze zaplanowano	59
15.	Grafika i silnik sprite'ów	60
16.	Testy i jakość	61
16.1	Trzy warstwy testów	61
16.2	Boty testowe	61
16.3	Scenariusze i niezmienniki	61
16.4	Testy modułów	62
16.5	Testy w przeglądarce	62
16.6	Bramka każdej fazy	62
16.7	Wydajność	63
16.8	Trudność i krzywa nauki	63
17.	Decyzje projektowe i historia	64
17.1	Historia prac	64
17.2	Decyzje użytkownika (wiążące)	64
17.3	Decyzje projektowe (z uzasadnieniem)	65
17.4	Decyzje otwarte	65
18.	Specyfikacja a implementacja	67
A	Wyniki symulatora ze specyfikacji	68
B	Wyniki botów i scenariuszy w grze	69
B.1	Normalna gra botów	69
B.2	Scenariusze akceptacyjne	69
C	Repozytorium, polecenia i budowa	72
C.1	Polecenia	72
C.2	Skrypty testowe	72
C.3	Zasady pracy z kodem	73
D	Procedura aktualizacji dokumentacji	74
D.1	Lista kontrolna po etapie	74
D.2	Który rozdział zmienia która zmiana	74
E	Fizyczna logistyka i ożywienie ludzi — analiza i projekt (Faza 9B)	75
E.1	Zamówienie	75
E.2	Stan obecny	75
E.3	Wniosek 1: tempo gry decyduje o tym, jak szybko mogą chodzić ludzie	75
E.4	Wniosek 2: skala transportu	76
E.5	Model: sprawność z cyklu pracy	76
E.6	Ochrona bilansu	78
E.7	Ożywienie wizualne	79
E.8	Plan etapów i bramki	79
E.9	Decyzje do potwierdzenia	79

1. Wprowadzenie

1.1 Czym jest Nova Roma

Nova Roma to gra strategiczno-ekonomiczna w stylu RTS lat 90. i 2000. (Settlers, Twierdza, Age of Empires), zapisana w **jednym pliku HTML** (`Nova_Roma.html`): JavaScript bez zależności, rysowanie na `canvas` 2D, grafika w całości proceduralna (żadnych plików graficznych). Gracz zakłada osadę wokół Dworu, buduje łańcuchy produkcji, dba o ludność i popularność, a jedyną presją jest **haracz seniora**: w ustalonych terminach trzeba dostarczyć towar w walucie swojej nacji. Nikt nie walczy na mapie — gra jest „czysto ekonomiczna”.

Do wyboru są cztery nacje, każda z innym napięciem ekonomicznym:

Nacja	Hasło	Waluta haraczu	Oś napięcia
Frankowie	kamień, chleb i żelazo	żołnierze (broń + osoba + żołd)	jedzenie ↔ kopalnie, podatek ↔ żołd
Saraceni	woda, daktyle i handel	złoto z handlu trzema towarami	woda i daktyle ↔ kadzidło, nasycenie rynku, napady
Wikingowie	drewno, morze i wyprawy	wyprawy okrętów	ludzie i miód pitny ↔ wyprawy, zużycie drewna
Słowianie	puszcza, bartnictwo i futra	danina koszykowa (futro, miód, воск)	zdrowie lasu i dobór koszyka

Poza ekonomią — którą opisuje specyfikacja v3.6 — gra ma warstwę przestrzenną dodaną w kolejnych fazach prac: **teren w czterech klimatach i sześciu typach map**, ruch po ścieżkach i drogach, zasady zabudowy (przerwa, ciasna zabudowa, wyrównywanie wzgórz), sprawność logistyczną producentów oraz **faunę z polowaniem z łukiem**. Warstwa przestrzenna jest rozłączna z ekonomią: scenariusze akceptacyjne specyfikacji liczą się z wyłączoną logistyką i fauną i dają te same wyniki co przed jej dodaniem (rozd. §16).

1.2 Zakres dokumentu i hierarchia źródeł

Dokument opisuje **całą logikę gry takiej, jaka jest w kodzie**, oraz decyzje projektowe, które do niej doprowadziły. Powstał na podstawie czterech źródeł; przy sprzeczności obowiązuje kolejność od góry:

1. **Kod** (`Nova_Roma.html`, `tools/`, `prototyp/`) — prawda o tym, jak gra działa. Tabele i wartości oznaczone w tekście jako „z kodu” są generowane z silnika podczas budowy dokumentu, więc nie rozjeżdżają się z grą.
2. **Specyfikacja ekonomii do wdrożenia (v3.6)** — zamknięta specyfikacja czterech nacji (w tytule pliku PDF wciąż widnieje „v3.2”, treść nosi oznaczenie v3.6). To ona jest źródłem liczb ekonomii (rozd. §3–§8) i kryteriów akceptacji (aneks §A).
3. **„Ekonomia Nova Roma — wersja finalna”** — starszy dokument historyczny (wersja v3.2 trzech nacji): uzasadnia, dlaczego mechaniki wyglądają tak, a nie inaczej (naprawione błędy v2, balans, decyzje). Część liczb została później zmieniona w v3.6; różnice zebrano w rozdz. §18.
4. **Plan faz i decyzje użytkownika** — wszystko, co dodano ponad specyfikację ekonomii: teren, zabudowa, ruch, fauna, menu (rozd. §17).

WAŻNE

Konwencje. *d* = deska, *k* = kamień, *gl* = glina, *zl* = złoto, **T** = budynek wymaga narzędzia przy obsadzeniu. „Minuta” oznacza minutę czasu gry (w grze 4 s realne = 1 min przy prędkości ×1). „Pole” to jednostka planu mapy (mapa ma 48 × 48 pól). *Osobominuta* to minuta pracy jednej osoby — wspólna miara, do której sprowadzono poziomy haracz wszystkich nacji. Wartości liczbowe zapisujemy z przecinkiem dziesiętnym.

1.3 Stan projektu

Prace prowadzono w fazach; każda kończy się bramką (testy, regresja względem wzorca, commit, wersja do ręcznego sprawdzenia). Stan na dzień zbudowania dokumentu:

Tabela 1. Fazy projektu

Faza	Zakres	Stan
Etapy 1–6	ekonomia wg specyfikacji v3.6, Słowianie, Doradca, statusy budynków, boty testowe R/D/N, scenariusze akceptacyjne	✓
Prototyp grafiki	proceduralne sprite'y 88 budynków czterech nacji, ludzie, przyroda	✓
0–3	wzorzec testów, głowy smocze Wikingów, Słowianie bez głów, silnik sprite'ów w grze	✓
4	obrysy budynków w×h, render sprite'ami, Dwór 4×4	✓
5	teren v2: 4 klimaty × 6 typów map, podłoże w chunkach	✓
6, 6b, 6c	A* i piesi (cień symulacji), drogi, przerwa i kara za ciasną zabudowę, sprawność logistyczna	✓
7	wyrównywanie terenu pod budynki, wzgórza decyzyjne	✓
8	fauna (18 gatunków) i polowanie z łukiem	✓
9	menu główne RTS i Piaskownica (nacja, klimat, typ mapy), ≡ Menu w grze	✓
9B	ożywienie ludzi: fizyczna logistyka, role i czynności, karawany (aneks SE)	następna
10	integracja: legenda, spójność Doradcy, zestaw regresji w repozytorium, dostawa	planowane
11	audyt wizualny całości i poprawki	planowane

Szczegółowy stan na okładce dokumentu pochodzi z pliku `docs/meta.json`; ten rozdział aktualizujemy po każdej fazie (rozd. [SD](#)).

1.4 Struktura repozytorium

Tabela 2. Zawartość repozytorium

Ścieżka	Zawartość
<code>Nova_Roma.html</code>	gra — cały kod w jednym pliku (ok. 10 700 linii; bloki <code>Sprites</code> i <code>SPRITE_META</code> są generowane)
<code>README.md</code>	opis projektu i zasad dodanych w nowym stylu
<code>tools/</code>	testy silnika bez przeglądarki (<code>node:vm</code> + atrapy DOM), testy w Playwright, <code>headless.js</code> (harness)
<code>prototyp/</code>	źródła silnika grafiki (<code>src/*.js</code>), <code>build.js</code> , <code>build_game.js</code> (osadza sprite'y w grze), <code>grafika_prototyp.html</code> , podglądy
<code>docs/</code>	ta dokumentacja : rozdziały (<code>chapters/*.md</code>), generator PDF (<code>build.js</code>), tabele z kodu (<code>tables.js</code>), wynik <code>Nova_Roma_dokumentacja.pdf</code>

Gałąź robocza to `claude/festive-meitner-ejo4l1`, szkic pull requestu #1 do `main`. Pełne polecenia budowy i testów — aneks [SC](#).

1.5 Uruchamianie i parametry adresu

Grę otwiera się z pliku (`file://`) w nowoczesnej przeglądarce; pokazuje się **menu główne** (rozd. [§14.1](#)), z którego Piaskownica prowadzi do wyboru nacji, klimatu i typu mapy. Po Starcie następuje wypiek grafiki (1–3 s), wypiek podłoża mapy i gra. Parametry adresu (wszystkie opcjonalne):

Tabela 3. Parametry adresu (query string)

Parametr	Działanie
?menu=0	omija menu główne i pokazuje dawny ekran wyboru nacji czterema kartami (kontrakt dla testów przeglądarkowych)
?play=nacja	omija menu i ekran wyboru nacji: <code>franks</code> , <code>saracens</code> , <code>vikings</code> , <code>slavs</code> (ta sama ścieżka <code>UI.startGame</code> co wybór kartą)
&climate=	klimat: <code>temperate</code> , <code>eastern</code> , <code>snow</code> , <code>desert</code> (domyślnie klimat nacji)
&map=	typ mapy: <code>coast</code> , <code>plain</code> , <code>river</code> , <code>lakes</code> , <code>mountains</code> , <code>wetlands</code> (Wikingowie zawsze <code>coast</code>)
&seed=N	ziarno mapy (liczba całkowita); bez niego stałe ziarno nacji (777 / 888 / 999 / 1111)
?q=1 lub ?q=2	jakość wypieku sprite'ów (1 = łżejsza, 2 = ostra; domyślnie 2 na ekranach HiDPI)
?classic=1	bez sprite'ów — dawne procedury rysowania <code>Gfx.ART</code> (zapas)
?fauna=0	wyłącza faunę i polowanie (domyślnie włączone w grze)
?logistics=0	wyłącza sprawność logistyczną (domyślnie włączona w grze)

Przycisk „Nowa mapa” w menu w grze i „Menu główne” działają przez zmianę parametrów adresu (rozdz. §14.1.2).

UWAGA

Parametry `?play=` oraz `?menu=0` są kontraktem dla testów przeglądarkowych — nie wolno zmieniać ich znaczenia bez aktualizacji `tools/browser_*.js`.

2. Architektura programu

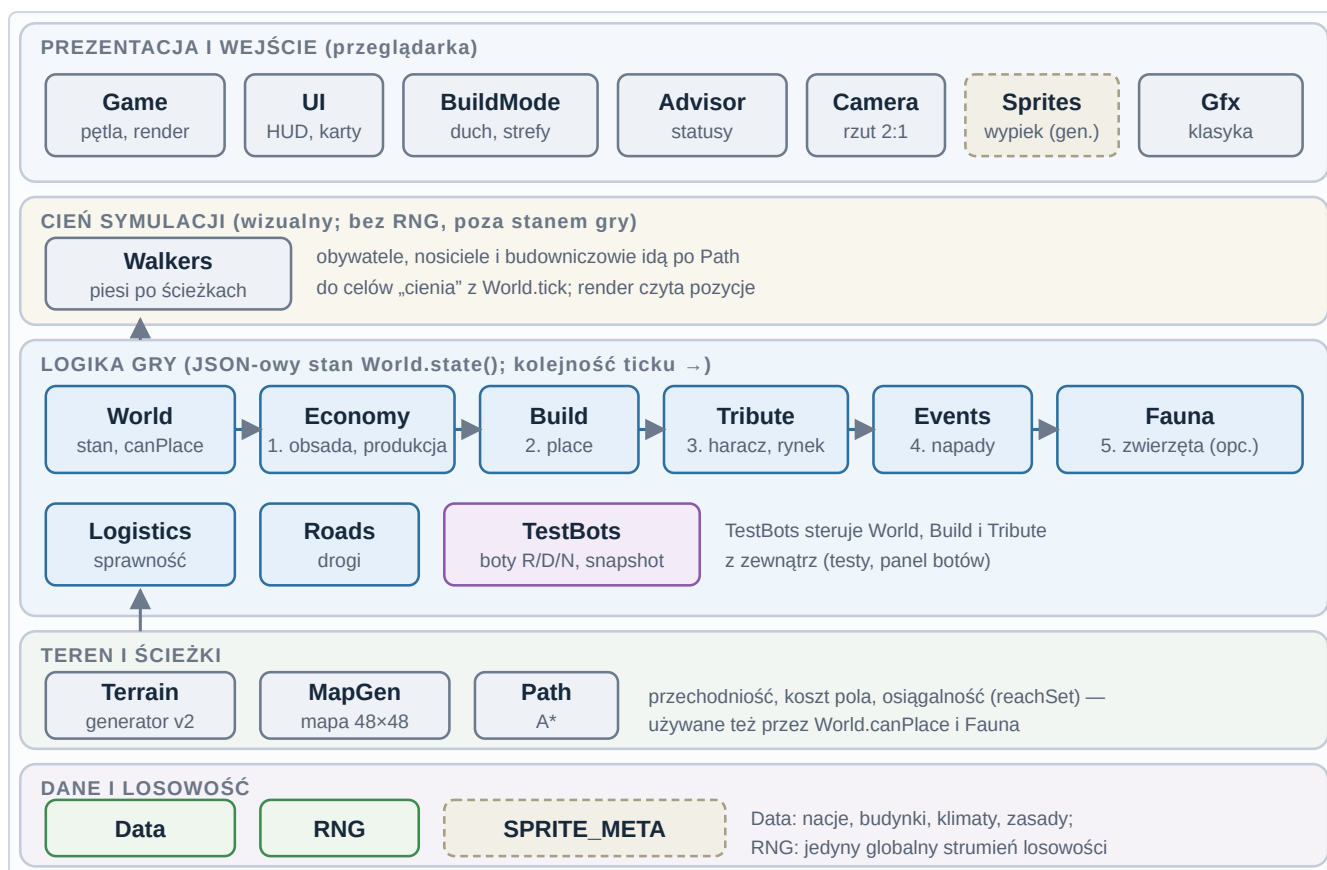
2.1 Jeden plik, wiele modułów

Cały kod gry mieści się w jednym znaczniku `<script>` pliku `Nova_Roma.html`. Moduły to wyrażenia IIFE (`const Economy = (() => { ... return { ... }; })();`) ułożone w kolejności zależności — bez bundlera i bez `import`. Każdy moduł eksportuje niewielkie API, a pozostałe moduły wołają je przez nazwę (`World.state()`, `Economy.tick(dt)` ...). Dwa bloki — `SPRITE_META` (katalog obrysów budynków) i `Sprites` (silnik grafiki) — są generowane z katalogu `prototyp/` skryptem `node prototyp/build_game.js` i nie wolno ich edytować ręcznie (rozd. §15).

Tabela 4. Moduły w `Nova_Roma.html` (kolejność w pliku; zakresy linii liczone z kodu)

Moduł	Linie	Dł.	Zadanie	API
RNG	353–5150	4798	Globalny generator <code>mulberry32</code> (<code>seed</code> , <code>next</code> , <code>range</code> , <code>int</code> , <code>pick</code>) — jedyne źródło losowości logiki gry	5
SPRITE_META	368–371	4	Katalog budynków z prototypu: obrys <code>w×h</code> , wejście, kotwice dymu, rozmiar <code>sprite'a</code> (blok generowany)	—
Sprites	373–5149	4777	Silnik grafiki proceduralnej: wypiek <code>sprite'ów</code> budynków, przyrody, ludzi i zwierząt, chunki podłoża (blok generowany z <code>prototyp/src</code>)	27
Data	5151–5360	210	Dane: nacje, budynki (<code>koszty</code> , <code>wejścia</code> i <code>wyjścia</code>), klimaty, typy map, zasady zabudowy, fauna, danina	17
Camera	5361–5392	32	Rzut dimetryczny 2:1, przesuwanie i zoom, przeliczanie świat ↔ ekran	13
Terrain	5393–5815	423	Generator terenu v2 (<code>woda</code> , <code>wysokości</code> , <code>klify</code> , <code>cechy klimatu</code> , <code>łączność</code> , <code>walidacja</code>), predykaty <code>przechodności</code> i <code>koszt pola</code>	21
MapGen	5816–5911	96	Mapa 48×48: stary generator (<code>morze</code> , <code>skały</code> , <code>drzewa</code> , <code>złoża</code>) + nakładka <code>Terrain.apply</code> , meta mapy	4
Gfx	5912–7263	1352	Grafika klasyczna (<code>zapas</code> , <code>?classic=1</code>) i miniatury kart nacji	7
Path	7264–7392	129	A* po polach (8 kierunków, bez <code>ścínania rogów</code>), pola <code>odległości</code> , najbliższe osiągalne pole; bez RNG	7
World	7393–7678	286	Stan gry, <code>init</code> , <code>canPlace</code> i <code>build</code> , <code>drzewa</code> jako <code>zasób</code> , kolejność <code>ticku</code> , <code>obywatele</code> i <code>nosiciele</code> (<code>cień symulacji</code>)	19
Economy	7679–7992	314	Ekonomia przepływowa: <code>obsada</code> , <code>produkcja</code> , <code>jedzenie</code> , <code>ludność</code> , <code>popularność</code> , <code>narzędzia</code> , <code>magazyny</code>	23
Build	7993–8132	140	Plac budowy, <code>budowniczo</code> , etapy 0–2, <code>anulowanie</code>	9
Tribute	8133–8382	250	Haracz seniora: <code>poziomy</code> , <code>terminy</code> , <code>żołnierze</code> / <code>złoto</code> / <code>wyprawy</code> / <code>danina</code> , <code>rynek Targu</code>	15
Events	8383–8485	103	Napady na karawanę, <code>kryzysy</code> (<code>zaraza</code> , <code>pożar</code> , <code>krach cen</code> , <code>wylesienie</code>) wg <code>harmonogramu</code>	12
TestBots	8486–8682	197	Boty testowe <code>R/D/N</code> , <code>przepisy nacji</code> , <code>snapshot</code> i <code>restore</code> , <code>runSync</code> , panel testowy	9
Advisor	8683–8888	206	Doradca: <code>statusy budynków</code> , <code>ostrzeżenia</code> , <code>plan haraczu</code> (tylko <code>czyta stan</code>)	8
Roads	8889–8917	29	Drogi: <code>wytyczanie</code> , <code>koszt</code> , <code>rozbiórka</code>	7
Logistics	8918–8950	33	Sprawność logistyczna <code>producentów</code> (<code>odległość do Składu / Dworu</code>)	3
BuildMode	8951–9125	175	Tryb budowy: <code>duch budynku</code> , <code>strefy zakazu</code> i <code>kary</code> , <code>narzędzia dróg</code> , <code>Klik</code> = <code>plac budowy</code>	9
UI	9126–9710	585	HUD, pasek budowy, <code>karty</code> i <code>podpowiedzi</code> , <code>Doradca</code> , <code>ekran wyboru nacji</code> , <code>UI.startGame</code>	9
Fauna	9711–9997	287	Zwierzęta i <code>polowanie</code> : <code>populacja</code> , <code>ruch</code> , <code>legowiska</code> , <code>wyprawy myśliwego</code> , <code>plon</code> zależny od <code>zwierzyny</code>	15
Walkers	9998–10130	133	Widoczni piesi: <code>ścieżki obywateli</code> , <code>nosicieli</code> i <code>budowniczych</code> (<code>cień symulacji</code>)	5
Game	10131–10684	554	Pętla gry, <code>render</code> (<code>teren</code> , <code>obiekty</code> , <code>zwierzęta</code> , <code>strzały</code> , <code>efekty</code>), <code>wejście</code> : <code>mysz</code> , <code>klawiatura</code> , <code>dotyk</code>	3

Warstwy modułów pokazuje rysunek poniżej. Najważniejsza reguła: **logika gry nie zna renderu** — `World`, `Economy`, `Build`, `Tribute`, `Events` i `Fauna` operują wyłącznie na stanie `World.state()` i mogą działać bez przeglądarki (testy `headless` ładują cały plik w `node:vm` z `atrapami DOM`). `Render` (`Game`), interfejs (`UI`, `BuildMode`, `Advisor`) i `ruch widocznych postaci` (`Walkers`) tylko czytają stan.



Rysunek 1. Warstwy modułów. Warstwa wyższa woła niższą; logika gry nie zna renderu ani Walkers (render tylko czyta stan). Zielone: bloki generowane z prototypu.

2.2 Zależności między modułami

Macierz poniżej liczona jest z kodu: dla każdej pary modułów zlicza odwołania typu `Moduł` w tekście (po usunięciu komentarzy). Wiersz = moduł używający, kolumna = moduł używany. Pozwala szybko zobaczyć, co trzeba przetestować po zmianie danego modułu (kolumna) i czego dany moduł potrzebuje (wiersz).

Tabela 5. Macierz zależności: liczba odwołań **Moduł** w kodzie wiersza do modułu kolumny (komentarze pominięte)

	RNG	SPRITE_META	Sprites	Data	Camera	Terrain	MapGen	Gfx	Path	World	Economy	Build	Tribute	Events	TestBots	Advisor	Roads	Logistics	BuildMode	UI	Fauna	Walkers	Game
RNG	.	1										10											
SPRITE_META		.																					
Sprites			.									10											
Data		2		.																			
Camera					.		1																
Terrain				4		.																	
MapGen	9			4		1	.																
Gfx					2			.															
Path						3			.														
World	14			11		2	8			.	3	1	1	1							1		
Economy				9						6	.		2					2			2		
Build							1			14		.	2										
Tribute				7						2			.										
Events	1			1						6				.									
TestBots	1			3			2			11	4	1	1		.						1		
Advisor				10						5	2		4			.					3		
Roads						1	3		1	4							.						
Logistics				2					1		1							.				2	
BuildMode			5	5	21	1	4	1		10		4					5		.	11			
UI			6	38	7		1	4		16	4	1	7	1		6	1		5	.	1		2
Fauna				4		3			2	3										.		1	
Walkers		2		1		2			10	5		1										.	
Game		1	34	8	62	2	2	14		13		2			1	1			9	6	6	2	.

Tabela 6. Najczęściej używane moduły każdego modułu (liczba odwołań) i liczba modułów, które go używają

Moduł	Używa (najczęstsze)	Używany przez
RNG	Build (10), SPRITE_META (1)	4
SPRITE_META	—	4
Sprites	Build (10)	3
Data	SPRITE_META (2)	14
Camera	MapGen (1)	4
Terrain	Data (4)	8
MapGen	RNG (9), Data (4), Terrain (1)	8
Gfx	Camera (2)	3
Path	Terrain (3)	4
World	RNG (14), Data (11), MapGen (8), Economy (3), Terrain (2), Build (1), Tribute (1), Events (1), Fauna (1)	12
Economy	Data (9), World (6), Tribute (2), Logistics (2), Fauna (2)	5
Build	World (14), Tribute (2), MapGen (1)	8
Tribute	Data (7), World (2)	6
Events	World (6), RNG (1), Data (1)	2
TestBots	World (11), Economy (4), Data (3), MapGen (2), RNG (1), Build (1), Tribute (1), Fauna (1)	1
Advisor	Data (10), World (5), Tribute (4), Fauna (3), Economy (2)	2
Roads	World (4), MapGen (3), Terrain (1), Path (1)	2
Logistics	Data (2), Walkers (2), Path (1), Economy (1)	1
BuildMode	Camera (21), UI (11), World (10), Sprites (5), Data (5), Roads (5), MapGen (4), Build (4), Terrain (1)	2
UI	Data (38), World (16), Camera (7), Tribute (7), Sprites (6), Advisor (6), BuildMode (5), Gfx (4), Economy (4)	2
Fauna	Data (4), Terrain (3), World (3), Path (2), Walkers (1)	6
Walkers	Path (10), World (5), SPRITE_META (2), Terrain (2), Data (1), Build (1)	3
Game	Camera (62), Sprites (34), Gfx (14), World (13), BuildMode (9), Data (8), UI (6), Fauna (6), Terrain (2)	1

W KODZIE

Zależności „w dół” są jawne: Economy używa World, Data, Fauna.gameMult i Logistics.effOf; World woła Economy, Build, Tribute, Events i Fauna tylko w World.tick. Jedyna zależność „w górę” to Economy → Fauna/Logistics, ale tylko przez wąskie funkcje czytające stan (mnożniki plonu), które przy wyłączonych przełącznikach zwracają 1.

2.3 Stan gry

Cały stan gry mieści się w jednym obiekcie `World.state()`. Jest **JSON-owy** (bez referencji obiekt → obiekt; powiązania przez `uid`), dlatego `TestBots.snapshot()` i `restore()` klonują go przez `JSON.parse(JSON.stringify(...))` — na tym opiera się „odtwarzalność” testów (rozdz. §16). Pola robocze budynków (`st`, `eff`, `logi`, `huntAcc` ...) są częścią budynku i przeżywają snapshot.

Tabela 7. Pola stanu gry (`World.state()`); stan jest JSON-owy, więc `TestBots.snapshot / restore` klonuje go przez JSON

Pole	Opis
<code>factionId</code>	id nacji: franks, saracens, vikings, slavs
<code>map</code>	mapa: size 48, tiles[2304], meta {climate, type, seed, valid, why, stats}, dirty (pola do odświeżenia chunków), roadRev
<code>buildings</code>	gotowe budynki {id, x, y, w, h, uid}; pola robocze: st (status), eff, logi, huntAcc, awayUntil, carrierT
<code>citizens</code>	obywatele „cienia symulacji” (losowania RNG logiki); widoczne postacie rysuje Walkers
<code>time</code>	czas gry w minutach
<code>speed</code>	mnożnik prędkości 0 / 1 / 2 / 4 (0 = pauza)
<code>res</code>	magazyn: towary (Economy.LIMIT), złoto, narzędzia, popularity
<code>nextId</code>	licznik uid budynków i placów
<code>pop</code>	ludność (liczba rzeczywista; w obliczeniach Math.floor)
<code>controls</code>	{ rations: 0,5 / 1 / 1,5 / 2, tax: 0–3 }
<code>starveTimer</code>	minuty ciągłego braku jedzenia (≥ 5 → ginie 1 osoba na minutę)
<code>immigrantTimer</code>	akumulator czasu do kolejnego imigranta
<code>tooled</code>	uid → true: budynek już dostał narzędzie
<code>staffed</code>	uid → true: budynek obsadzony w bieżącym ticku
<code>saplings</code>	sadzonki { t: czas dojrzewania }
<code>treeFrac</code>	ułamek ściętych drzew do rozliczenia (Chata drwala)
<code>plantFrac</code>	ułamek posadzonych drzew do rozliczenia (Leśniczówka)
<code>welfare</code>	Dobrobyt z poprzedniego ticku: liczba zasilonych budynków temple / bathhouse / sauna
<code>foodRep</code>	ostatni bilans jedzenia: potential, people, builders, buildings, consumption, pmax
<code>workersStaffed</code>	liczba obsadzonych budynków
<code>workersNeeded</code>	liczba budynków potrzebujących pracownika
<code>waitingForTool</code>	czy jakiś budynek czeka na narzędzie
<code>stats</code>	liczniki raportowe: toolsBought, toolsForged, planksSpent, starveMin
<code>settings</code>	przełączniki scenariusza: raids, voyageGuard, shoreSpots, logistics, fauna
<code>animals</code>	zwierzęta (lista JSON-owa, Fauna 8)
<code>fauna</code>	stan modułu Fauna: rs (PRNG), nid, bt, dens, hunts, kills (null przy wyłączonej faunie)
<code>depositLeft</code>	opcjonalne skończone złoża, np. { clay: 60 }
<code>sites</code>	plac budowy {id, x, y, w, h, uid, need, got, work, workNeed, levelLeft, crowd, assigned}
<code>builders</code>	ustawienie liczby budowniczych (2–5)
<code>buildersActive</code>	budowniczowie faktycznie rekrutowani w ticku
<code>buildersWorking</code>	budowniczowie przydzieleni do placów (jedzą 1,5 racji)
<code>carriers</code>	nosiciele „cienia symulacji” (wizualni; ekonomia jest nominalna)
<code>tribute</code>	bieżący haracz: preset, level, firstTerm, terms, met, missed, delivered, byGood, log...
<code>marketPrices</code>	ceny Targu (kadzidło, tkanina, ceramika)

Pole	Opis
<code>voyageMode</code>	tryb wypraw Wikingów: normal mixed
<code>crewAway</code>	załoga na wyprawach (osoby poza pracą)
<code>voyages</code>	trwające wyprawy {until, crew, great, ships}
<code>voyageStats</code>	wykonane wyprawy: done (jednostki haraczu), trips
<code>raids</code>	napady na karawanę: count, lost, next (czas kolejnego sprawdzenia)
<code>crisis</code>	harmonogram kryzysów (null = wyłączony) lub { idx }
<code>eventMsgs</code>	komunikaty zdarzeń { text, time }
<code>keep</code>	wskaźnik na budynek Dworu (po restore odtwarzany z listy budynków)

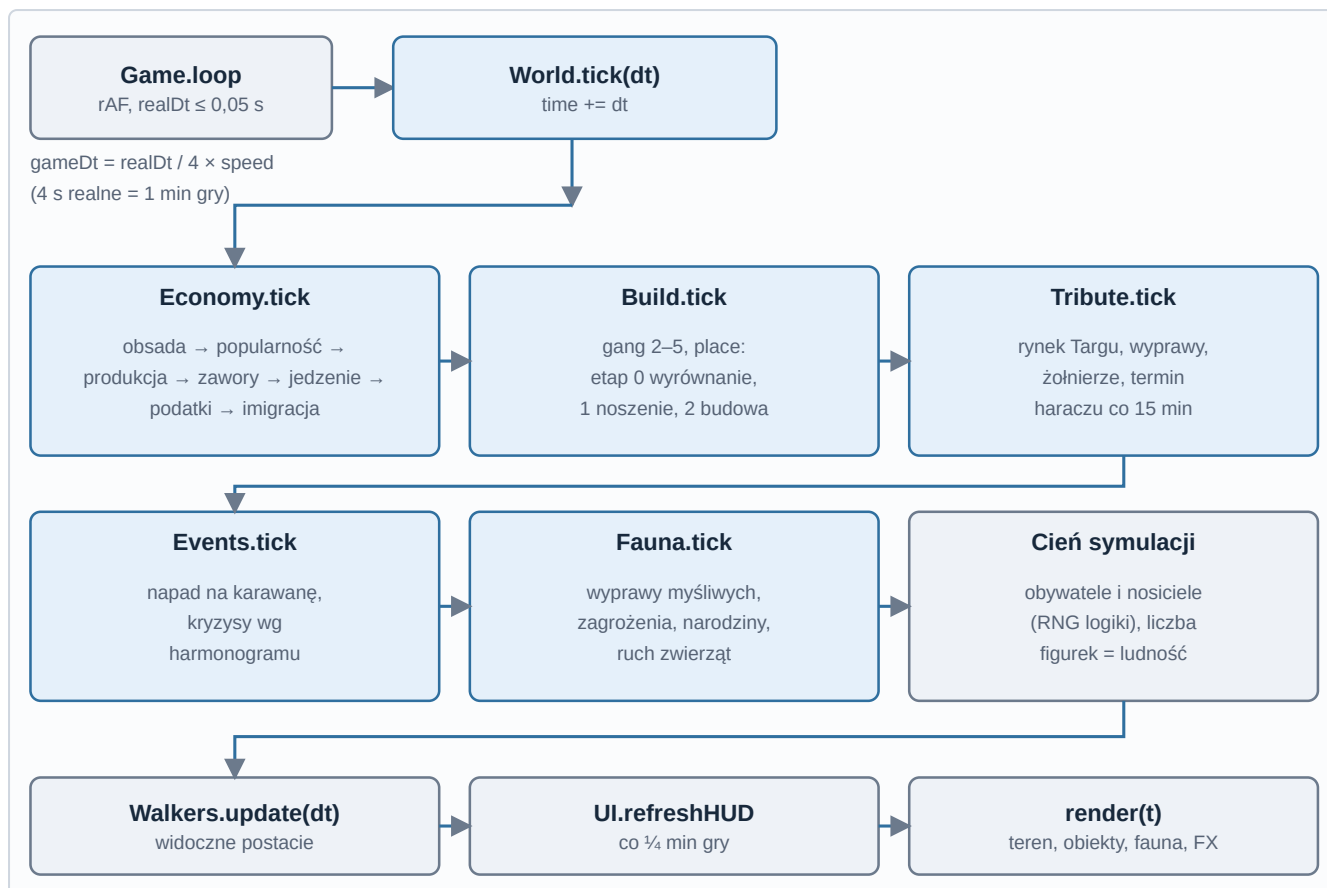
Kafel mapy (`map.tiles[i]`) ma pola: `h` (0 woda / 1 ląd / 2 skała), `e` (wysokość 1–4; woda 0), `wk` (rodzaj wody: `sea`, `lake`, `river`, `ford`, `ice`), `k` (cecha: `bog`, `dune`, `quick`, `oasis`, `cliff`, `drift`), `trees` (0–3), `deposit` (`stone`, `iron`, `coal`, `clay`, `peat`), `occup` (uid budynku lub placu), `road` (0/1) oraz `pad` (pole zarezerwowane przez generator — drzewa tu nie odrastają).

2.4 Pętla gry i kolejność ticku

`Game.loop` jest wołana z `requestAnimationFrame`. Czas rzeczywisty (ograniczony do 0,05 s na klatkę) przeliczany jest na czas gry: $\text{gameDt} = \text{realDt} / 4 \times \text{speed}$ — **4 s realne = 1 minuta gry** przy prędkości $\times 1$ (przyciski $\times 1$ / $\times 2$ / $\times 4$; $\times 0$ to pauza). Dla każdej klatki:

1. `World.tick(gameDt)` — logika (kolejność poniżej),
2. `Walkers.update(gameDt)` — ruch widocznych postaci po ścieżkach,
3. `UI.refreshHUD()` — co ¼ minuty gry,
4. `render(t)` — rysowanie.

W `World.tick` moduły wołane są **zawsze w tej samej kolejności**: `Economy` → `Build` → `Tribute` → `Events` → `Fauna`, a potem aktualizowany jest „cień symulacji” (obywatele i nosiciele). Kolejność ma znaczenie: `Build` zna obsadę wyliczoną przez `Economy`, a `Tribute` korzysta z ludności i zapasów po bieżącej produkcji.



Rysunek 2. Kolejność ticku: dwa tory — logika (World.tick, w minutach gry) i prezentacja (Walkers, HUD, render). W testach botów World.tick wołany jest bezpośrednio krokiem 0,1 min.

W KODZIE

Testy botów wołają `World.tick(0.1)` w pętli (krok 0,1 min — tyle samo, ile w symulatorze specyfikacji). W grze `dt` jest zmienny; model przepływowy jest od kroku niezależny (akumulatory `treeFrac`, `plantFrac`, `immigrantTimer` itd. zbierają ułamki).

2.5 Losowość i determinizm

Gra ma **jeden globalny generator** RNG (mulberry32) i kilka lokalnych PRNG ze stanem w danych. Rozdział odpowiedzialności jest sztywny, bo każdy dodatkowy `RNG.next()` w logice zmieniłby przebieg gier w testach specyfikacji:

Tabela 8. Kto używa losowości

Moduł	Źródło losowości	Uwagi
MapGen.generate	globalny RNG (zasiewany ziarnem mapy)	stary generator: morze, skały, drzewa, złoża; zasiewa strumień — <code>World.init</code> zasiewa go ponownie
Terrain.apply	własny PRNG <code>Terrain.prng(ziarno)</code>	nie dotyczy globalnego RNG (test <code>terrain.js</code>)
<code>World.takeTrees</code> , <code>World.addTree</code> , obywatele i nosiciele (cień)	globalny RNG	to jest „przeplot RNG” logiki, który testy specyfikacji zakładają
Events (napady na karawanę)	globalny RNG	jedyne losowe zdarzenie ekonomii (Saraceni)
Path, Roads, Logistics, Walkers	brak	deterministyczne z natury

Moduł	Źródło losowości	Uwagi
Fauna	własne mulberry32, stan w <code>state.fauna.rs</code>	liczba całkowita w JSON; zero wywołań RNG (test <code>fauna.js</code>)
Render (Game)	własny <code>fxRand</code> , ziarna per obiekt	wygląd, nie logika
Prototyp grafiki (Sprites)	mulberry32 per tekstura / sprite	deterministyczne — te same wypieki za każdym razem

Skutek: ta sama nacja, klimat, typ i ziarno dają tę samą mapę; ta sama kolejność decyzji bota daje ten sam wynik; włączenie fauny, logistyki i dróg nie zmienia strumienia RNG, więc scenariusze specyfikacji pozostają bez zmian przy wyłączonych przełącznikach.

2.6 Cień symulacji

Obywatele i nosiciele w `World.tick` poruszają się po prostych liniach i losują cele z globalnego RNG. Chcemy jednak, by na ekranie ludzie chodzili po ścieżkach (omijali wodę, klify i budynki), nie zmieniając tego strumienia. Rozwiązanie to **cień symulacji**: logika zostaje jak była („cień”), a moduł `Walkers` utrzymuje osobne, widoczne postacie, które dążą do celów cienia po trasach `Path` i same niczego nie losują. Stan `Walkers` żyje **poza stanem gry** (nie trafia do snapshotu), a testy sprawdzają, że stan logiki z `Walkers` i bez niego jest identyczny. Ten sam wzorec zastosowano dla pozycji myśliwego (czysta funkcja czasu gry) i animacji w renderze.

2.7 Przełączniki scenariusza

`state.settings` zawiera przełączniki, które w testach specyfikacji mają wartości „neutralne”, a w grze ustawia je `UI.startGame`:

Tabela 9. Przełączniki w `state.settings`

Ustawienie	W testach	W grze	Znaczenie
<code>raids</code>	<code>true</code>	<code>true</code> (przycisk  , tylko Saraceni)	napady na karawanę (rozdz. §9)
<code>voyageGuard</code>	<code>true</code>	<code>true</code> (przycisk  Zapas załogi)	wysyłka Wikingów zostawia ≥ 90% robotników
<code>shoreSpots</code>	6	6	maks. liczba Chat rybaka na brzegu (parametr generatora map)
<code>logistics</code>	<code>false</code>	<code>true</code> (<code>?logistics=0</code> wyłącza; przycisk )	sprawność logistyczna producentów (rozdz. §12)
<code>fauna</code>	<code>false</code>	<code>true</code> (<code>?fauna=0</code> wyłącza)	zwierzęta i plon Chaty myśliwego zależny od zwierzyny (rozdz. §13)

Wzorec jest jednolity: stan domyślny „neutralny” w literale stanu i w `World.init` (żeby testy ze specyfikacji nie zmieniły ani jednej liczby), włączenie przez `UI.startGame`, a funkcja pomocnicza (`Logistics.off`, `Fauna.gameMult`) zwraca 1 przy wyłączeniu.

2.8 Wejście do gry: `UI.startGame`

Jedynym wejściem do rozgrywki jest `UI.startGame(fid, opts)` (jednorazowe: flaga `gameStarted`). Używają go: karta nacji na ekranie wyboru, skrót `?play=`, testy przeglądawkowe i — w fazie 9 — Piaskownica. Kolejność kroków: wypiek sprite'ów wybranej nacji i klimatu (`Sprites.bake`) → `World.init(fid, opts)` (mapa, zasoby, Dwór, 14 obywateli) → ustawienie kamery na Dwór → budowa paska budowy → wypiek podłoża mapy w chunkach (`Game.initGround`) → podpięcie przycisków HUD i przełączników → `Fauna.enable` → `Game.start()` → `window.__gameReady = true` (znacznik dla testów).

3. Ekonomia przepływowa

Moduł `Economy` implementuje specyfikację v3.6 jako **model przepływowy**: obsadzony budynek produkuje stałą ilość towaru na minutę gry, o ile ma zapłacone wejścia i miejsce w magazynie. Brak wejścia albo pełny magazyn zatrzymuje budynek **bez żadnej kary** (i bez zużycia wejść). Model nie zna tras ani wozów — robotnik „fizycznie” nosi towar tylko na ekranie (cień symulacji), a wydajności są nominalne; odległość uwzględnia opcjonalna sprawność logistyczna (rozdz. §12).

3.1 Zasady nadrzędne

1. **Produkcja powstaje tylko z zapłaconych wejść**, a nadwyżka bez popytu zatrzymuje budynek (limity magazynu).
2. **Jeden pracownik na budynek**. Chata, Skład, Przysań, Okręt i Dwór nie mają pracownika. Tempa w tabelach to szt./min na jeden obsadzony budynek.
3. **Premie nigdy nie są warunkiem przetrwania**: popularność, różnorodność jedzenia i Dobrobyt tylko przyspieszają grę (żaden budynek ani łańcuch nie wymaga Dobrobytu do działania).
4. **Zawory bezpieczeństwa zamiast blokad**: pierwsza Chata drwała jest darmowa, Dwór ma piłę zamkową, narzędzia można dokupić na Targu lub dorobić w kuźni zamkowej, a ludność nigdy nie spada poniżej 10 osób. Żadna sekwencja budowy nie może zablokować gry bez wyjścia (kryteria w aneksie §A).
5. **Żaden towar nie zalega bez odbiorcy, którego da się zbudować** (niezmiennik zaprojektowany w specyfikacji; zalegające towary handlowe Saracenów trafiają od razu na Targ).

3.2 Jednostki i czas

Czas gry liczy się w minutach (`state.time`). `Economy.tick(dt)` przyjmuje `dt` w minutach; przy `dt ≤ 0` nie robi nic. Wszystkie tempa są „na minutę”, więc wynik dla kroku `dt` to `tempo × dt`. Ułamkowe części sztuk zbierają akumulatory (`treeFrac`, `plantFrac`, `immigrantTimer`, `starveTimer`).

3.3 Zasoby i magazyny

Zasoby gry to towary z tabeli limitów oraz trzy zasoby specjalne: **złoto** (`res.złoto`, bez limitu), **narzędzia** (`res.narzędzia`, limit 10) i **popularność** (`res.popularity`, 0–100). Każdy towar ma twardy górny limit w całej osadzie:

Tabela 10. Limity magazynowe towarów w całej osadzie (`Economy.LIMIT`)

Towar	Limit	Towar	Limit	Towar	Limit
pień	100	daktyl	150	miód pitny	60
deski	250	ryby	150	bawełna	40
kamień	300	ruda	40	tkanina	60
glina	300	węgiel	40	ceramika	60
zboże	40	sztabki	40	plaster	40
mąka	40	narzędzia	10	futro surowe	40
woda	40	broń	60	futro wyprawione	150
chleb	150	żywica	40	miód	150
mięso	150	kadzidło	40	wosk	150
ser	150	winogrona	40		
owoce	150	wino	60		

Faktyczną pojemność towaru w danej chwili wyznacza `Economy.cap0f` — wzór ze specyfikacji („Składy i logistyka”):

```
pojemność(towar) = min( limit(towar), 6 × (czynni producenci towaru) + 20 (komora Dworu) + 80 × (liczba Składów) )
```

Producent odkłada towar „u siebie” (6 szt. każdego wytwarzanego towaru); dopóki jest miejsce, produkcja idzie. Gdy lokalny zapas jest pełny, a Składu nie ma lub jest pełny, producent **stoi** (status **Z**) — towar nie ginie. Skład (+80 szt. każdego towaru, bez pracownika i bez narzędzia) jest neutralny w zwykłej grze, a niezbędny przy dużych jednorazowych zadaniach (np. 40 żołnierzy naraz, 12 sztuk miodu i broni na wielką wyprawę, daniny Słowian powyżej poziomu 6). „Czynny producent” to budynek **obsadzony**, którego dowolne wyjście jest danym towarem.

PUŁAPKA

Pojemność liczy się per towar, a nie per budynek: np. trzy tartaki i jeden Skład dają pojemność desek $\min(250, 18 + 20 + 80) = 118$. Dlatego bez Składu deski potrafią zatrzymać Tartak, gdy gracz nie wydaje ich dość szybko (Doradca ostrzega, gdy ≥ 2 budynki stoją z pełnym magazynem).

3.4 Obsada pracowników

W każdym ticku `Economy.staff` wyznacza od nowa, które budynki są obsadzone (`state.staffed[uid]`):

1. Liczba wolnych ludzi = `floor(ludność)` – budowniczy w gangu – załoga na wyprawie.
2. Budynki są sortowane: **najpierw budynki żywności** w kolejności listy priorytetów, potem reszta w kolejności budowy.
3. Dla każdego budynku z pracownikiem: gdy brakuje wolnych ludzi — status **P** (brak pracownika); gdy budynek wymaga narzędzia (T) i jeszcze go nie dostał — zużywa 1 narzędzie z magazynu (`tooled[uid] = true`, jednorazowo na budynek), a gdy narzędzi brak — status **T** i budynek czeka.
4. Obsadzony budynek zmniejsza pulę wolnych ludzi o 1.

Tabela 11. Priorytet obsady, gdy brakuje ludzi (`Economy.FOOD_PRIORITY`): budynki żywności pierwsze, reszta w kolejności budowy

Kolejność	Budynek
1	Chata myśliwego
2	Mleczarnia
3	Sad
4	Chata zbieracza
5	Chata nosiwody
6	Qanat
7	Farma
8	Pole stałe
9	Młyn
10	Kaszarnia
11	Piekarnia
12	Gaj daktylowy
13	Chata rybaka

Obsada jest **dyskretna** (budynek jest obsadzony albo nie), a nie proporcjonalna — to implementacyjna realizacja zasady ze specyfikacji „gdy ludzi brakuje, najpierw obsadzane są budynki żywności”. Narzędzie raz wydane „przywiązuje się” do budynku (po wyburzeniu lub pożarze nie wraca).

3.5 Algorytm produkcji budynku

Dla każdego obsadzonego producenta (w kolejności z tabeli poniżej) `Economy.tick` wykonuje:

- Budynki jedzące z nadwyżki** (Kopalnia żelaza, Kopalnia węgla, Wytwórnia kadzidła, Kopalnia darniowa): jeśli zapas jedzenia jest mniejszy niż 5 minut zużycia ludności, budynek stoi (status **J**) — ludność ma pierwszeństwo.
- Świątynia** (Kościół / Meczet / Kapliczka) nie produkuje; spala $0,3 \times dt$ dobra nacji i — jeśli jest z czego — liczy się jako zasilona do Dobrobytu (rozdz. §4).
- Wyczerpane złoża** (tylko gdy ustawiono skończone złoża): status **X**.
- Wejścia.** Dla każdego wejścia `g` liczony jest ułamek $\min(1, \text{zapas } g / \text{popyt } g)$, gdzie popyt to suma zapotrzebowania **wszystkich obsadzonych odbiorców** tego towaru w bieżącym ticku — gdy kilku odbiorców dzieli niedobór (pnie: Tartak, Wypalarka, Garncarnia, Bania; węgiel: Huta i Zbrojownia), każdy dostaje ten sam ułamek. Najmniejszy ułamek wejść daje `frac`.
- Drwa!** `frac` ogranicza liczba drzew na mapie ($\text{drzewa} / (\text{tempo} \times dt)$).
- Mnożniki lasu i logistyki:** `frac` $\times=$ `forestMult` $\times$ `Logistics.eff0f`. `forestMult` to $\min(1, \text{drzewa} / \text{próg lasu})$ dla budynków leśnych (dziś Słowianie), a dla Chaty myśliwego `Fauna.gameMult` (zwierzyna w łowisku; Chata myśliwego nie zależy od lasu); `eff0f` to sprawność logistyczna (1 przy wyłączonej opcji).
- Kara głodu:** `hungerK` = $0,75$ dla producentów, którzy nie wytwarzają jedzenia, gdy trwa głód ≥ 5 min (producenci żywności pracują z pełną mocą), w przeciwnym razie 1.
- Pełny magazyn:** dla każdego wyjścia towaru z listy limitów `frac` jest ograniczany tak, by produkcja nie przekroczyła `pojemność - zapas`; wtedy status **Z** i wejścia **nie** są zużywane.
- Zużycie wejść: `zapas -= wejście \times frac \times dt`.
- Wyjścia: `zapas += tempo \times skala \times frac \times popMult \times hungerK \times dt` (z ograniczeniem do wolnego miejsca). Leśniczówka zamiast towaru dodaje do `plantFrac`; Chata drwala zdejmuję z mapy tyle drzew, ile zrobiła pni; Chata myśliwego zwiększa licznik `huntAcc` (rozdz. §13).

Zbiorczo, wydajność obsadzonego budynku wynosi:

```
wyjście/min = tempo \times skala_Wikingów \times frac \times popMult \times hungerK
frac         = min( 1, ułamek_wejść, drzewa_dla_drwala, wolne_miejsce_w_magazynie ) \times forestMult
\logistyka
popMult      = clamp( 1 + 0,004 \times (popularność - 50), 0,8; 1,2 )
```

`popMult` jest liczony raz na tick z popularności bieżącego ticku; `b.eff = frac \times popMult \times hungerK` to liczba pokazywana w interfejsie jako „Pracuje — N% wydajności”.

W KODZIE

Zużycie wejść **nie** jest mnożone przez `popMult` — wyższa popularność daje więcej towaru z tych samych wejść (premii), a niższa mniej. Skala Wikingów (`Economy.VIK_RATE`) mnoży zarówno wejścia, jak i wyjścia: Chata drwala 1,3, Leśniczówka 2,0, Tartak 1,3 (czyli 1,3 pnia $\rightarrow$ 2,6 deski).

3.5.1 Kolejność przetwarzania

Kolejność ma znaczenie tylko tam, gdzie dwa budynki sięgają po ten sam towar w jednym ticku. Producenci są przetwarzani w poniższej kolejności („woda i drewno $\rightarrow$ surowce $\rightarrow$ przetwórstwo $\rightarrow$ Dobrobyt”); budynki spoza listy idą na końcu:

Tabela 12. Kolejność przetwarzania producentów w ticku (`Economy.PROD_ORDER`; budynki spoza listy idą na końcu)

#	Budynek	id	#	Budynek	id
1	Qanat	qanat	21	Barć	bartnik
2	Chata nosiwody	watercarrier	22	Chata łowcy futer	trapper
3	Chata drwala	woodcutter	23	Wypalarka węgla	charburner

#	Budynek	id	#	Budynek	id
4	Leśniczówka	forester	24	Tartak	sawmill
5	Chata rybaka	fisherhut	25	Młyn	mill
6	Kopalnia darniowa	peatmine	26	Kaszarnia	kasha
7	Glinianka	claypit	27	Winiarz	winery
8	Kamieniołom	quarry	28	Huta	smelter
9	Kopalnia żelaza	ironmine	29	Piekarnia	bakery
10	Kopalnia węgla	coalmine	30	Wytwórnia kadzidła	incense
11	Chata myśliwego	hunter	31	Kuźnia narzędzi	toolforge
12	Mleczarnia	dairy	32	Zbrojownia	armory
13	Sad	orchard	33	Miodosytnia	mead
14	Chata zbieracza	gatherer	34	Tkalnia	weaver
15	Pole stałe	field	35	Garncarnia	pottery
16	Farma	farm	36	Woskarnia	waxery
17	Gaj daktylowy	dategrove	37	Garbarnia	tannery
18	Plantacja bawełny	cotton	38	Bania	sauna
19	Las kadzidlany	incensewood	39	Kościół / Meczeta / Kapliczka	temple
20	Winnica	vineyard	40	Łaźnia	bathhouse

Po pętli produkcji: dojrzewają sadzonki (`saplings` → drzewa), działają zawory Dworu, następuje zużycie jedzenia, podatki i imigracja (rozdz. §4).

3.6 Narzędzia i zawory bezpieczeństwa

Jedno uniwersalne **narzędzie (T)** jest potrzebne wyłącznie do obsadzenia budynków specjalistycznych. Budynki podstawowe (drwal, leśnik, myśliwy, farma, rybak, las kadzidlany, winnica...) obsadza się bez narzędzia, bo klasyczna pułapka z Settlers (gracz zużywa narzędzia na proste budynki i staje bez żelaza) w testach zabijała casualowych Franków.

Tabela 13. Budynki wymagające narzędzia przy obsadzeniu (flaga tool w Data.BUILDINGS)

Nacja	Budynki T
Frankowie	Tartak, Kamieniołom, Młyn, Kopalnia żelaza, Kopalnia węgla, Huta, Kuźnia narzędzi, Zbrojownia
Saraceni	Tartak, Glinianka, Wytwórnia kadzidła, Tkalnia, Garncarnia
Wikingowie	Tartak, Huta, Kuźnia narzędzi, Zbrojownia, Kopalnia darniowa
Słowianie	Tartak, Kaszarnia, Woskarnia, Garbarnia

Źródła narzędzi i zawory:

- **zapas startowy** (Frankowie 14, Saraceni 8, Wikingowie 14, Słowianie 8), limit magazynu 10;
- **Kuźnia narzędzi** (Frankowie, Wikingowie): sztabka → narzędzie, 1,2/min;
- **kuźnia zamkowa** (zawór Dworu): 0,125 narzędzia/min, **tylko** gdy zapas narzędzi jest mniejszy niż 1 i jakiś budynek czeka na narzędzie (ok. 1 sztuka na 8 min, do pełnej sztuki);
- **zakup na Targu**: 20 zł za sztukę, automatycznie, gdy stoi czynny Targ, jest złoto i brakuje narzędzia. Frankowie nie wydają złota poniżej **rezerwy na żołd** brakujących do najbliższego terminu żołnierzy (`Tribute.soldierReserve`);
- **Targ nie wymaga narzędzia** (inaczej Saracen bez zapasu nie wyszedłby z blokady).

W normalnej grze po zmianach żaden budynek nie czeka na narzędzie; Targ zostaje ratunkiem przy błędnej kolejności budowy.

3.7 Drewno i las

Drzewa są **globalnym zasobem mapy**: `World.treeCount()` sumuje `tile.trees`. Chata drwala zdejmuję drzewa (`World.takeTrees` — losowe pole z drzewami, globalny RNG) i nie wytnie więcej drzew, niż jest na mapie. Leśniczówka co chwilę „sadzi” sadzonki: `plantFrac` rośnie o `tempo × popMult × dt`, a każda całkowita jedynka tworzy sadzonkę, która po czasie dojrzwania nacji staje się drzewem (`World.addTree` — losowe wolne pole bez drogi, złoża i cechy blokującej, `trees < 3`).

Tabela 14. Las według nacji

	Frankowie	Saraceni	Wikingowie	Słowianie
Drzewa na mapie	300	150	450	300
Dojrzwianie [min]	5	12	3	6
Chata drwala [pnie/min]	1	1	1,3	1
Leśniczówka [drzewa/min]	1	1	2	1
Tartak	1 pień → 2 deski	1 → 2	1,3 → 2,6	1 → 2
Próg gęstości lasu ($\text{plon} \times \min(1, \text{drzewa} / \text{próg})$)	—	—	—	200 (Zbieracz, Barć, Łowca futer)

Drewno jest osiłą każdej nacji, ale w innym kierunku: Wiking opiera się na nim najmocniej (szybki wzrost i największe zużycie — 1 leśnik utrzymuje 1,5 drwala), Saracen najsłabiej (powolny wzrost, mała zależność). U Słowian las jest dodatkowo **źródłem plonu**: poniżej 200 drzew plony zbieracza, Barci i Chaty łowcy maleją liniowo (rozdz. §7.4).

Piła zamkowa (zawór Dworu) zawsze przerabia do 0,6 pnia/min na deski w stosunku 1:1, jeśli są pnie i miejsce — chroni przed zablokowaniem gry bez tartaku.

Zawory budulca: gdy nie stoi Kamieniołom (Frankowie) albo Glinianka (Saraceni) i zapas kamienia (gliny) jest poniżej 6, Dwór dodaje 0,4 sztuki na minutę — pozwala zbudować pierwsze budynki wymagające tego surowca.

3.8 Woda

Woda jest zasobem globalnym (limit 40). Producenty: **Chata nosiwody** 1,5/min oraz **Qanat** (Saraceni) 4,5/min. Odbiorcy: Piekarnia 0,75, Gaj daktylowy 0,6, Plantacja bawełny 0,6, Miodosytia 0,5, Łaźnia 0,5, Garbarnia 0,4, Bania 0,3 wody/min. Mieszkańcy czerpią ze studni tylko wizualnie. U Saracenów woda jest realnym ograniczeniem: trzeci Gaj przy jednej nosiwodzie pracuje na 83%; Qanat opłaca się dopiero przy 6 lub więcej odbiorcach wody (zastępuje trzy nosiwody jednym robotnikiem).

3.9 Złoża

Kamieniołom, Kopalnia żelaza, Kopalnia węgla, Glinianka i Kopalnia darniowa stoją **tylko na złożu** swojego surowca (obrys 3×3 musi obejmować pole ze złożem), a złożo jest zarezerwowane dla pasującej kopalni (nie postawimy tam innego budynku). Liczba miejsc pod kopalnie nie jest parametrem gry, lecz **mapy**; generator gwarantuje minima, żeby gra się nie zacięła:

Tabela 15. Złoża na mapie (`Data.FACTIONS[].deposits`; minimum gwarantuje generator terenu)

Budynek	Złożo	Domyślnie miejsc	Minimum	Skutek braku (spec.)
Kamieniołom (Frankowie)	kamień	3	1	0 miejsc: brak Tartaku i Huty, haracz 0/7
Kopalnia żelaza (Frankowie)	żelazo	2	1	0: brak broni

Budynek	Złoże	Domyślnie miejsc	Minimum	Skutek braku (spec.)
Kopalnia węgla (Frankowie)	węgiel	3	2	1 miejsce: pół broni; 0: haracz 2/7 (sztuka broni zużywa 2 węgle)
Glinianka (Saraceni)	głina	2	1	0: ludność 14, 22 budowy w kolejce
Kopalnia darniowa (Wikingowie)	ruda darniowa	2	1	0: 0 wypraw
Chata rybaka (Wikingowie)	brzeg morza	6 miejsc na brzegu	6	limit <code>settings.shoreSpots</code>

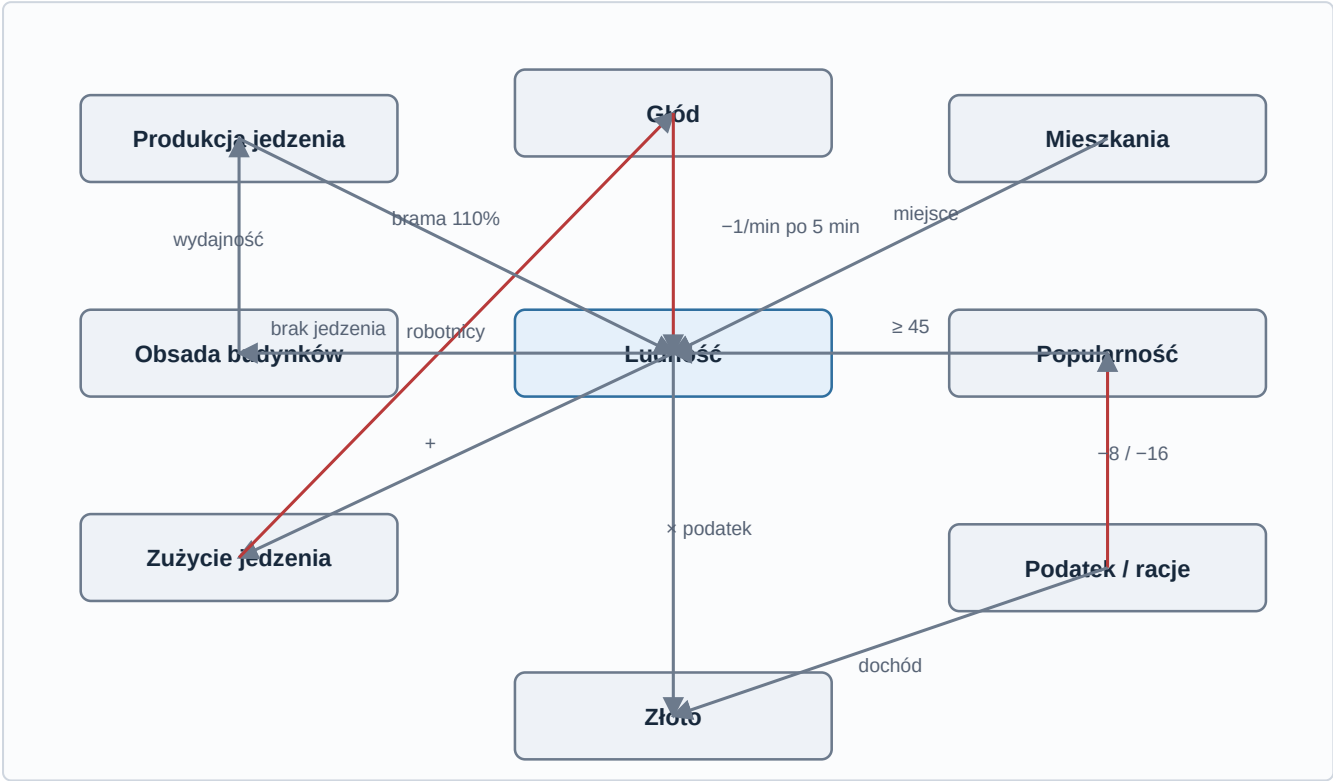
Opcjonalnie scenariusz może ustawić **skończone złożo** (`state.depositLeft` , np. `{ clay: 60 }`): kopalnia pracuje, aż zasób się wyczerpie, po czym stoi (status **X**) — wyczerpanie zatrzymuje budowę, a nie ekonomię.

3.10 Złoto

Złoto pochodzi wyłącznie z **podatków** (0 / 0,05 / 0,10 / 0,15 zł na osobę na minutę), **sprzedaży na Targu** (Saraceni, rozdz. §8) i **łupu z wypraw** (Wikingowie: 30 zł, wielka wyprawa 80 zł); żaden budynek nie ma stałej produkcji złota. Wydatki: koszty złotem przy otwarciu placu (Kościół / Meczet / Kapliczka 80 zł, Koszary 60, Qanat 60, Łaźnia 40, Bania 40, Strażnica 40, Karawanseraj 100), zakup narzędzi (20 zł), żołd Franków (3 zł na żołnierza), utrzymanie Strażnicy (0,6 zł/min) i płatność haraczu (Saraceni).

4. Ludność, jedzenie i popularność

Ludność jest sprzężona z jedzeniem, mieszkaniami i popularnością (rysunek poniżej). Wzrost jest **monotoniczny do poziomu, który da się wykarmić** — bramka imigracji patrzy na *potencjalną* produkcję jedzenia, więc ludność nie „piłuje się” (20 → 13 → 32), jak w pierwszych wersjach modelu.



Rysunek 3. Główne sprzężenia ekonomii: ludność rośnie tylko przy nadwyżce jedzenia ($\geq 110\%$ zużycia), miejscu w chatach i popularności ≥ 45 ; podatek daje złoto kosztem popularności.

4.1 Jedzenie

Jedzenie to **jedna wspólna pula sześciu rodzajów**: ryby, chleb, mięso, ser, owoce i daktyl (kasza Słowian liczy się jako chleb, zbiory leśne jako owoce). Rodzaje są wymienne — mieszkaniec zjada porcję dowolnego z nich.

- Zużycie ludzi:** $\text{ludność} / 6 \times \text{racje}$ porcji na minutę (1 porcja na 6 minut na osobę przy racjach $\times 1$; 1 porcja/min żywi 6 osób). Racje ustawia gracze: $\times 0,5 / \times 1 / \times 1,5 / \times 2$.
- Budowniczości w pracy** (noszący lub budujący) zjadają 1,5 racji: dodatek $0,5 \times \text{racje} / 6$ porcji/min na osobę; czekający 1 rację (są częścią ludności).
- Kopalnie i Wytwórnia kadzidła** („podatek górniczy”): Kopalnia żelaza i węgla 0,5, Wytwórnia kadzidła 0,5, Kopalnia darniowa 0,25 porcji/min — ale **tylko z nadwyżki**: gdy zapas jedzenia jest mniejszy niż 5 minut zużycia ludności, budynek stoi, a ludność ma pierwszeństwo.
- Kolejność poboru:** najpierw **ryby**, potem pozostałe rodzaje **proporcjonalnie do zapasów**.
- Zapas startowy:** 36 porcji pierwszego rodzaju (chleb / daktyl / ryby / kasza), ok. 15 minut dla 14 osób.

Źródła jedzenia (jeden obsadzony budynek, racje $\times 1$):

Tabela 16. Źródła jedzenia

Źródło	Nacje	Porcje/min	Osób	Porcje na robotnika	Uwagi
Sad	wszystkie	1,0	6,0	1,0	bez wejść

Źródło	Nacje	Porcje/min	Osób	Porcje na robotnika	Uwagi
Mleczarnia	wszystkie	1,0	6,0	1,0	bez wejść („krowy doją się same”)
Chata myśliwego	wszystkie	0,8	4,8	0,8	nie wymaga lasu; plon × min(1, zwierzyzna w łowisku / 5 pkt)
Łańcuch chleba (Farma + Młyn + Piekarnia + 0,5 nosiwody)	Frankowie	3,3	19,8	0,94	wejścia: zboże 1,5, mąka 1,5, woda 0,75
Gaj daktylowy (+ 0,4 nosiwody)	Saraceni	1,6	9,6	1,14	wymaga wody 0,6/min
Chata rybaka	Wikingowie	1,1	6,6	1,1	pole nad morzem, zasób nieskończony, limit miejsc na brzegu
Chata zbieracza	Słowianie	1,0 × gęstość lasu	do 6	do 1,0	zbiory leśne liczą się jako owoce; zależy od Puszczy
Pole stałe + Kaszarnia	Słowianie	2,25 (Kaszarnia)	13,5	—	kasza liczy się jako chleb

4.2 Głód

Gdy w ticku zabraknie jedzenia ($niedobór > 0$), rośnie `starveTimer`; gdy jedzenia wystarcza, wraca do 0. Po **5 minutach** ciągłego głodu ludność maleje o 1 osobę na minutę, **nie poniżej 10 osób**. W czasie głodu (`starveTimer` ≥ 5) producenci żywności pracują z pełną mocą, a pozostałe budynki tracą 25% wydajności ($hungerK = 0,75$). Głód to miękka porażka: podłoga 10 jest zawsze możliwa do wykarmienia przez dwa tanie budynki żywności, więc kryzys nie przekreśla gry. Statystyka `stats.starveMin` zlicza minuty głodu po upływie tych 5 minut (ludność ginie albo stoi już na podłodze 10 osób).

4.3 Mieszkania i imigracja

Mieszkania: 20 bazowych (Dwór) + 6 na każdą Chatę (Wikingowie: Długi dom — 8). Nowy osadnik przybywa tylko wtedy, gdy spełnione są **wszystkie** warunki:

1. brak głodu w tym ticku (jedzenia starczyło),
2. **potencjał produkcji jedzenia $\geq 110\%$ zużycia** (ludzie + budowniczy w pracy + kopalnie),
3. ludność < liczba mieszkań,
4. popularność ≥ 45 .

Tempo: jedna osoba co $0,5 \times 50 / \max(10, \text{popularność})$ minuty (przy popularności 50: co 0,5 min).

Potencjał jedzenia liczy produkcję jedzenia „przy dostępnych wejściach”: obsadzone budynki żywności, których wejścia leżą w magazynie albo są wytwarzane przez inny czynny budynek (chleb liczy Piekarnia; Farma i Młyn same nie są jedzeniem). Potencjał uwzględnia las i zwierzynę (Myśliwy, Puszcza) oraz sprawność logistyczną, ale **nie** premię popularności. Margines 110% (a nie 105%) pochłania narzut transportu i błędy casualowego gracza: przy bramie 1,05 casualowi Frankowie przechodzili poziom 4 w 85% przypadków, przy 1,10 w 95%, kosztem ok. 5 minut wolniejszego startu.

Maksymalna ludność wynikająca z bramki (pokazywana w panelu Żywność i w Doradcy jako „Maks. ludność”):

$$P_{\max} = \lfloor (F / 1,1 - B - E) / (\text{racje} / 6) \rfloor + 1$$

F – potencjał jedzenia [porcji/min], B – budowniczy w pracy ($0,5/6$ na osobę × racje), E – podatek górniczy kopalń ($0,5 / 0,25$ porcji/min na obsadzoną)

SPECYFIKACJA

Przykład ze specyfikacji (wersja z bramą 105%): 2 łańcuchy chleba, Chata myśliwego, Mleczarnia, Sad i 3 kopalnie dają $F = 8,8$ i $P_{\max} \approx 41,7$; symulator ustabilizował ludność na 42. W grze z bramą 110% wartość jest odpowiednio niższa; Doradca pokazuje ją na bieżąco.

4.4 Popularność

Popularność startuje od 50 i jest sumą modyfikatorów ograniczoną do 0–100. Wpływa na **wydajność** (popMult , 0,8–1,2) i na **tempo imigracji**, a poniżej 45 zamraża imigrację.

Tabela 17. Składniki popularności (Economy.popularity)

Składnik	Wartość
Wartość bazowa	50
Racje $\times 0,5$ / $\times 1$ / $\times 1,5$ / $\times 2$	0 / +15 / -25 / +8
Podatek 0 / 1 / 2 / 3	+8 / 0 / -8 / -16
Różnorodność jedzenia	+4 za każdy rodzaj poza pierwszym, którego zapas przekracza 5 szt.
Dobrobyt: Kościół / Meczet / Kapliczka	+10, gdy zasilonych budynków $\times 50 \geq$ ludność
Dobrobyt: Łaźnia (Saraceni) / Bania (Słowianie)	+6, gdy zasilonych budynków $\times 50 \geq$ ludność
Zakres końcowy	0–100
Mnożnik wydajności	$\text{clamp}(1 + 0,004 \times (\text{popularność} - 50), 0,8; 1,2)$
Imigracja	tylko przy popularności ≥ 45 ; interwał $0,5 \times 50 / \max(10, \text{popularność})$ min

Racje. Wyższe racje ($\times 1,5$ i $\times 2$) są w tej gospodarce pułapką: w teście dawały głód 47 i 123 minuty, bo zużycie rośnie szybciej niż popularność; racje $\times 0,5$ zamrażają imigrację (popularność $37 < 45$).

Podatki. Poziom 1 jest domyślny. Poziom 0 daje popularność, ale nie wystarcza na Koszary i żołąd (Frankowie) — to świadomy wybór „popularność kosztem haraczu”. Poziom 3 jest pułapką startową ($-16 \rightarrow$ popularność poniżej progu 45); Doradca ostrzega przed nim, a Kościół z winem pozwala go sensownie podnieść w grze środkowej.

Tabela 18. Podatki i racje

Poziom podatku	Dochód [zł / os. / min]	Popularność
0	0	+8
1	0,05	0
2	0,1	-8
3	0,15	-16

Różnorodność jedzenia liczy rodzaje, których zapas przekracza 5 porcji (+4 za każdy poza pierwszym), więc zbudowanie Sadu czy Mleczarni obok głównego źródła podnosi popularność o 4–8.

4.5 Dobrobyt

Dobrobyt zastępuje manę z wcześniejszych wersji: kościół lub meczet spala dobro własnej nacji (0,3/min) i daje **+10 popularności**, a Saraceni i Słowianie dostają dodatkowo **+6** z Łaźni lub Bani. Jeden budynek obsługuje 50 osób: premia obowiązuje, gdy $\text{liczba zasilonych budynków} \times 50 \geq \text{ludność}$; liczba zasilonych liczona jest w ticku produkcji (zapas dobra musi pokryć zużycie tej chwili) i dopiero w następnym ticku wchodzi do popularności.

Tabela 19. Dobrobyt według nacji

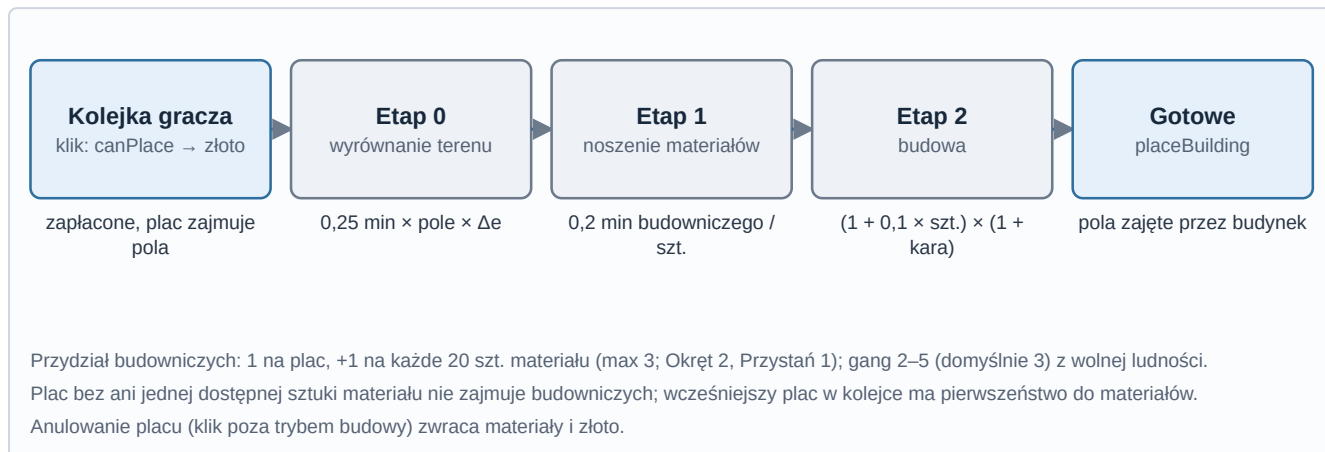
	Frankowie	Saraceni	Wikingowie	Słowianie
Budynek	Kościół 8 d + 80 zł	Meczet 6 d + 4 gl + 80 zł; Łaźnia 6 d + 6 gl + 40 zł	Kościół 12 d + 80 zł	Kapliczka 8 d + 80 zł; Bania 6 d + 40 zł

	Frankowie	Saraceni	Wikingowie	Słowianie
Dobro (0,3/min)	wino (Winnica + Winiarz)	kadzidło (z łańcucha)	miód pitny (z łańcucha)	wosk (z łańcucha miodu i wosku)
Premia	+10	+10 (Meczeta), +6 (Łaźnia: woda 0,5/min)	+10	+10 (Kapliczka), +6 (Bania: woda 0,3 + pień 0,2/min)
Koszt dobra	0,3 wina/min	ok. 0,6–1,3 zł/min utraconej sprzedaży	konkuruje z Okrętem	ok. 0,9 osobominuty/min utraconej daniny

Decyzja gracza jest łagodna, ale prawdziwa: Frank kupuje dwoma budynkami na wino zdolność do podatku 3, Saracen wybiera między sprzedażą kadzidła a spaleniem go oraz między Łaźnią a wodą dla Gajów (przepis bez Meczetu i Łaźni daje pojemność haraczu 9 wobec 8: Dobrobyt kosztuje 1 poziom), Wiking — między miodem na wyprawę a miodem do kościoła, a Słowianin — między woskiem do Kapliczki a daniną. Zakłęcia, gdyby kiedyś wróciły, mają być wydatkiem tego samego zasobu, a nie nowym łańcuchem.

5. Budowa i budownicowie

Budynki powstają na **placach budowy**. Gracz wskazuje miejsce (rozdz. §11), gra rezerwuje pola pod obrys i pobiera złoto, a **budownicowie** — rekrutowani z wolnej ludności — sami noszą materiały na plac, ewentualnie wyrównują teren i budują, aż budynek stanie. Cały proces obsługuje moduł `Build`.



Rysunek 4. Etapy budowy (moduł `Build`): od otwarcia placu do gotowego budynku.

5.1 Otwarcie placu

`Build.enqueue(id, x, y) :`

1. sprawdza `World.canPlace` (zasady zabudowy, teren, wymagania budynku);
2. **pierwsza Chata drwala jest darmowa** — także wtedy, gdy pierwszy plac stoi dopiero w kolejce;
3. sprawdza złoto: koszt złotem musi mieścić się w złocie **po odjęciu rezerwy na żołd** (`Tribute.soldierReserve`, Frankowie);
4. **pobiera złoto od razu** (materiały — dopiero przy wniesieniu na plac) i rezerwuje pola obrysu (`occup = uid placu`);
5. zapisuje potrzebny materiał (`need`), liczbę sztuk, **karę za ciasną zabudowę** (`crowd`), poziom docelowy i czas **wyrównania terenu** (`levelTotal`) oraz czas pracy `workNeed = (1,0 + 0,1 × sztuki) × (1 + kara)`.

Place są w **kolejce gracza**: wcześniejszy plac ma pierwszeństwo do materiałów. Kliknięcie placu poza trybem budowy **anuluje** go: materiały i złoto wracają do magazynu.

5.2 Budownicowie

Liczbę budowniczych ustawia gracz (przycisk 🧑): **domyślnie 3, minimum 2, maksimum 5**. Rekrutacja idzie z wolnej ludności (`ludność - załoga na wyprawie - pracownicy budynków`); pracownicy mają pierwszeństwo, ale gang nie schodzi poniżej 2. W testach 3 budowniczych odtwarza tempo dawnego modelu (2 min na budynek, 2 równolegle), a 1 budowniczy wywraca grę (ludność @20 spada do 10–15), dlatego minimum wynosi 2. Od 3 w górę budownicowie nie są wąskim gardłem — tempo budowy ogranicza podaż desek, nie ręce.

Przydział do placów (w kolejności kolejki):

- plac **bez ani jednej dostępnej sztuki wymaganego materiału** (po rezerwacji przez wcześniejsze place) nie zajmuje budowniczych — przechodzą do następnego placu (bez tej reguły zacięty plac z gliną zablokował całą budowę Saracenów);
- jeden budowniczy na plac; plac z ponad 20 sztukami materiału dostaje +1 na każde 20 sztuk, najwyżej 3 (**Okręt 2, Przystań 1**);

- budowniczy zjada 1,5 racji w pracy (rozdz. §4).

5.3 Etapy

Tabela 20. Etapy placu budowy

Etap	Co się dzieje	Czas
0 — wyrównanie	plac na pochyłym terenie: budownicowie kopią, materiałów jeszcze nie noszą; po zakończeniu pola obrysu dostają poziom docelowy (<code>flatten</code>) i chunk podłoża jest odświeżany	$0,25 \text{ min} \times \text{pola} \times \Delta e$ (pracy budowniczych; dzielone przez liczbę przydzielonych)
1 — noszenie	przydzieleni budownicowie noszą materiały ze Składu / magazynu; plac czeka, dopóki nie dotrze komplet	0,2 min budowniczego na sztukę
2 — budowa	przydzieleni budownicowie budują razem; po zebraniu <code>workNeed</code> powstaje budynek	$(1 + 0,1 \times \text{sztuki}) \times (1 + \text{kara za ciasną zabudowę})$

Postęp pokazywany w HUD (`Build.progressOf`) to 50% noszenie + 50% budowa. Na ekranie plac jest rusztowaniem odsłaniającym sprite od dołu, ze stosami materiałów przy rogu; budownicowie (Walkers) noszą z Dworu na plac i stoją przy obrysie.

5.3.1 Przykładowe czasy

Poniższa tabela jest wynikiem **symulacji w grze** (wygenerowana podczas budowy dokumentu): 3 budowniczych, płaski teren (brak etapu 0), brak sąsiadów (brak kary), pełny magazyn materiałów. W prawdziwej grze dochodzi czas oczekiwania na deski, wyrównania stoku i kara za sąsiadów.

Tabela 21. Czas budowy od otwarcia placu do ukończenia [min gry]: 3 budowniczych, płaski teren, bez sąsiadów, zapas materiałów w magazynie (symulacja `Build.tick`; w nawiasie liczba sztuk materiału)

Budynek	Frankowie	Saraceni	Wikingowie	Słowianie
Chata	2,2 (4 szt.)	2,7 (6 szt.)	3,3 (8 szt.)	2,2 (4 szt.)
Chata myśliwego	2,2 (4 szt.)	2,2 (4 szt.)	2,2 (4 szt.)	2,2 (4 szt.)
Mleczarnia	2,7 (6 szt.)	2,7 (6 szt.)	2,7 (6 szt.)	2,7 (6 szt.)
Sad	2,2 (4 szt.)	2,2 (4 szt.)	2,2 (4 szt.)	2,2 (4 szt.)
Chata nosiwody	2,2 (4 szt.)	2,2 (4 szt.)	2,2 (4 szt.)	2,2 (4 szt.)
Farma	3,3 (8 szt.)	—	—	—
Młyn	3,3 (8 szt.)	—	—	—
Piekarnia	3,3 (8 szt.)	—	—	—
Gaj daktylowy	—	3,3 (8 szt.)	—	—
Qanat	—	5,8 (16 szt.)	—	—
Chata rybaka	—	—	2,7 (6 szt.)	—
Chata zbieracza	—	—	—	2,2 (4 szt.)
Pole stałe	—	—	—	2,2 (4 szt.)
Kaszarnia	—	—	—	2,7 (6 szt.)
Chata drwala	1 (0 szt.)	1 (0 szt.)	1 (0 szt.)	1 (0 szt.)
Leśniczówka	2,2 (4 szt.)	2,2 (4 szt.)	2,2 (4 szt.)	2,2 (4 szt.)
Tartak	3,3 (8 szt.)	2,7 (6 szt.)	3,6 (9 szt.)	3,3 (8 szt.)
Kamieniołom	2,2 (4 szt.)	—	—	—
Kopalnia żelaza	3,3 (8 szt.)	—	—	—
Kopalnia węgla	3,3 (8 szt.)	—	—	—
Glinianka	—	2,2 (4 szt.)	—	—
Kopalnia darniowa	—	—	3,9 (10 szt.)	—
Wypalarka węgla	—	—	3,3 (8 szt.)	—
Barć	—	—	—	2,2 (4 szt.)
Chata łowcy futer	—	—	—	2,2 (4 szt.)

Budynek	Frankowie	Saraceni	Wikingowie	Słowianie
Huta	5,2 (14 szt.)	—	6,1 (17 szt.)	—
Kuźnia narzędzi	3,9 (10 szt.)	—	4,5 (12 szt.)	—
Winnica	3,3 (8 szt.)	—	—	—
Winiarz	3,3 (8 szt.)	—	—	—
Las kadzidlany	—	2,2 (4 szt.)	—	—
Wytwórnia kadzidła	—	3,3 (8 szt.)	—	—
Plantacja bawełny	—	3,3 (8 szt.)	—	—
Tkalnia	—	3,3 (8 szt.)	—	—
Garncarnia	—	3,3 (8 szt.)	—	—
Miodosytnia	—	—	3,6 (9 szt.)	—
Woskarnia	—	—	—	2,7 (6 szt.)
Garbarnia	—	—	—	2,7 (6 szt.)
Targ	2,7 (6 szt.)	3,3 (8 szt.)	2,7 (6 szt.)	2,7 (6 szt.)
Skład	2,2 (4 szt.)	2,2 (4 szt.)	2,2 (4 szt.)	2,2 (4 szt.)
Karawanseraj	—	3,4 (20 szt.)	—	—
Zbrojownia	5,2 (14 szt.)	—	6,1 (17 szt.)	—
Koszary	5,8 (16 szt.)	—	—	—
Strażnica	—	5,2 (14 szt.)	—	—
Przystań	—	—	5,7 (15 szt.)	—
Okręt	—	—	6,7 (40 szt.)	—
Kościół / Meczet / Kapliczka	3,3 (8 szt.)	3,9 (10 szt.)	4,5 (12 szt.)	3,3 (8 szt.)
Łaźnia	—	4,5 (12 szt.)	—	—
Bania	—	—	—	2,7 (6 szt.)

Dla przykładu Chata Franków (4 d) to 0,8 min noszenia (4 szt. × 0,2 min, jeden budowniczy) i 1,4 min pracy (1,0 + 0,1 × 4), razem 2,2 min. Specyfikacja podaje dla swojego modelu: Chata ok. 3,3 min, Przystań (15 d) ok. 9 min, Okręt (40 d) ok. 11 min — z czekaniem na deski. W grze, przy zapasie materiałów, czasy są krótsze (Chata 2,2, Przystań 5,7, Okręt 6,7 min); realny czas wydłużają kolejka materiałów, kara za ciasną zabudowę (do +40% czasu pracy) i wyrównanie stoku. Noszenie jest w modelu budowy czasem budowniczego, a nie drogi, więc odległości uwzględnia dopiero sprawność logistyczna producentów (rozdz. §12).

5.4 Złoto, rezerwa, anulowanie

- Koszt w złocie (Kościół, Qanat, Łaźnia, Bania, Karawanseraj, Koszary, Strażnica) jest pobierany **przy otwarciu placu**.
- Frankowie:** otwarcie placu z kosztem w złocie i zakup narzędzia nie schodzą poniżej złota potrzebnego na żołd brakujących do najbliższego terminu żołnierzy (3 zł × brakujący) — Doradca przypomina o Koszarach 30 minut przed pierwszym terminem.
- Anulowanie placu zwraca wniesione materiały i złoto; wyburzenie gotowego budynku (zdarzenie pożaru) nie zwraca niczego, a narzędzie „przywiązane” do budynku przepada.
- Budowniczowie, którzy nic nie mają do roboty (brak materiału na każdym placu), stoją beczynnie — w teście 3 budowniczych jest beczynnych 60–73% czasu, co potwierdza, że wąskim gardłem są deski.

5.5 Odbudowa po pożarze

Odbudowa 6 budynków produkcyjnych po pożarze w 100. minucie trwa w modelu specyfikacji ok. 9 / 9 / 10 minut (Frankowie / Saraceni / Wikingowie) przy 3 budowniczych, 7 min przy 5; z jednym budowniczym 24–49 minut. Reguła pierwszeństwa pracowników jest konieczna: bez niej trzech beczynnych budowniczych blokowało odbudowę Franka po głodzie do 40. minuty na stałe (ludność 10 do końca gry); z regułą ludność wraca do 26 i haracz do 6/7.

6. Katalog budynków

Budynki definiuje `Data.BUILDINGS`; ich liczby (koszty, wejścia, wyjścia, tempo) pochodzą ze specyfikacji v3.6, a obrysy z katalogu sprite'ów (`SPRITE_META`, rozdz. §15). Tabele w tym rozdziale są **generowane z kodu** podczas budowy dokumentu — jeśli zmieni się koszt lub tempo w `Data`, po przebudowie PDF zmieni się także tabela.

6.1 Jak czytać tabele

- **Obrys** — szerokość × głębokość w polach (północny róg obrysu to `b.x`, `b.y`); wejście (drzwi) jest zdefiniowane w katalogu sprite'ów (strona S lub E i punkt na ścianie) i służy nosicielom, budowniczym i myśliwym jako cel marszu.
- **Koszt** — *d* deski, *k* kamień, *gl* glina, *zł* złoto (płatne przy otwarciu placu). **T** — budynek wymaga narzędzia przy obsadzeniu.
- **Prac.** — liczba pracowników (0 lub 1).
- **Wejścia / Wyjścia** — tempo na minutę gry przy obsadzie 1 pracownika i mnożniku 1 (Wikingowie: wartości już po skali nacji, np. Tartak 1,3 → 2,6). Dla budynków wielowyjściowych (Woskarnia) wyjścia dzielą się w podanej proporcji.
- **Wymagania terenu** — dodatkowe warunki `World.canPlace` (rozdz. §11).

Każda nacja ma zestaw własnych budynków i wspólnych, które różnią się kosztem (Frank płaci deskami i kamieniem, Saracen deskami i gliną, Wiking i Słowianin tylko deskami):

Tabela 22. Liczba budynków budowanych przez nację (bez Dworu)

Nacja	Razem	W tym własne
Frankowie	23	12
Saraceni	22	11
Wikingowie	20	9
Słowianie	19	8

6.2 Budynki wspólne

Jedenaście budynków występuje u wszystkich czterech nacji:

Tabela 23. Budynki wspólne i ich koszty w poszczególnych nacjach (d = deski, k = kamień, gl = glina)

Budynek	Frankowie	Saraceni	Wikingowie	Słowianie	T
Chata	4 d	2 d + 4 gl	8 d	4 d	
Chata drwała	4 d	4 d	4 d	4 d	
Leśniczówka	4 d	4 d	4 d	4 d	
Tartak	6 d + 2 k	4 d + 2 gl	9 d	8 d	T
Chata myśliwego	4 d	4 d	4 d	4 d	
Mleczarnia	6 d	4 d + 2 gl	6 d	6 d	
Sad	4 d	4 d	4 d	4 d	
Chata nosiwody	4 d	2 d + 2 gl	4 d	4 d	
Targ	6 d	4 d + 4 gl	6 d	6 d	
Skład	4 d	2 d + 2 gl	4 d	4 d	
Kościół / Meczeta	8 d + 80 zł	6 d + 4 gl + 80 zł	12 d + 80 zł	8 d + 80 zł	

U Wikingów Chata (Długi dom) mieści 8 osób zamiast 6, a drwał, leśnik i tartak pracują szybciej (rozdz. §3); u Słowian cennik bazuje na Wikingach bez zwiększonej Chaty (Chata 4 d, Tartak 8 d, reszta jak u Franków bez kamienia). Kościół nazywa się u Saracenów Meczetem, a u Słowian Kapliczką.

6.3 Dwór

Dwór (obrys 4×4, wejście od południa, stoi pośrodku mapy) jest „sercem osady” i nie można go budować ani burzyć. Daje:

- **20 mieszkań** bazowych i komorę **20 sztuk każdego towaru** (zwiększa pojemność każdego towaru o 20, patrz rozdz. §3),
- **piłę zamkową** (do 0,6 pnia/min → deski) i zawory budulca (kamień dla Franków, glina dla Saracenów),
- cel nosicielei bez Składu, punkt startowy budowniczych i obywateli,
- ustawienie liczby budowniczych (przycisk 🧑 w HUD; 2–5).

6.4 Frankowie

Najpełniejszy zestaw (23 budynki): łańcuch chleba (Farma → Młyn → Piekarnia), metalurgia (Kopalnie żelaza i węgla → Huta → Kuźnia narzędzi / Zbrojownia), wino (Winnica → Winiarz) i Koszary.

Tabela 24. Budynki nacji: Frankowie — 23 budynków poza Dworem (4×4). T = wymaga narzędzia przy obsadzeniu; tempo na minutę gry przy obsadzie 1 pracownika

Budynek	Obrys	Koszt	T	Prac.	Wejścia /min	Wyjścia /min	Wymagania terenu	Uwagi
Chata	2×2	4 d		0		mieszkania +6 (Wikingowie +8)		
Chata myśliwego	2×2	4 d		1		mięso 0,8		nie wymaga lasu; plon × min(1, zwierzyna w łowisku / 5 pkt)
Mleczarnia	2×3	6 d		1		ser 1		
Sad	3×3	4 d		1		owoce 1		
Chata nosiwody	2×2	4 d		1		woda 1,5		
Farma	4×3	6 d + 2 k		1		zboże 1,5		
Młyn	3×3	6 d + 2 k	T	1	zboże 1,5	mąka 1,5		
Piekarnia	2×3	6 d + 2 k		1	mąka 1,5 + woda 0,75	chleb 3,3		
Chata drwala	2×2	4 d		1		pień 1		pierwsza Chata drwala jest darmowa
Leśniczówka	2×2	4 d		1		sadzonki 1; sadzi drzewa 1,0		Sadzi drzewa
Tartak	3×2	6 d + 2 k	T	1	pień 1	deski 2		
Kamieniołom	3×3	4 d	T	1		kamień 1,5	złóże: kamień	
Kopalnia żelaza	3×3	4 d + 4 k	T	1		ruda 1	złóże: żelazo	zjada 0,5 porcji/min tylko z nadwyżki
Kopalnia węgla	3×3	4 d + 4 k	T	1		węgiel 1	złóże: węgiel	zjada 0,5 porcji/min tylko z nadwyżki
Huta	3×3	8 d + 6 k	T	1	ruda 1,5 + węgiel 1,5	sztabki 1,5		
Kuźnia narzędzi	2×3	6 d + 4 k	T	1	sztabki 1,2	narzędzia 1,2		
Winnica	3×3	6 d + 2 k		1		winogrona 1		
Winiarz	2×3	6 d + 2 k		1	winogrona 1	wino 1		
Targ	3×3	6 d		1		zakup narzędzi (20 zł); Saraceni: sprzedaż towarów		
Skład	3×2	4 d		0		pojemność +80 szt. każdego towaru		
Zbrojownia	3×2	8 d + 6 k	T	1	sztabki 1,2 + węgiel 1,2	broń 1,2		

Budynek	Obrys	Koszt	T	Prac.	Wejścia /min	Wyjścia /min	Wymagania terenu	Uwagi
Koszary	3×3	8 d + 8 k + 60 zł		1		warunek dostaw żołnierzy		
Kościół	4×2	8 d + 80 zł		1	wino 0,3	Dobrobyt +10 (1 budynek na 50 os.)		

6.5 Saraceni

Saracen płaci haracz złotem z handlu trzema towarami; ma 22 budynki, w tym Strażnicę i Karawanseraj.

Tabela 25. Budynki nacji: Saraceni — 22 budynków poza Dworem (4×4). T = wymaga narzędzia przy obsadzeniu; tempo na minutę gry przy obsadzie 1 pracownika

Budynek	Obrys	Koszt	T	Prac.	Wejścia /min	Wyjścia /min	Wymagania terenu	Uwagi
Chata	2×2	2 d + 4 gl		0		mieszkania +6 (Wikingowie +8)		
Chata myśliwego	2×2	4 d		1		mięso 0,8		nie wymaga lasu; plon × min(1, zwierzyna w łowisku / 5 pkt)
Mleczarnia	2×3	4 d + 2 gl		1		ser 1		
Sad	3×3	4 d		1		owoce 1		
Chata nosiwody	2×2	2 d + 2 gl		1		woda 1,5		
Gaj daktylowy	3×3	4 d + 4 gl		1	woda 0,6	daktyle 1,6		
Qanat	3×2	6 d + 10 gl + 60 zł		1		woda 4,5		Zastępuje 3 nosiwody
Chata drwala	2×2	4 d		1		pień 1		pierwsza Chata drwala jest darmowa
Leśniczówka	2×2	4 d		1		sadzonki 1; sadi drzewa 1,0		Sadzi drzewa
Tartak	3×2	4 d + 2 gl	T	1	pień 1	deski 2		
Glinianka	3×3	4 d	T	1		głina 2	złóże: glina	
Las kadzidlany	3×3	4 d		1		żywica 1		
Wytwórnia kadzidla	2×3	4 d + 4 gl	T	1	żywica 1	kadzidło 1		zjada 0,5 porcji/min tylko z nadwyżki
Plantacja bawełny	3×3	4 d + 4 gl		1	woda 0,6	bawełna 1,2		
Tkalcia	2×3	4 d + 4 gl	T	1	bawełna 1,2	tkanina 1		
Garncarnia	2×3	4 d + 4 gl	T	1	głina 1 + pień 0,3	ceramika 1		
Targ	3×3	4 d + 4 gl		1		zakup narzędzi (20 zł); Saraceni: sprzedaż towarów		
Skład	3×2	2 d + 2 gl		0		pojemność +80 szt. każdego towaru		
Karawanseraj	4×3	10 d + 10 gl + 100 zł		1		ceny +20%, liczy się jak 2 Targi (max 2)		
Strażnica	2×2	8 d + 6 gl + 40 zł		1	złoto 0,6 (utrzymanie)	ryzyko napadu –40%, łup –25% (max 2)		
Meczet	3×3	6 d + 4 gl + 80 zł		1	kadzidło 0,3	Dobrobyt +10 (1 budynek na 50 os.)		
Łaźnia	3×3	6 d + 6 gl + 40 zł		1	woda 0,5	Dobrobyt +6 (1 budynek na 50 os.)		

6.6 Wikingowie

Wiking opiera się na drewnie, morzu i wyprawach: 20 budynków, węgiel wyłącznie z Wypalarek, Przystań i Okręt jako rdzeń mechaniki haraczu.

Tabela 26. Budynki nacji: Wikingowie — 20 budynków poza Dworem (4×4). T = wymaga narzędzia przy obsadzeniu; tempo na minutę gry

przy obsadzie 1 pracownika

Budynek	Obrys	Koszt	T	Prac.	Wejścia /min	Wyjścia /min	Wymagania terenu	Uwagi
Chata	2×2	8 d		0		mieszkania +6 (Wikingowie +8)		
Chata myśliwego	2×2	4 d		1		mięso 0,8		nie wymaga lasu; plon × min(1, zwierzyzna w łowisku / 5 pkt)
Mleczarnia	2×3	6 d		1		ser 1		
Sad	3×3	4 d		1		owoce 1		
Chata nosiwody	2×2	4 d		1		woda 1,5		
Chata rybaka	2×2	6 d		1		ryby 1,1	brzeg morza, jeziora lub lodu	
Chata drwala	2×2	4 d		1		pień 1,3		pierwsza Chata drwala jest darmowa
Leśniczówka	2×2	4 d		1		sadzonki 2; sadi drzewa 1,0		Sadzi drzewa
Tartak	3×2	9 d	T	1	pień 1,3	deski 2,6		
Kopalnia darniowa	3×3	10 d	T	1		ruda 1,5	złozę: ruda darniowa (torf)	zjada 0,25 porcji/min tylko z nadwyżki
Wypalarka węgla	3×3	8 d		1	pień 2	węgiel 1		Jedynę źródło węgla
Huta	3×3	17 d	T	1	ruda 1,5 + węgiel 1,5	sztatki 1,5		
Kuźnia narzędzi	2×3	12 d	T	1	sztatki 1,2	narzędzia 1,2		
Miodosytnia	2×3	9 d		1	woda 0,5	miód pitny 1		
Targ	3×3	6 d		1		zakup narzędzi (20 zł); Saraceni: sprzedaż towarów		
Skład	3×2	4 d		0		pojemność +80 szt. każdego towaru		
Zbrojownia	3×2	17 d	T	1	sztatki 1,2 + węgiel 1,2	broń 1,2		
Przystań	4×2	15 d		0		warunek dla Okrętu	brzeg morza	
Okręt	4×2	40 d		0	miód pitny + broń + załoga	wyprawa (6/6/6 albo 12/12/12)	brzeg morza	Wyprawa: 6 załogi, 6 miodu, 6 broni
Kościół	3×3	12 d + 80 zł		1	miód pitny 0,3	Dobrobyt +10 (1 budynek na 50 os.)		

6.7 Słowianie

Słowianin żyje z lasu i płaci haracz w towarach: 19 budynków, plony leśne zależne od gęstości lasu (Puszcza).

Tabela 27. Budynki nacji: Słowianie — 19 budynków poza Dworem (4×4). T = wymaga narzędzia przy obsadzeniu; tempo na minutę gry przy obsadzie 1 pracownika

Budynek	Obrys	Koszt	T	Prac.	Wejścia /min	Wyjścia /min	Wymagania terenu	Uwagi
Chata	2×2	4 d		0		mieszkania +6 (Wikingowie +8)		
Chata myśliwego	2×2	4 d		1		mięso 0,8		nie wymaga lasu; plon × min(1, zwierzyzna w łowisku / 5 pkt)
Mleczarnia	2×3	6 d		1		ser 1		
Sad	3×3	4 d		1		owoce 1		
Chata nosiwody	2×2	4 d		1		woda 1,5		
Chata zbieracza	2×2	4 d		1		zbiory leśne 1		plon × min(1, drzewa / 200)
Pole stałe	3×3	4 d		1		zboże 1,2		
Kaszarnia	2×3	6 d	T	1	zboże 1,5	kasza 2,25		Kasza liczy się jako chleb
Chata drwala	2×2	4 d		1		pień 1		pierwsza Chata drwala jest darmowa
Leśniczówka	2×2	4 d		1		sadzonki 1; sadi drzewa 1,0		Sadzi drzewa
Tartak	3×2	8 d	T	1	pień 1	deski 2		

Budynek	Obrys	Koszt	T	Prac.	Wejścia /min	Wyjścia /min	Wymagania terenu	Uwagi
Barć	3×3	4 d		1		plaster 1,2		plon × min(1, drzewa / 200)
Chata łowcy futer	2×2	4 d		1		futro surowe 0,8		plon × min(1, drzewa / 200)
Woskarnia	2×3	6 d	T	1	plaster 1,2	miód 0,84 + wosk 0,36		Miód i wosk idą do daniny; wosk spala też Kapliczka
Garbarnia	2×3	6 d	T	1	futro surowe 0,8 + woda 0,4	futro wyprawione 0,8		Wymaga Chaty nosiwody (woda)
Targ	3×3	6 d		1		zakup narzędzi (20 zł); Saraceni: sprzedaż towarów		
Skład	3×2	4 d		0		pojemność +80 szt. każdego towaru		
Kapliczka	3×3	8 d + 80 zł		1	wosk 0,3	Dobrobyt +10 (1 budynek na 50 os.)		
Bania	2×2	6 d + 40 zł		1	woda 0,3 + pień 0,2	Dobrobyt +6 (1 budynek na 50 os.)		

7. Nacje

Cztery nacje mają to samo szkielet ekonomii (rozdz. §3–§5), ale **inną oś napięcia** i inną walutę haraczu. Poziomy trudności i kolejność kampanii wynikają z pomiarów botami (rozdz. §16.8): Słowianie → Frankowie → Wikingowie → Saraceni; trudniejsza nacja jest odblokowywana później, zamiast wyrównywać liczby na siłę.

Tabela 28. Zasoby startowe i parametry mapy nacji (Data.FACTIONS)

Parametr	Frankowie	Saraceni	Wikingowie	Słowianie
Ludność	14	14	14	14
Deski	30	30	40	40
Kamień	12	—	—	—
Glina	—	12	—	—
Jedzenie startowe	36 (chleb)	36 (daktyle)	36 (ryby)	36 (kasza)
Narzędzia	14	8	14	8
Złoto	80	100	80	80
Drzewa na mapie	300	150	450	300
Dojrzewanie drzewa [min]	5	12	3	6
Złoża (liczba miejsc)	kamień 3, żelazo 2, węgiel 3	glina 2	ruda darniowa (torf) 2	—
Próg gęstości lasu (Puszcza)	— (brak budynków leśnych)	— (brak budynków leśnych)	— (brak budynków leśnych)	200 drzew
Klimat domyślny	umiarkowany	pustynny	śnieżny	Europa Wschodnia
Etap kampanii / trudność	2/4 · ★★☆☆	4/4 · ★★★★★	3/4 · ★★★☆	1/4 · ★☆☆☆

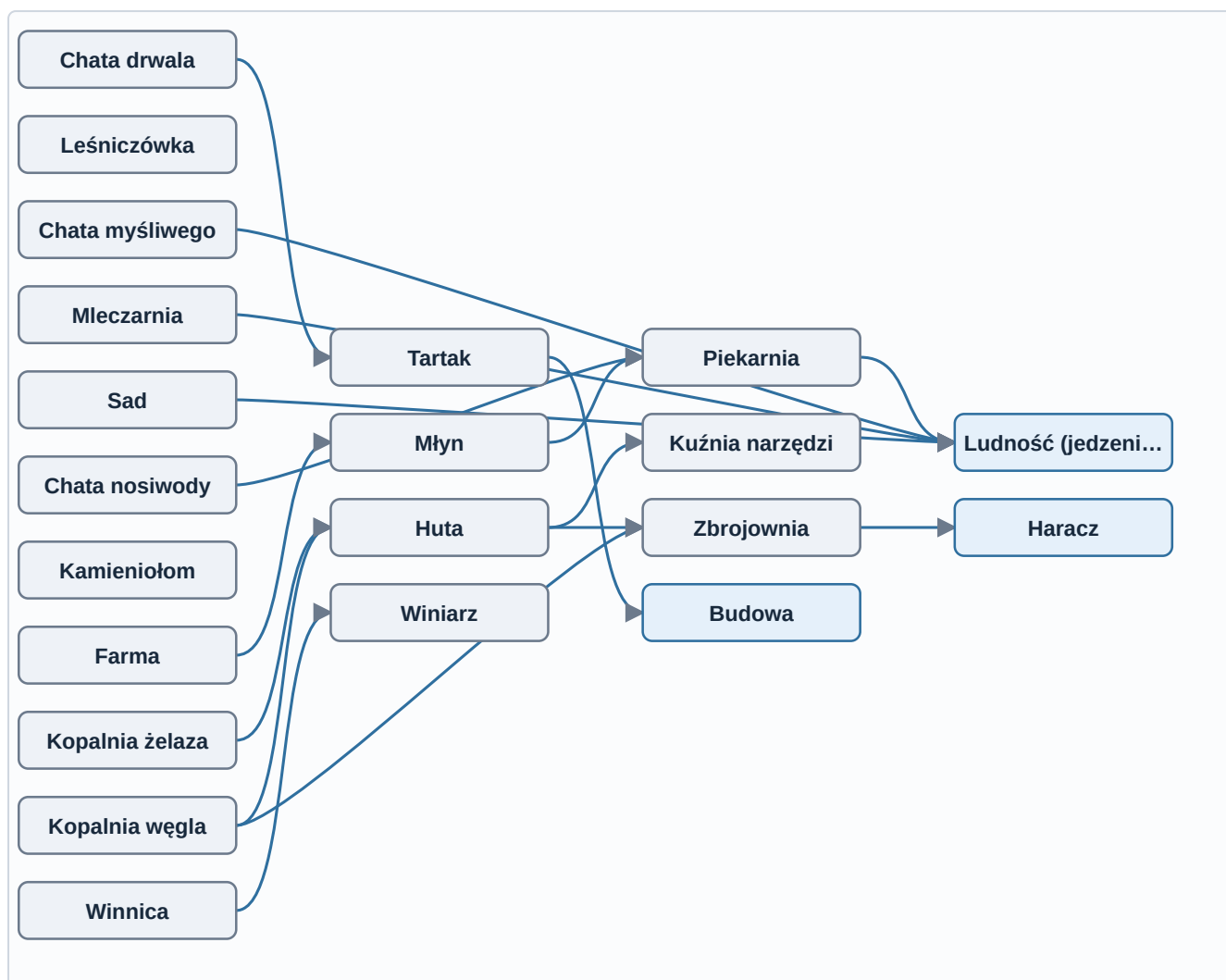
Każda nacja startuje z 14 osobami; podatek poziom 1 i racje ×1; budowniczowie 3 / 2 / 5 (domyślnie / minimum / maksimum); komora Dworu 20 szt. każdego towaru. Mapa musi gwarantować minimum złóż nacji (rozdz. §10), a Wiking dodatkowo startuje zawsze nad morzem.

7.1 Frankowie

Frank jest nacją bez ograniczeń miejsca i bez zależności od mapy: ma najpełniejszy zestaw budynków (23) i jest najbardziej stabilny, ale jego napięcie leży w tym, że **kopalnie zjadają chleb**, a **haracz w żołnierzach wymaga podatku**, który obniża popularność. Etap kampanii 2/4.

Tabela 29. Łańcuchy Franków

Łańcuch	Proporcja	Wyjście	Uwagi
Drewno	1 drwal : 1 leśnik : 1 tartak	2,0 deski/min	3 robotników
Chleb	1 Farma : 1 Młyn : 1 Piekarnia (+ 0,5 nosiwody)	3,3 chleba/min = 19,8 osoby	3,5 robotnika na 3,3 porcji (0,94 na robotnika); wejścia: zboże 1,5, mąka 1,5, woda 0,75
Broń	1 Kopalnia żelaza : 2 Kopalnie węgla : 1 Huta : 1 Zbrojownia	do 1,0 broni/min	5 robotników + 1,5 porcji/min (tylko z nadwyżki); sztuka broni ≈ 6,3 osobominuty
Narzędzia	Huta + Kuźnia narzędzi	do 1,2 narzędzia/min	konkuruje ze Zbrojownią o sztabki i węgiel
Wino	1 Winnica : 1 Winiarz	1,0 wina/min	starcza na 3 kościoły (0,3/min każdy)



Rysunek 5. Łańcuchy produkcji nacji Frankowie (diagram liczony z Data.BUILDINGS; strzałka = towar produkowany przez budynek po lewej i zużywany przez budynek po prawej; skrócone nazwy)

Haracz: żołnierz = 1 broń + 1 osoba + 3 zł żołdu; dostawa wymaga czynnych **Koszar**; złoto na żołd chroni automatyczna rezerwa na najbliższy termin (rozdz. §8). Pojemność przepisu wynosi 9 poziomów, po rozbudowie ludności 13. **Podatek 3 jest pułapką** (popularność 42 < 45 zamraża imigrację), więc podatek 1 lub 2 jest rozsądnym zakresem; Koszary stawiamy najpóźniej 30 minut przed pierwszym terminem, a Doradca przypomina o nich w 30. minucie.

Przepis na gospodarkę (kolejność budowy z testu normalnej gry): (1) dwie Chaty drwala (pierwsza gratis), Tartak, Sad, Chata myśliwego, Kamieniołom, Leśniczówka; (2) Chata nosiwody, Farma, Młyn, Piekarnia, Koszary, dwie Chaty, Mleczarnia, Targ, Skład; (3) druga Leśniczówka, trzeci drwal, Kopalnia żelaza, dwie Kopalnie węgla, Huta, Zbrojownia; (4) drugi Tartak, drugi łańcuch chleba, Chata, Kuźnia narzędzi, kolejne Chaty.

Wynik w grze (bot R bez szumu, haracz Łatwy): ludność @20/40/60/90/150 = 25 / 31 / 43 / 43 / 43, minuta 36 osób: 49, głód 0 min, terminy 6/6, dostarczonych żołnierzy 32, w skarbcu 200 zł. Specyfikacja (model bez przestrzeni) podaje 20 / 22 / 43 / 43 / 39 po haraczu poziomym 3 i 28 żołnierzy; gra jest zgodna co do kryteriów (6/6 terminów, brak głodu, ludność 43), a różnice na początku wynikają z modelu budowy i bramy imigracji.

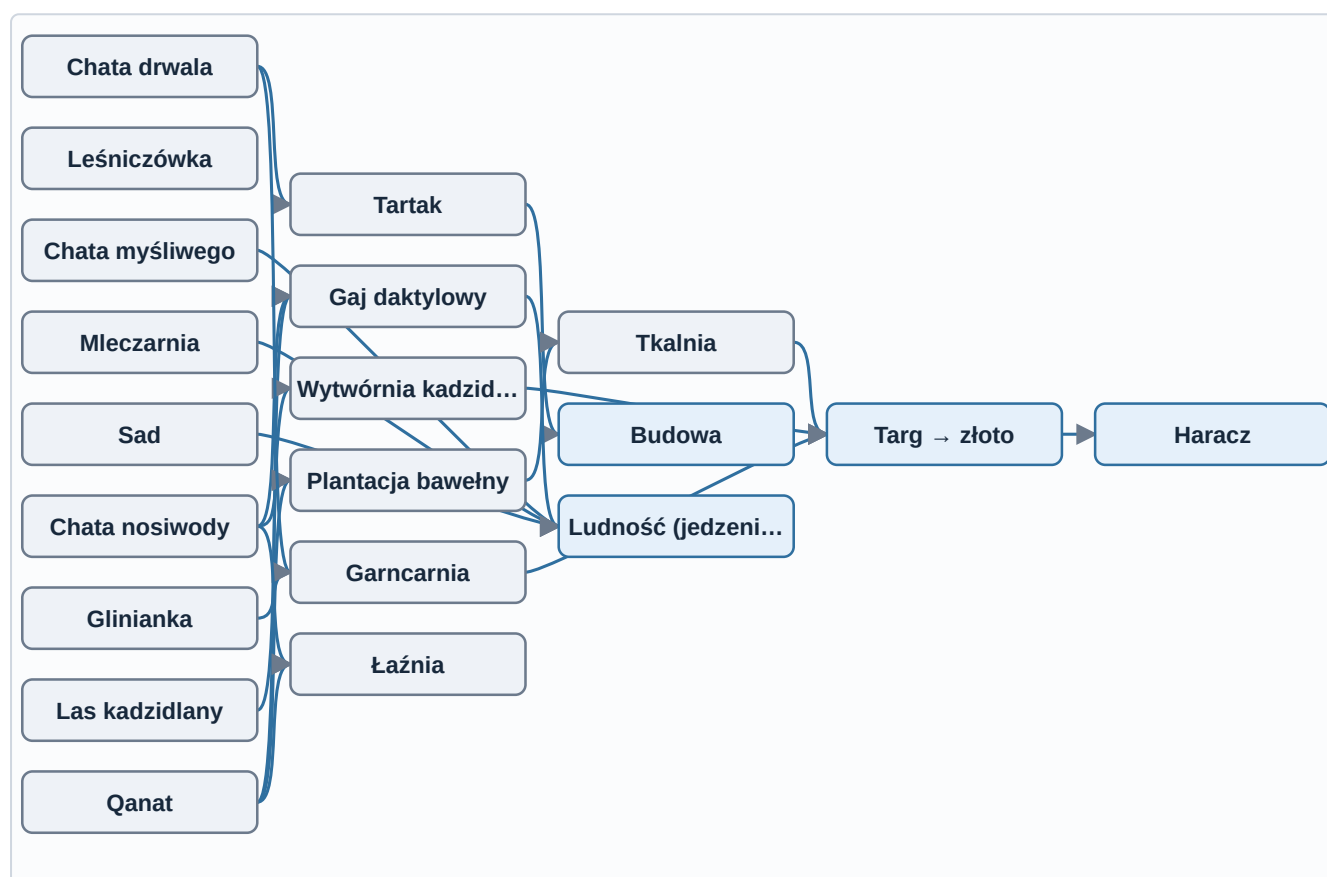
7.2 Saraceni

Saracen płaci haracz **złotem z handlu trzema towarami** (kadzidło, tkanina, ceramika). Jego napięcie leży w wodzie, w daktylach dzielonych między ludzi i Wytwórnię kadzidła, w nasyceniu rynku i w ryzyku napadu na zgromadzone

złoto. To nacja trudniejsza, planowana na późniejszy etap kampanii (4/4): słaby gracz przechodzi u niego poziom 3 w 62% przypadków (Frankowie 83%, Wikingowie 67%, Słowianie 100%).

Tabela 30. Łańcuchy Saracenów

Łańcuch	Proporcja	Wyjście	Uwagi
Daktyl	1 Gaj + 0,4 nosiwody	1,6 porcji/min = 9,6 osoby	1,14 porcji na robotnika
Kadzidło	1 Las kadzidlany : 1 Wytwórnia	1,0 kadzidla/min	Wytwórnia zjada 0,5 porcji/min (z nadwyżki)
Tkanina	1 Plantacja : 1 Tkálnia	1,0 tkaniny/min	nadwyżka 0,2 bawełny/min się marnuje (magazyn 40)
Ceramika	1 Garncarnia + część Glinianki	1,0 ceramiki/min	zużywa 1,0 gliny i 0,3 pnia/min; konkuruje z budową o glinę i drewno
Woda	1 nosiwoda na 2,5 odbiorcy; 1 Qanat na 7	1,5 lub 4,5 wody/min	Qanat opłaca się przy ≥ 6 odbiorcach



Rysunek 6. Łańcuchy produkcji nacji Saraceni (diagram liczony z Data.BUILDINGS; strzałka = towar produkowany przez budynek po lewej i zużywany przez budynek po prawej; skrócone nazwy)

Rynek: Targ kupuje trzy towary po osobnych cenach; każdy towar ma własną krzywą, a sprzedaż odbywa się z całego zapasu naraz (wykres w rozdz. §8). **Karawanseraj** podnosi ceny bazowe i podłogi o 20% i liczy się jak 2 Targi — pojemność przepisu Saracena rośnie z 6 do 8 (drugi Karawanseraj niczego nie dodaje); bez niego Saraceni mieliby najniższy sufit z czterech nacji o ponad 2 poziomy.

Napad na karawanę i Strażnica: im więcej złota w skarbcu, tym większa szansa napadu (rozdz. §9); Strażnica zmniejsza ryzyko za cenę utrzymania. Pierwsze 150 zł jest nietykalne, więc zwykli i słabi gracze napadów nie odczuwają.

Przepis na gospodarkę: (1) dwie Chaty drwala, Tartak, Chata nosiwody, pierwszy Gaj, Glinianka, dwie Leśniczówki, Chata myśliwego; (2) druga nosiwoda, drugi Gaj, Targ, Skład, Las kadzidlany i Wytwórnia, dwie Chaty; (3) Qanat, Plantacja bawełny i Tkálnia, trzeci Gaj, Mleczarnia, Sad, Chata; (4) Meczet, Garncarnia, Łaźnia, druga para Las + Wytwórnia, drugi Targ, Karawanseraj, kolejne Chaty, druga Plantacja i Tkálnia.

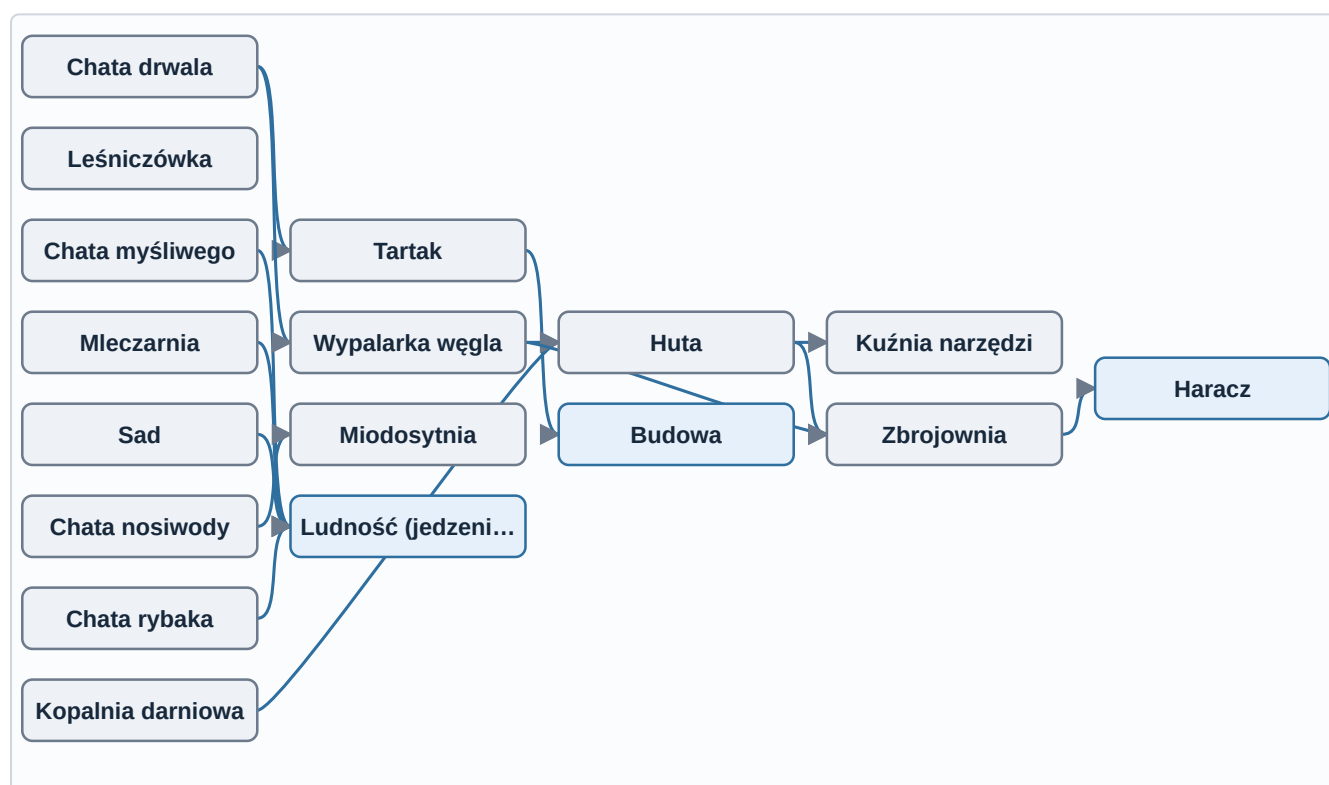
Wynik w grze (bot R, bez szumu, haracz Łatwy): ludność 20 / 38 / 38 / 38 / 38, minuta 36 osób: 35, głód 0 min, terminy 6/6, zapłacono 486 zł haraczu, w skarbcu 702 zł (specyfikacja bez napadów: 891 zł, ludność 20 / 30 / 36 / 36 / 36).

7.3 Wikingowie

Wiking opiera się na **drewnie, morzu i wyprawach**: wyprawy okrętów są rdzeniem jego mechaniki i jedyną walutą haraczu. Startuje wolno (36 osób dopiero ok. 52.–55. minuty), wydaje najwięcej desek i płaci haracz załogą, bronią i miodem pitnym, które okręt zabiera na kilkanaście minut. Węgiel pochodzi wyłącznie z Wypalarek (2 pnie → 1 węgiel), więc broń jest droższa niż u Franka (ok. 9,4 wobec 6,3 osobominuty), ale nie zjada jedzenia kopalń węgla. Ma najdłuższy łańcuch do pierwszej dostawy (Przystań i Okręt budują się ok. 20 minut od zebrania materiałów), więc jego **pierwszy termin przypada najpóźniej: w 110. minucie**. Etap kampanii 3/4.

Tabela 31. Łańcuchy Wikingów

Łańcuch	Proporcja	Wyjście	Uwagi
Drewno	1 drwal : 0,65 leśnika : 1 tartak	2,6 deski/min	450 drzew na mapie, dojrzewanie 3 min
Miód pitny	1 Miodosytnia : 0,33 nosiwody	1,0 miodu pitnego/min	starcza na wyprawy i kościół (miód z wody w jednym budynku — Pasieka usunięta)
Broń	1 Kopalnia darniowa : 2 Wypalarki : 1 Huta : 1 Zbrojownia	do 1,0 broni/min	4 pnie na sztukę; pracuje poniżej połowy mocy
Jedzenie	4–6 Chat rybaka + Myśliwy + Mleczarnia + Sad	7,2–9,4 porcji/min	ryby to ok. 60% dopływu



Rysunek 7. Łańcuchy produkcji nacji Wikingowie (diagram liczony z Data.BUILDINGS; strzałka = towar produkowany przez budynek po lewej i zużywany przez budynek po prawej; skrócone nazwy)

Rybołówstwo. Chata rybaka stoi nad morzem i działa jak pole: ryby są nieskończone, a ogranicza je tylko czas produkcji i **liczba miejsc na wybrzeżu** (domyślnie 6, `settings.shoreSpots`). Przy 3 miejscach ludność spada z 40 do 34, ale Chata myśliwego, Mleczarnia i Sad ją zastępują (ludność 50 w teście). Reguła generatora map: pozycja startowa Wikingów leży zawsze nad morzem, z co najmniej 6 miejscami pod Chatą rybaka i miejscem na Przystań i Okręty.

Wyprawy są deterministyczne i bez losu. Gracz ma dwa rodzaje; czas, załoga i łup dobrano tak, by wielka wyprawa była o ok. 25% tańsza w osobominutach na jednostkę haraczu, ale wymagała dużej osady i dwóch okrętów naraz:

Tabela 32. Wyprawy Wikingów

	Wyprawa zwykła	Wielka wyprawa
Okręty	1	2 jednocześnie
Załoga / miód pitny / broń	6 / 6 / 6	12 / 12 / 12
Czas	10 min	12 min
Straty po powrocie	1 osoba i 1 broń	2 osoby i 2 bronie
Łup	30 zł	80 zł i 2 narzędzia
Liczy się jako	1 wyprawa	3 wyprawy
Koszt w osobominutach	ok. 60 (miód 8 + załoga 60 + broń 9,4 + osoba 10 – łup 27)	ok. 135 (ok. 45 na jednostkę)
Wymagania do wypłynięcia	Przystań, wolny Okręt, 6 miodu i 6 broni w magazynie	Przystań, 2 wolne Okręty, 12 miodu i 12 broni w magazynie
Reguła wysyłki (obie)	po odjeździe zostaje ≥ 10 osób i — przy włączonym „Zapasie załogi” — $\geq 90\%$ robotników potrzebnych budynkom (po odliczeniu budowniczych)	jak obok

Tryb wypraw (przycisk 

Przepis na gospodarke: (1) trzy Chaty drwala (pierwsza gratis), Tartak, Leśniczówka, dwóch rybaków, Chata myśliwego, dwie Chaty; (2) Chata nosiwody, druga Leśniczówka, drugi Tartak, czwarty drwal, trzeci rybak, Mleczarnia, Sad, Targ, Skład; (3) Miodosytnia, Kopalnia darniowa, dwie Wypalarki, Huta, Zbrojownia; (4) Przystań i pierwszy Okręt, Chata, Kuźnia narzędzi, Kościół, czwarta Chata, czwarty rybak, drugi Okręt, kolejne Chaty.

Wynik w grze (bot R, bez szumu, haracz Łatwy): ludność 18 / 32 / 38 / 38 / 38, minuta 36 osób: 54, głód 0 min, terminy 6/6, 22 jednostek haraczu, w skarbcu 1004 zł (specyfikacja: 21 / 32 / 38 / 38 / 38, 11 wypraw, 677 zł).

7.4 Słowianie

Słowianin żyje z lasu i płaci haracz **w towarach**: puszcza daje mu jedzenie, futra, miód i воск, a senior (poborca) zbiera daninę koszykiem tych towarów co 15 minut. Ma 19 budynków i jest nacją najłatwiejszą: bot reaktywny bez przepisu przechodzi u niego poziomy 3–8 w 96–100% przypadków (Frankowie 21%, Saraceni 17% na poziomie 3), więc to **nacja wprowadzająca** (etap kampanii 1/4). Głębia leży gdzie indziej niż u pozostałych: w zdrowiu lasu i w doborze koszyka.

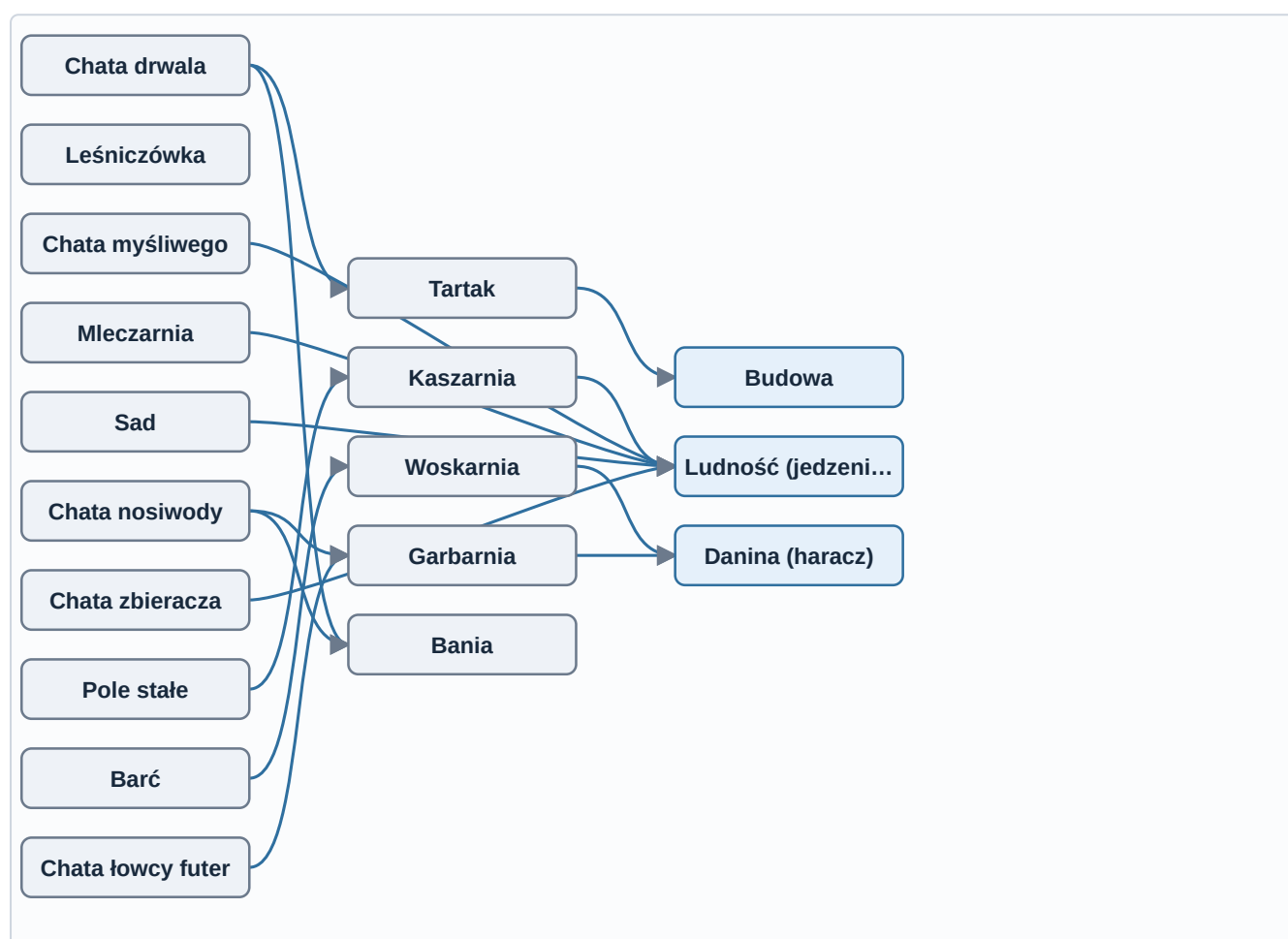
Mechanika własna 1: Puszcza. Las jest żywym zasobem, nie tylko drewnem. Chata zbieracza, Barć i Chata łowcy futer dają plon proporcjonalny do gęstości lasu (Chata myśliwego poluje na zwierzęta i od lasu nie zależy): **plon $\times$ min(1, drzewa / 200)**. Mapa Słowian ma 300 drzew z dojrzewaniem 6 min; poniżej progu plony maleją liniowo (150 drzew = 75% plonu), więc wycinka (Drwal 1,0 pnia/min) konkuruje z daniną, a Leśniczówka (sadzi 1,0 drzewa/min) jest narzędziem gospodarki, nie ozdobą. Test poziomu 5: przepis (2 drwali, 2 leśników) kończy z 299 drzewami i spełnia 6/6 terminów; 5 drwali bez leśników kończy z 32 drzewami, ludność spada do 33 i haracz przechodzi 4 z 6 terminów; na poziomie 3 ten sam błąd daje 11 minut głodu (zbieracze przestają karmić).

Mechanika własna 2: Danina koszykowa. Poborca żąda *wartości*, nie sztuk. Trzy towary mają wartość w osobominutach: futro wyprawione 2,8, miód 1,5, воск 3; udziały w żądaniu: 40% / 35% / 25%. W terminie liczy się łączna wartość dostaw, a każdy z trzech towarów musi pokryć co najmniej 50% swojego udziału; nadwyżka jednego towaru może pokryć resztę niedoboru innego. Towar leży w magazynach do terminu, więc o poziomie 6 i wyżej

decyduje **pojemność** (6 szt. u producenta + 20 w komorze + 80 na Skład) — bez Składu pojemność przepisu spada o 1 poziom.

Tabela 33. Łańcuchy Słowian

Łańcuch	Proporcja	Wyjście	Uwagi
Miód i воск	1 Barć : 1 Woskarnia	0,84 miodu + 0,36 wosku /min	jedna para daje oba towary w proporcji 2,3 : 1; wartość 2,34 osobominuty na 2 osobominuty pracy
Futra	1 Chata łowcy : 1 Garbarnia : 0,27 nosiwody	0,8 futra wyprawionego /min	wartość 2,24 na ok. 2,3 osobominuty; Garbarnia wymaga wody
Jedzenie	3 Chaty zbieracza + Sad + Mleczarnia + 2 Pola + Kaszarnia	7,25 porcji/min	zbieracze to 40% dopływu, więc zależą od lasu
Drewno	2 Drwali : 2 Leśniczówki : 1 Tartak	2,0 deski/min	Tartak 8 d; 300 drzew, dojrzewanie 6 min



Rysunek 8. Łańcuchy produkcji nacji Słowianie (diagram liczony z Data.BUILDINGS; strzałka = towar produkowany przez budynek po lewej i zużywany przez budynek po prawej; skrócone nazwy)

Warstwy mistrzostwa (pojemność przepisu — najwyższy spełniany poziom): przepis nacji 7; + 1 / 2 / 3 pary łańcuchów (futra, miód i воск) 9 / 11 / 12; rozbudowa ludności bez efektu; bez Składu 6; bez Kapliczki i Bani (Dobrobyt kosztuje 1 poziom) 8. **Pole żarowe** (wycinka lasu pod pole) przetestowano i odrzucono: nie zmienia pojemności (7 wobec 7).

Przepis na gospodarkę: (1) dwie Chaty drwala (pierwsza gratis), Tartak, dwie Chaty zbieracza, Leśniczówka, dwie Chaty, Sad, Chata nosiwody; (2) trzecia Chata zbieracza, dwa Pola, Kaszarnia, pierwsza para: Chata łowcy futer + Garbarnia + Barć + Woskarnia; (3) Chata, Mleczarnia, Skład, Targ, druga para łańcuchów, Chata; (4) Kapliczka, Bania, druga Leśniczówka, trzecia para miodu i wosku, dwie Chaty, trzecia para futer.

Wynik w grze (bot R, bez szumu, haracz Łatwy): ludność 32 / 40 / 40 / 40 / 40, minuta 36 osób: 25, głód 0 min, terminy 6/6, 540 osobominut daniny, w skarbcu 236 zł (specyfikacja: ludność 32 / 40 / 40 / 40 / 40, 36 osób w 25. minucie, 540 osobominut: futra 216, miód 189, wosk 135).

7.5 Przepisy botów

Przepis nacji to lista budynków w kolejności budowy, którą wykonują boty R i D (`TestBots.RECIPE`); jest to zarazem „samouczek ekonomii” — preset Łatwy zakłada, że gracz idzie za taką kolejnością (bot reaktywny N bez przepisu przechodzi poziom 3 tylko w 21% u Franków i 17% u Saracenów).

Tabela 34. Przepisy nacji — kolejność budowy botów R/D (`TestBots.RECIPE`); odpowiada rozdziałom „Przepis na gospodarkę” specyfikacji

Nacja	Kolejność budowy		
Frankowie	1. Chata drwala 2. Chata drwala 3. Tartak 4. Sad 5. Chata myśliwego 6. Kamieniołom 7. Leśniczówka 8. Chata nosiwody 9. Farma 10. Młyn 11. Piekarnia	12. Koszary 13. Chata 14. Chata 15. Mleczarnia 16. Targ 17. Skład 18. Leśniczówka 19. Chata drwala 20. Kopalnia żelaza 21. Kopalnia węgla 22. Kopalnia węgla	23. Huta 24. Zbrojownia 25. Tartak 26. Farma 27. Młyn 28. Piekarnia 29. Chata 30. Kuźnia narzędzi 31. Chata
Saraceni	1. Chata drwala 2. Chata drwala 3. Tartak 4. Chata nosiwody 5. Gaj daktylowy 6. Glinianka 7. Leśniczówka 8. Leśniczówka 9. Chata myśliwego 10. Chata nosiwody 11. Gaj daktylowy 12. Targ	13. Skład 14. Las kadzidlany 15. Wytwórnia kadzidla 16. Chata 17. Chata 18. Qanat 19. Plantacja bawełny 20. Tkalnia 21. Gaj daktylowy 22. Mleczarnia 23. Sad 24. Chata	25. Meczet 26. Garncarnia 27. Łaźnia 28. Las kadzidlany 29. Wytwórnia kadzidla 30. Targ 31. Karawanseraj 32. Chata 33. Chata 34. Plantacja bawełny 35. Tkalnia
Wikingowie	1. Chata drwala 2. Chata drwala 3. Chata drwala 4. Tartak 5. Leśniczówka 6. Chata rybaka 7. Chata rybaka 8. Chata myśliwego 9. Chata 10. Chata 11. Chata nosiwody 12. Leśniczówka	13. Tartak 14. Chata drwala 15. Chata rybaka 16. Mleczarnia 17. Sad 18. Targ 19. Skład 20. Miodosytnia 21. Kopalnia darniowa 22. Wypalarka węgla 23. Wypalarka węgla 24. Huta	25. Zbrojownia 26. Przystań 27. Okręt 28. Chata 29. Kuźnia narzędzi 30. Kościół 31. Chata 32. Chata rybaka 33. Okręt 34. Chata
Słowianie	1. Chata drwala 2. Chata drwala 3. Tartak 4. Chata zbieracza 5. Chata zbieracza 6. Leśniczówka 7. Chata 8. Chata 9. Sad 10. Chata nosiwody 11. Chata zbieracza 12. Pole stałe	13. Pole stałe 14. Kaszarnia 15. Chata łowcy futer 16. Garbarnia 17. Barć 18. Woskarnia 19. Chata 20. Mleczarnia 21. Skład 22. Targ 23. Chata łowcy futer 24. Garbarnia	25. Barć 26. Woskarnia 27. Chata 28. Kapliczka 29. Bania 30. Leśniczówka 31. Barć 32. Woskarnia 33. Chata 34. Chata 35. Chata łowcy futer 36. Garbarnia

8. Haracz seniora

Haracz jest **jedynym sposobem wywierania presji** w tej grze: scenarzysta (gracz w menu HUD) wybiera preset trudności albo poziom 1–11, a gra przelicza go na jednostki nacji tak, żeby każda płaciła w przybliżeniu tyle samo pracy. Za niedobór nie ma kary ekonomicznej w locie — niedobór zwiększa licznik braków (reputacja albo porażka misji to decyzja scenariusza). Moduł **Tribute** obsługuje terminy, płatności nacji, rynek Targu i wyprawy Wikingów.

8.1 Poziomy i osobominuty

Poziom **L** żąda **2L osobominut pracy na minutę** (1 osoba pracująca minutę = 1 osobominuta), czyli ok. 5% siły roboczej na poziom przy 42 osobach. Wartość osoby (10 osobominut) i ceny jednostek są założeniami modelu. Nacje płacą różnymi jednostkami:

Tabela 35. Waluta haraczu według nacji

	Frankowie	Saraceni	Wikingowie	Słowianie
Jednostka	żołnierz	złoto	wyprawa okrętu	koszyk daniny: futro wyprawione, miód, воск
Koszt jednostki	ok. 19 osobominut: broń 6,3 + żołd 3 zł (2,7) + osoba 10	1 zł ≈ 1,1 osobominuty	ok. 60: załoga 60 + miód 8 + broń 9,4 + osoba 10 – łąp 27	wartość towaru w osobominutach: futro 2,8, miód 1,5, воск 3,0
Warunek dostawy	czynne Koszary, broń, 3 zł żołdu na żołnierza, ludność ponad 10	złoto w skarbcu	Przystań i Okręt (wielka: 2 Okręty), miód pitny, broń i załoga	towary w magazynach w terminie; łączna wartość ≥ żądanie, każdy towar ≥ 50% swojego udziału
Pierwszy termin	60. min	50. min	110. min	60. min

Pierwszy termin zależy od czasu dojścia do pierwszej dostawy: Wiking potrzebuje Miodosytni, nosiwody, Wypalarek, Huty, Zbrojowni, Przystani i Okrętu, więc dostaje 50 minut więcej niż Frank. Przesunięcie z 95. na 110. minutę podniosło odsetek przejść poziomów 7 / 8 przez zwykłego gracza z 67 / 33% do 83 / 75%.

8.1.1 Skala poziomów

Wartość żądana na 15 minut (jedna „rata”), wg tabeli **Tribute.RATE** :

Tabela 36. Skala poziomów haraczu: wartość żądana co 15 min (jednostki: żołnierze / zł / wyprawy / osobominuty daniny)

Poziom	Frankowie [żołn.]	Saraceni [zł]	Wikingowie [wypr.]	Słowianie [os.min.]	Osobominuty
1	1,6	27	0,5	30	30
2	3,2	54	1	60	60
3	4,7	81	1,5	90	90
4	6,3	108	2	120	120
5	7,9	135	2,5	150	150
6	9,5	162	3	180	180
7	11,1	189	3,5	210	210
8	12,6	216	4	240	240
9	14,2	243	4,5	270	270
10	15,8	270	5	300	300
11	17,4	297	5,5	330	330

Wykres poniżej pokazuje, że po przeliczeniu na osobominuty wszystkie cztery nacje płacą tyle samo pracy na każdym poziomie (linie nakładają się na prostą 30 × poziom):



Rysunek 9. Skala haraczu w osobominutach na 15 min: każda nacja płaci tyle samo pracy ($30 \times$ poziom), choć w innych jednostkach

8.2 Presety i terminy

Presety dobrano tak, żeby przechodził je podobny odsetek casualowych graczy w każdej nacji (wyniki w aneksie [5A](#)). Poziom Wikinga w presecie Mistrz (12) wykracza poza tabelę — żądanie liczone jest liniowo od poziomu 1 ($\text{RATE}[1] \times L$).

Tabela 37. Presety trudności haraczu: poziom stały od pierwszego terminu

Preset	Frankowie	Saraceni	Wikingowie	Słowianie
Łatwy	3	3	3	3
Normalny	5	5	5	6
Trudny	6	6	6	8
Mistrz	10	7	12	11
Własny	1–11	1–11	1–11	1–11

Tabela 38. Terminy haraczu

Parametr	Frankowie	Saraceni	Wikingowie	Słowianie
Pierwszy termin [min]	60	50	110	60
Kolejne terminy	co 15 min	co 15 min	co 15 min	co 15 min
Ogłoszenie	30 min przed pierwszym terminem	30 min przed pierwszym terminem	30 min przed pierwszym terminem	30 min przed pierwszym terminem
Jednostka żądania	żołn.	zł	wypr.	os.min.
Lista budynków łańcucha w ogłoszeniu	Koszary, Kopalnia żelaza, Kopalnia węgla, Huta, Zbrojownia	Targ, Chata nosiwody, Las kadzidlany, Wytwórnia kadzidła	Miodosytnia, Chata nosiwody, Kopalnia darniowa, Wypalarka węgla, Huta, Zbrojownia, Przysań, Okręt	Barć, Woskarnia, Chata łowcy futer, Garbarnia, Chata nosiwody, Skład

Terminy co 15 minut, żądanie jest **skumulowane**: w terminie nr n liczy się łączna dostawa od początku haraczu względem $\lfloor \text{rata} \times n \rfloor$ (wyprawy zaokrągla się w dół), a braki przechodzą na następny termin. **Ogłoszenie** pojawia się 30 minut przed pierwszym terminem — komunikat z listą budynków łańcucha, których jeszcze nie ma (`Tribute.NEED_LIST`), ponawiany w Doradcy aż do ukończenia łańcucha. Wynik każdego terminu zapisuje `tribute.log (n, czas, wymagane, dostarczone, spełnione)`, a liczniki `met` i `missed` pokazuje panel Haracz.

SPECYFIKACJA

Kryterium przejścia poziomu w testach: co najmniej 80% terminów z żądaniem spełnionych, spełniony ostatni termin, najwyżej 10 min głodu i co najmniej 24 osoby na końcu. Ocena na 6 terminach po pierwszym (okno 150 / 140 / 200 / 150 min dla Franków / Saracenów / Wikingów / Słowian — `TestBots.EVAL_END`).

8.3 Frankowie: żołnierze

Gdy haracz jest ogłoszony, a **Koszary są obsadzone**, co minutę przybywa do 2 żołnierzy (`soldierFrac += 2 × dt`), dopóki dostawa nie pokryje żądania na najbliższy termin. Każdy żołnierz zużywa **1 broń**, **3 zł żołądu** i **1 osobę** (`ludność - 1`), pod warunkiem że ludność wynosi więcej niż 10; gdy któregoś brakuje, dostawy wstrzymuje licznik. Nie ma losu — jeśli broń, złoto i ludzie są, żołnierz jest dostarczony.

Rezerwa złota (`soldierReserve`): po ogłoszeniu haraczowi zakup narzędzia na Targu i otwarcie placu z kosztem w złocie nie mogą zejść poniżej $3 \text{ zł} \times \text{brakujący żołnierze do najbliższego terminu}$. To zawór bezpieczeństwa Franka przed „pułapką złota” — bez niego casual wydawał żołąd na budynki. Żołąd wiąże haracz z podatkami: poziom wymaga podatku ≥ 1 , a bardzo wysokie poziomy (≥ 10) — rozbudowy ludności, podatku 2 i wina (Dobrobyt).

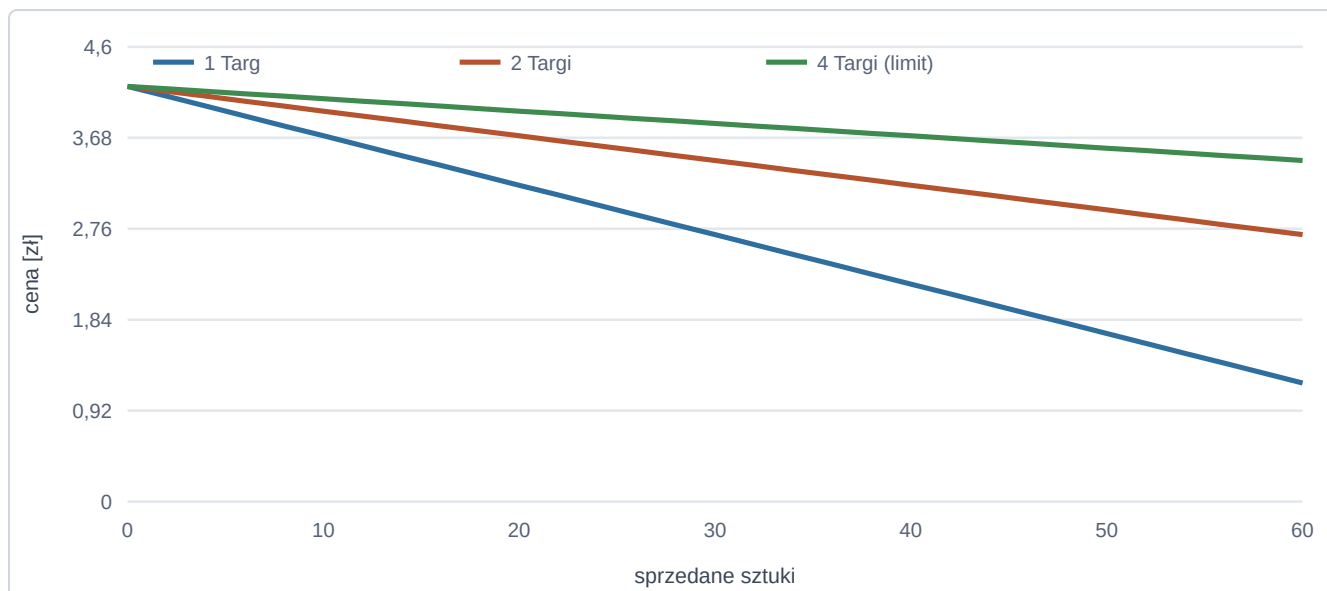
8.4 Saraceni: złoto i rynek

W terminie skarbiec płaci $\min(\text{złoto}, \text{brakująca część żądania})$; zapłacone złoto zwiększa `delivered` (płatność częściowa się liczy, braki przechodzą dalej). Źródłem złota jest **sprzedaż na Targu** — czynnym Targ (lub Karawanseraj) sprzedaje **z całego zapasu naraz** trzy towary po osobnych cenach:

Tabela 39. Ceny towarów handlowych Saracenów (`Tribute.TRADE`); spadek 0,05 zł za sztukę ÷ liczba Targów, odnowa 1,43% luki na minutę

Towar	Cena bazowa	Podłoga	Baza z Karawanserajem (+20%)	Podłoga z Karawanserajem
kadzidło	4,2 zł	0,9 zł	5,04 zł	1,08 zł
tkanina	3,6 zł	0,9 zł	4,32 zł	1,08 zł
ceramika	2,4 zł	0,6 zł	2,88 zł	0,72 zł

Algorytm (`marketTick`): liczba efektywnych Targów $nM = \text{Targi} + \min(2, 2 \times \text{Karawanseraje})$; mnożnik cen 1,2 przy dowolnym Karawanseraju (bez kumulacji). Co tick cena odnawia się o 1,43% luki do ceny bazowej na minutę; zapas towaru jest sprzedawany **po jednej sztuce**, każda po aktualnej cenie (nie niższej niż podłoga), a po każdej sztuce cena spada o $0,05 / nM$. Ponieważ boty i gracz sprzedają zapas na bieżąco, krzywa sprowadza się do równowagi „dochód $\approx$ odnowa”; w normalnej grze po 150 minutach wszystkie ceny leżą przy podłodze, więc kolejna wytwórnia tego samego towaru nic nie daje, a kolejny towar albo drugi Targ już tak.



Rysunek 10. Cena kadzidła po sprzedaży n sztuk naraz: 1 Targ, 2 Targi (lub Karawanseraj), z odnową pomijaną (Tribute.TRADE)

DECYZJA PROJEKTOWA

Ceny w specyfikacji są o 40% niższe niż w pierwszej wersji (trzy towary po dawnych cenach dawały 2500 zł nadwyżki przy zerowym haraczu). Karawanseraj jest świadomym pogłębieniem handlu: podnosi bazę i podłogi o 20% i liczy się jak 2 Targi.

8.5 Wikingowie: wyprawy

`voyagesTick` działa niezależnie od haraczu (wyprawy przynoszą też łup): **po powrocie** statku (czas `until`) załoga wraca do puli, ludność maleje o stratę (1 osoba, wielka 2), magazyn traci broń (1 / 2), skarbiec dostaje łup (30 zł; wielka 80 zł i 2 narzędzia), a do haraczu doliczają się 1 lub 3 wyprawy. Wypłynięcie jest automatyczne, gdy stoi Przystań, jest wolny Okręt, a magazyn zawiera miód pitny i broń na wyprawę oraz spełniona jest reguła wysyłki (rozdz. §7.3). W trybie „Mieszane” najpierw próbowana jest wielka wyprawa na dwóch wolnych okrętach, potem zwykła na jednym. Załoga na morzu (`crewAway`) nie pracuje i nie liczy się do wolnych ludzi.

8.6 Słowianie: danina koszykowa

W terminie `settleBasket` rozlicza **brakującą wartość** `D` = żądanie skumulowane – dotychczas dostarczone:

- dla każdego z trzech towarów liczy jego udział `udział × D` (futro wyprawione 40%, miód 35%, воск 25%) i bierze z magazynu tyle sztuk, ile pokrywa ten udział (lub ile jest);
- jeśli któryś towar pokrył mniej niż **50% swojego udziału**, termin nie może być spełniony (`minOk = false`);
- gdy po kroku 1 nadal brakuje wartości, **nadwyżka** dowolnego towaru (ponad to, co już wzięto) zasila niedobór — najpierw ta o największej wartości;
- pobrane towary znikają z magazynu, ich wartość trafia do `delivered` i `byGood` ; termin jest spełniony, gdy łączna zapłata $\geq D$ i `minOk` .

Brak któregośkolwiek towaru (np. воск spalany przez Kapliczkę, brak Woskarni) daje 0/6 terminów — spec. potwierdza to scenariuszami W4–W7 i W11 (aneks §B). Panel Haracz pokazuje `koszyk` : stan magazynu względem wymaganego i minimalnego udziału każdego towaru.

8.7 Pojemność przepisu

Pojemność to najwyższy poziom, który zdrowa osada spełnia na 6 terminach po pierwszym. Są dwie wartości: stały *przepis nacji* i *przepis po rozbudowie ludności* (3 Chaty, Sad, Mleczarnia, Myśliwy na każdy krok), bo przepis jest ograniczony liczbą chat.

Tabela 40. Pojemność haraczu (specyfikacja v3.6)

Nacja	Przepis nacji	Po rozbudowie ludności	Co ogranicza
Frankowie	9	13 (11 po jednym kroku)	ludność: żołnierz zabiera osobę, a żołd bierze złoto z podatków
Saraceni	8 (6 bez Karawanseraju)	8 (7 po jednym kroku)	nasycenie rynku trzech towarów; rozbudowa ludności nic nie daje
Wikingowie, wyprawy zwykłe	7	13 (10 po jednym kroku)	miód pitny i ludność
Wikingowie, tryb „Mieszane”	8	16 (11 po jednym kroku)	ludność i liczba okrętów (parzysta)
Słowianie	7	12 po trzech parach łańcuchów (9 / 11 po jednej i dwóch)	liczba par futer, miodu i wosku oraz pojemność magazynów

Głębia jest inna w każdej nacji: Frank mierzy się z ludnością (rozbudowa daje +4 poziomy, podatek 3 jest pułapką), Saracen z handlem (trzy towary i drugi rynek dają +6 poziomów, a Dobrobyt i Strażnica kosztują po 1), Wiking z logistyką wypraw (wielka wyprawa daje do +3 poziomów po rozbudowie osady), a Słowianin z lasem i koszykiem daniny (każda para łańcuchów daje ok. +2 poziomy, Skład +1).

8.8 Doradca a haracz

`Advisor.plan(s)` podaje dla wybranego poziomu **listę budynków** potrzebnych do jego spełnienia (np. Frank: Koszary 1, Zbrojownia / Huta / Kopalnia żelaza [żołnierze/min], Kopalnie węgla ×2, Piekarnia; Saracen: Targ, nosiwoda, Las kadzidlany i Wytwórnia, od poziomu 3 druga para, od 4 bawełna i Tkalnia, od 5 Garncarnia, od 7 Karawanseraj i drugi Targ; Wiking: Przystań, Miodosytnia, nosiwoda, Kopalnia darniowa, 2 Wypalarki, Huta, Zbrojownia, Okręty wg poziomu; Słowianin: pary Barć + Woskarnia i Chata łowcy + Garbarnia wg [2L / 4,58]) z licznikiem „mam / potrzeba” oraz notatki o ograniczeniach. Dodatkowo `Advisor.warnings` ostrzega przed brakiem Koszar (od 30 min przed terminem), brakami łańcucha i zbyt dużym zapasem złota Saracena (napad).

9. Zdarzenia i kryzysy

Moduł `Events` odpowiada za dwie rzeczy: **napady na karawanę** (Saraceni, losowe) i **kryzysy** wg harmonogramu (zaraza, pożary, krach cen, wylesienie) — scenariusze z kryteriów akceptacji, które sprawdzają, czy gra „da się uratować” po katastrofie. Silnik pozostaje deterministyczny: jedynym losowaniem jest napad na karawanę, rozstrzygany globalnym `RNG` z ziarnem.

9.1 Napad na karawanę

Dotyczy tylko **Saracenów** i można go wyłączyć w scenariuszu (przycisk  w HUD — `settings.raids`; dla presetu Łatwy, jeśli samouczek tego wymaga). Gromadzenie złota jest u Saracenów ryzykowne: im więcej złota w skarbcu, tym większa szansa napadu, a Strażnica zmniejsza ryzyko za cenę stałego utrzymania. To dynamiczny balans — napad dotyka tylko tego, kto ma nadwyżkę, a gracz w kłopotach go nie czuje.

Tabela 41. Reguły napadu na karawanę (`Events.tick`)

Zasada	Wartość
Sprawdzenie napadu	co 10 min, od 40. minuty (<code>raids.next</code>)
Złoto bezpieczne	pierwsze 150 zł jest nietykalne (przy złocie ≤ 150 zł napadu nie ma)
Szansa napadu	$\min(0,5; (\text{złoto} - 150) / 1000) \times (1 - 0,40 \times \text{liczba czynnych Strażnic})$
Łup	$\min(400 \text{ zł}; 0,30 \times (\text{złoto} - 150) \times (1 - 0,25 \times \text{liczba czynnych Strażnic}))$
Strażnica	8 d + 6 gl + 40 zł, 1 pracownik, bez narzędzia; utrzymanie 0,6 zł/min; max 2 liczą się w sumie
Utrzymanie	gdy złota brak, Strażnica nie działa w tym ticku (nie liczy się jako czynna)

Szansa jest porównywana z jednym losowaniem `RNG.next()` na sprawdzenie (losowanie następuje tylko wtedy, gdy złoto przekracza 150 zł). Skutek: `złoto -= łup`, `raids.count++`, `raids.lost += łup` i komunikat „✖ Napad na karawanę! Zrabowano N zł” (ze wzmianką o Strażnicach albo podpowiedzią „zbuduj Strażnicę!”).

SPECYFIKACJA

Wyniki testu (12 ziaren, mistrz, poziom 3, zapas ok. 890 zł): bez napadów 891 zł; z napadami bez Strażnicy 586 zł (strata 305 zł, 2,7 napadu); z jedną Strażnicą 620 zł (strata 139 zł + 48 zł utrzymania); z dwiema 593 zł (strata 33 zł + 93 zł utrzymania). Strażnica zwraca się więc przy zapasie powyżej ok. 500 zł. Przy poziomie 6 (zapas ok. 400 zł) każda Strażnica obniża końcowe złoto (340 / 260 / 139 zł), bo koszt utrzymania przewyższa straty. Zwykli i słabi gracze napadów nie odczuwają (rzadko mają więcej niż 150 zł); przy poziomie maksymalnym napad nie ma czym zaszkodzić (pojemność 8 w każdym ziarnie), a Strażnica odbiera po 1 poziomie na sztukę.

Gracz ma więc wybór: Strażnice przy dużym zapasie i niskim poziomie haraczu, brak Strażnic przy wysokim. Ostrzeżenie na 2 minuty przed napadem i napad na samą sprzedaż nie zostały zamodelowane (propozycja „wersji 2” mechanizmu).

W grze statystyka napadów jest w panelu **NAPADY** (liczba napadów, strata, czynne Strażnice), a Doradca informuje, gdy skarbiec przekracza 300 zł. Skrypt `tools/raids.js` zbiera rozkład strat po wielu ziarnach (ze Strażnicami i bez).

9.2 Kryzysy wg harmonogramu

Przycisk  **Kryzysy** włącza `Events.setCrisis(true)`; od tej chwili `runCrisis` sprawdza w każdym ticku listę zdarzeń nacji i wykonuje te, których czas już minął. Scenariusze pochodzą z kryteriów akceptacji (v3.5):

Tabela 42. Harmonogram kryzysów (Events.SCHEDULE; włączany przyciskiem  Kryzysy)

Nacja	Minuta	Zdarzenie
Frankowie	60	pożar: Piekarnia, Młyn, Farma, Sad (po jednym) + zaraza (-12 osób)
Saraceni	60	pożar: 2 Gaje daktylowe i Qanat
Saraceni	70	krach cen: kadzidło i tkanina 0,9 zł, ceramika 0,6 zł
Wikingowie	90	pożar: Przysań i Okręt
Słowianie	60	zaraza (-12 osób)
Słowianie	70	pożar lasu (-60% drzew) + pożar 3 Barci i 3 Woskarni

Rodzaje zdarzeń:

- **Zaraza** (`plague`): ludność -12, nie poniżej 10.
- **Pożar** (`fire`): z listy typów budynków pali się **po jednym budynku każdego typu** (pierwszy znaleziony): zwalnia pola obrysu i usuwa budynek (`Events.destroy`) — bez zwrotu materiałów i narzędzia; odbudowa jest zwykłą budową.
- **Krach cen** (`priceCrash`): ceny trzech towarów handlowych spadają na podłogę krzywej (0,9 / 0,9 / 0,6 zł) i odnawiają się 1,43% luki na minutę.
- **Wylesienie** (`deforest`) i **pożar lasu** (`forestFire`): ścina zadaną liczbę lub 60% drzew; plony leśne Słowian maleją liniowo, gdy drzew < 200, a Leśniczówki odtwarzają las.

Każde zdarzenie dopisuje komunikat do `eventMsgs`, który `UI.refreshHUD` pokazuje w podpowiedzi u dołu ekranu.

Kryteria akceptacji wymagają, by po każdym z tych kryzysów gra dała się uratować bez „cheatów”: np. brak żywności do 40. minuty i późniejsza odbudowa kończy się u Franka głodem 31 min, ludnością 21 na końcu i terminami 6/6 (przy głodzie > 10 min przejście liczy się jako niezaliczone), zaraza nie zostawia trwałej straty, a pożar łańcucha produkcji (chleb i Sad; 2 Gaje i Qanat; Przysań i Okręt albo Miodosytnia) daje głód 0 i haracz 6/6. Pełne tabele wyników są w aneksie [SA](#), a bieżące wyniki scenariuszy w aneksie [SB](#).

10. Teren, klimaty i mapy

Teren to warstwa przestrzenna dodana do ekonomii w Fazie 5 („teren v2”). Jej zasada: **klimat zmienia teren, wygląd i faunę — nie ekonomię**. Te same liczby (koszty, tempa, haracz) obowiązują na każdej mapie; mapa wpływa na to, gdzie da się budować, jak daleko jest do surowców i czy trzeba iść brodem. Generator jest deterministyczny: ta sama nacja, klimat, typ i ziarno dają tę samą mapę.

10.1 Mapa i kafle

Mapa ma **48 × 48 pól** (`MapGen.SIZE`). Dwór (4×4) stoi pośrodku: jego obrys zaczyna się w polu (22, 22), środek planu to (24, 24). Pole (kafel) ma strukturę opisaną w rozdz. §2.3: `h` (0 woda, 1 łąd, 2 skała), `e` (poziom wysokości 1–4), `wk` (rodzaj wody), `k` (cecha terenu), `trees`, `deposit`, `occup`, `road`, `pad`. Po wygenerowaniu mapa dostaje `map.meta: climate, type, seed, fid, attempt, valid, why` (lista powodów odrzucenia), `stats` (liczniki z walidacji) oraz `river` (punkty rzeki i brody) i `fx` (parametry wizualne).

10.2 Klimaty i typy map

Cztery klimaty × sześć typów map daje 24 kombinacje (Wikingowie zawsze dostają mapę nadmorską, więc dla nich 4). Skrót `?play=nacja&climate=...&map=...&seed=N` uruchamia dowolną; domyślnie nacja startuje w swoim klimacie (Frankowie: umiarkowany, Saraceni: pustynny, Wikingowie: śnieżny, Słowianie: Europa Wschodnia), na mapie nadmorskiej i ze stałym ziarnem (777 / 888 / 999 / 1111) — układ drzew i złóż jak w pierwszych wersjach.

Tabela 43. Klimaty (Data.CLIMATES): zmieniają teren, wygląd i faunę — nie ekonomię

Klimat	Region	Nacja natywna	Grunt	amp	bagna	zasy	wydm	plaży ruchome	oazy	lód	Opis
Umiarkowany	Europa Zachodnia	Frankowie	grass	1	0,02	0	0	0	—	nie	Łąki i dąbrowy, łagodne wzgórza, błękitne rzeki z brodami.
Europa Wschodnia	Puszcza i step	Słowianie	grass	0,7	0,05	0	0	0	—	nie	Puszcza i brzozy, rozlewiska, wolne rzeki ze starorzeczami.
Śnieżny	Północ	Wikingowie	snow	1,15	0,01	0,05	0	0	—	tak	Śnieg i sosny, zamrożone jeziora, fiordy i lodowe klify.
Pustynny	Bliski Wschód	Saraceni	sand	0,9	0	0	0,18	0,01	2	nie	Wydm, oazy, dolina rzeki, mesy i ruchome piaski.

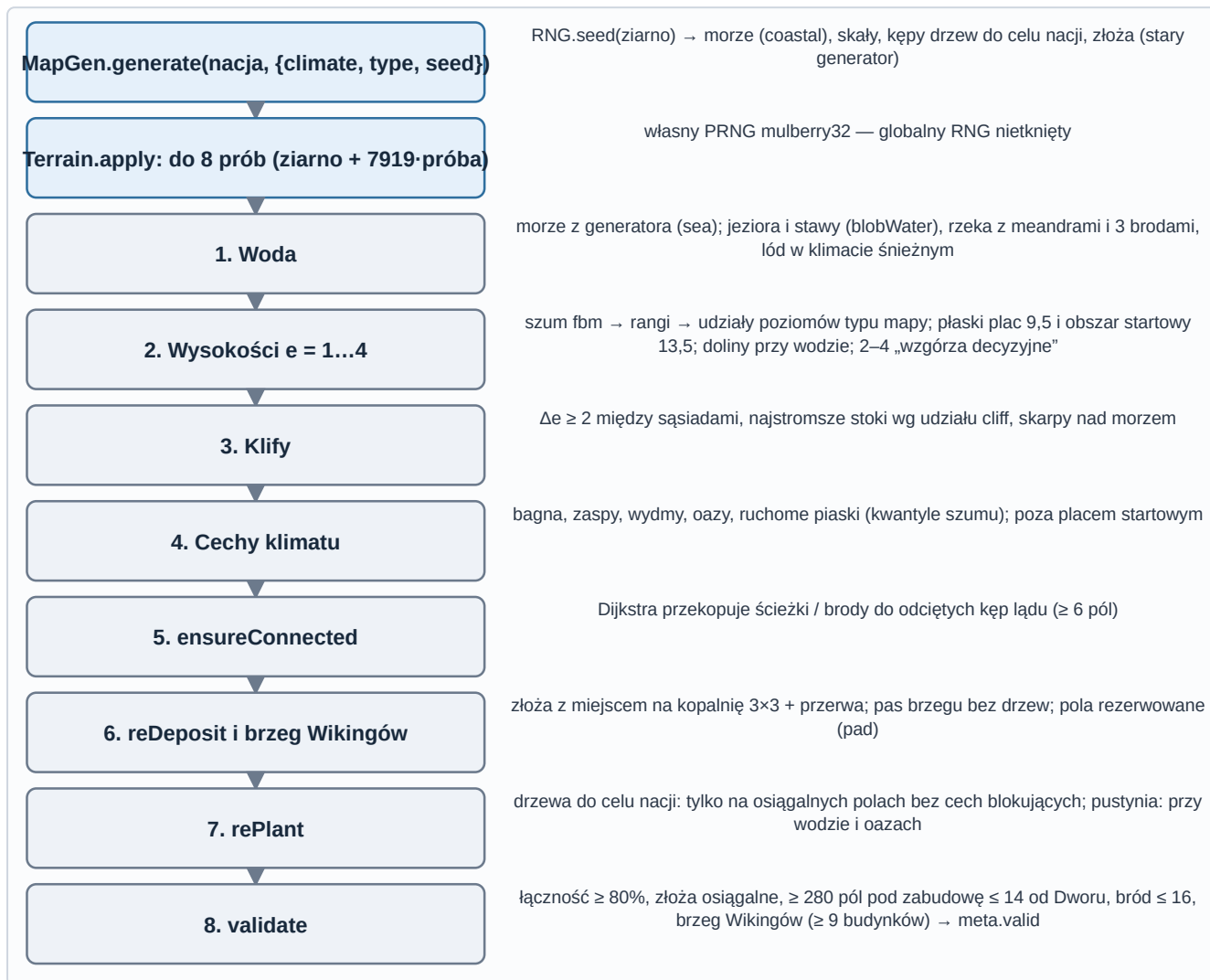
Tabela 44. Typy map (Data.MAPTYPES). `shares` = udział poziomów wysokości; `cliff` = udział stromych stoków zamienianych w klify

Typ	Morze	Poziomy (udział)	cliff	amp	grzbiety	rzeka	jeziora	stawy	bagna ×	Opis
Nadmorska	tak	72% / 28%	0,06	0,55	0,05	—	0–1	1–3	1	Morze przy zachodniej krawędzi, plaże, zatoki i klify.
Nizinna	nie	66% / 34%	0,04	0,5	0,1	—	0–1	2–4	1	Łagodne wzgórza, stawy i dużo miejsca pod zabudowę.
Rzeczna	nie	70% / 30%	0,05	0,55	0,1	tak	0–1	0–2	1	Rzeka z meandrami i brodami — przeszkoda i szlak.
Jeziorna	nie	68% / 32%	0,04	0,55	0,1	—	3–5	1–3	1,2	Kilka jezior, półwyspy i przesmyki.
Górzysta	nie	30% / 28% / 24% / 18%	0,3	1	0,6	—	0–1	0–1	0,5	Grzbiety, płaskowyże i klify; ruch tylko przełęczami.
Bagnista	nie	100%	0	0,25	0	—	1–2	4–7	6	Rozległe bagna z wyspami suchego łądu.

Nazwy typów map zależą od klimatu (`Data.mapTypeName`): np. w pustyni „rzeczna” to „Dolina rzeki”, „jeziorna” — „Oazy”, „bagnista” — „Wydm i oazy”, „górzysta” — „Mesy i kaniony”; w śniegu „nadmorska” to „Fiordy”, „jeziorna” — „Jeziorna (zamarznięta)”, „bagnista” — „Torfowiska”; w Europie Wschodniej „bagnista” to „Rozlewiska”, a „nizinna” — „Step i puszcza”.

10.3 Potok generatora

`MapGen.generate(fid, opts)` buduje najpierw bazę starym generatorem (zasiewa globalny RNG): pas morza przy zachodniej krawędzi (tylko mapa nadmorska; dla Wikingów szeroki — start zawsze nad morzem), skały (nie bliżej niż 9 pól od środka), kępy drzew, złoża (gwarantowana liczba miejsc z danych nacji, poza placem Dworu), wyczyszczony plac 8×8 pod Dwór i dopasowanie liczby drzew do celu nacji (300 / 150 / 450 / 300). Potem `Terrain.apply` nakłada teren v2 **własnym PRNG** (`Terrain.prng`), więc dotychczasowy układ drzew i złożów oraz przeplot losowań logiki pozostają takie jak dawniej.



Rysunek 11. Potok generatora terenu v2: stary generator daje bazę (morze, skały, drzewa, złoża), Terrain.apply nakłada wodę, wysokości, klify i cechy klimatu, po czym naprawia łączność i waliduje wynik.

`Terrain.apply` próbuje **do 8 razy** (ziarno próby: `hash(klimat|typ) * (ziarno + 7919 * próba)`); pierwsza próba, która przejdzie walidację, wygrywa, a gdy żadna nie przejdzie, mapa zostaje z `meta.valid = false` i listą powodów (testy wymagają `valid` dla całej macierzy `nacja × klimat × typ × ziarno`).

10.3.1 Woda

- **Morze** (`sea`) — pas z generatora; w klimacie śnieżnym przy brzegu pojawia się lód.
- **Jeziora i stawy** (`blobWater`): nieregularne plamy o promieniu 2,6–4,2 (typ „jeziorna”: 3,2–6,0) i 1,3–2,3 (stawy); liczby wg typu mapy; nie bliżej Dworu niż `plac + promień + 2,5`, nie na istniejącej wodzie. W klimacie śnieżnym zamarzają (`ice` — **przechodnie**), w pustynnym każde jezioro i staw otacza oaza.
- **Rzeka** (`makeRiver`, typ „rzeczna”): meandrująca krzywa w poprzek mapy w odległości $\geq 9,5 + 1,6$ pola od Dworu, szerokość 1,6–2,6; **trzy brody** (`ford`, przechodnie płycizny): najbliższy Dworowi punkt rzeki i po jednym w

odległości 24–34 punktów wzdłuż biegu.

10.3.2 Wysokości, płaski start i wzgórze

Wysokość `e` powstaje z szumu `f1m`: pola lądu są szeregowane rangą i dzielone na poziomy wg udziałów typu mapy (`shares`; typ „górzysty” ma 4 poziomy, „bagnista” 1). Potem: (1) **płaski plac** wokół Dworu (promień 9,5 pola) i **obszar początkowej zabudowy** (do 13,5 pola wszystko ma `e = 1` — nic nie utrudnia pierwszych budynków, wzgórza zaczynają się dalej); (2) niziny przy wodzie (plaże, doliny rzek); (3) usunięcie pojedynczych kolców wysokości; (4) „**wzgórza decyzyjne**”: 2–4 płaskowzgórza (`e = 2`) w pierścieniu 16–24 pól od Dworu (0 w typie „górzystym”, 1 w „bagnistym”), których stoki wymagają wyrównania (rozdz. §11) — wpływają na wybór miejsc dopiero po rozbudowie osady.

10.3.3 Klify

Pole lądu, którego sąsiad leży o co najmniej 2 poziomy niżej, zostaje **klifem** (`cliff`); dodatkowo najbardziej strome stoki (udział `cliff` typu mapy: 0–30%) oraz skarpy nad morzem (z szumem) — wszystko z dala od Dworu. Klif nie jest przechodni ani budowlany.

10.3.4 Cechy klimatu

Cechy blokujące zabudowę wybierane są z kwantyli szumu spośród pól lądu poza placem startowym i poza pasem 2 pól od wody w pobliżu Dworu (żeby zostało miejsce pod Przystań, Okręt i rybaków):

- **bagno** (`bog`, na poziomie 1): udział `bog` × mnożnik typu (typ „bagnista”: co najmniej 22%, maks. 45%),
- **zasy** (`drift`, na poziomie ≤ 2) — klimat śnieżny, **wydm** (`dune`) — pustynny (mnożnik typu: 2,2 dla „wydm i oaz”, 0,45 dla gór),
- **ruchome piaski** (`quick`) — skupiska 3×3 (80% pól), ≥ 11,5 pola od Dworu; nieprzechodnie,
- **oazy** (`oasis`) — zielone pierścienie wokół każdej wody (pustynia); drzewa rosną tylko tu.

10.3.5 Łączność

`ensureConnected` zapewnia, że ląd nie jest podzielony na odcięte kępy: każdą kępę ≥ 6 pól łączy z obszarem osiągalnym z Dworu najkrótszą ścieżką (Dijkstra po koszcie przeszkód), **przekopując** ją — woda na trasie zamienia się w bród, klif w obniżony stok, cecha blokująca znika. Osiągalność (`Terrain.reachSet`) liczona jest od Dworu po polach przechodnich i zapamiętywana per mapę.

10.3.6 Drzewa i złoża

- `rePlant`: usuwa drzewa z pól nieosiągalnych i cech blokujących, po czym dosadza do celu nacji z wagą rosnącą w kępach (szum), słabszą na wzgórzach; pustynia: tylko przy wodzie i oazach; Europa Wschodnia ×1,25, śnieg ×0,9. Drzewa nie rosną na polach z `pad`.
- `reDeposit`: złoża musi mieć wolny obrys 3×3 pod kopalnię z przerwą (`Data.RULES.gap`), osiągalny z Dworu, w odległości 7–22 pól od środka (do 30 przy trudnościach; w górach najpierw na wzgórzach). Obrys kopalni jest **rezerwowany** (`pad = 1`, drzewa usunięte) — gracz nie może go zalesić ani wyciąć, więc generator gwarantuje miejsce.
- **Pas brzegu Wikingów**: w promieniu 20 pól od Dworu i ≤ 3 pola od morza drzewa są usuwane i pola rezerwowane — miejsce na Przystań, 2 Okręty i 4–6 Chat rybaka.

10.4 Walidacja

`Terrain.validate` odrzuca mapę, gdy: łączność lądu < 80%, złoża są nieosiągalne, osiągalne są < 85% drzew, w odległości ≤ 14 pól od Dworu jest mniej niż 280 wolnych pól budowlanych, rzeka nie ma brodu w zasięgu 16 pól, złoża nie mieści kopalni 3×3, a dla Wikingów na mapie nadmorskiej: brzegu jest za mało (≥ 12 pól w zasięgu 16, najbliższy ≤ 13) lub nie mieści się 9 budynków nadmorskich (Przystań, 2 Okręty, 6 Chat rybaka — `shoreCapacity`). Test `tools/terrain.js` sprawdza całą macierz.

10.5 Rodzaje pól: koszty i przechodność

Tabela 45. Rodzaje pól: koszt przejścia (Terrain.cost), przechodność i możliwość zabudowy (Terrain.passable / buildable)

Pole	Występowanie	Koszt	Przechodnie	Budowlane
Ląd (h = 1)	trawa, piasek, tundra	1	tak	tak
Skała (h = 2)	z generatora; nie na starcie	∞ (nieprzechodnie)	—	nie
Woda: morze, jezioro, rzeka	wk = sea / lake / river	∞ (nieprzechodnie)	—	nie
Bród (wk = ford)	płycizna na rzece	3	—	nie
Lód (wk = ice)	zamarznęte jeziora (klimat śnieżny)	2	—	nie
Bagno (k = bog)	rozlewiska, torfowiska	3	tak	nie
Wydma (k = dune)	pustynia	1,5	tak	nie
Zaspa (k = drift)	klimat śnieżny	2	tak	nie
Ruchome piaski (k = quick)	pustynia	∞ (nieprzechodnie)	nie	nie
Klif (k = cliff)	strome stoki, skarpy nad morzem	∞ (nieprzechodnie)	nie	nie
Oaza (k = oasis)	pustynia; zielony pierścień wokół wody	1	tak	tak (rosną drzewa)

Predykaty `Terrain.passable`, `buildable`, `cost`, `isWater` używane są też przez `World.canPlace`, `Path` i `Fauna`; droga (`road`) obniża koszt pola lądu do 0,6, a prędkość pieszych podnosi $\times 1,6$.

10.6 Statystyki map domyślnych

Wynik `tools/mapstats.js` dla ziaren domyślnych nacji (mapa nadmorska w klimacie nacji): liczba drzew równa się celowi nacji; złoża zgodne z danymi.

Tabela 46. Statystyki map domyślnych

Nacja	Drzewa / cel	Pola wody	Złoża	Brzeg (pola ≤ 16)	Najbliższy brzeg
Frankowie	300 / 300	489	żelazo 2, kamień 3, węgiel 3	95	10,6
Saraceni	150 / 150	518	glina 2	152	12,0
Wikingowie	450 / 450	666	ruda darniowa (torf) 2	121	8,1
Słowianie	300 / 300	457	brak	67	12,0

11. Zasady zabudowy

UWAGA

Rozdział w opracowaniu — zostanie uzupełniony w kolejnym kroku prac nad dokumentacją (plan: `docs/PLAN.md`).

12. Ruch, drogi i logistyka

Ruch ludzi jest w grze osobną warstwą nad ekonomią. Dziś składa się z czterech części: **Path** (A* po polach mapy), **Walkers** (widoczni piesi — „cień symulacji”), **Roads** (drogi przyspieszające ruch) i **Logistics** (sprawność producentów zależna od odległości do Składu lub Dworu). Tylko ostatnia z nich wpływa na liczby ekonomii; pozostałe są wizualne lub służą kontroli dostępu. Plan ożywienia ludzi tak, by ich czynności były spójne z ekonomią (Faza 9B), opisuje aneks [§E](#).

12.1 Ścieżki: moduł Path

Path to siatkowy A* po polach mapy: **8 kierunków, bez ścinania rogów**, bez losowości (nie dotyczy `RNG`). Koszt wejścia na pole to `Terrain.cost` (łąd 1; wydma 1,5; zaspą 2; lód 2; bagno 3; bród 3; droga 0,6) plus $0,5 \times |\Delta e|$ za różnicę wysokości i narzut na drzewa ($0,6 + 0,4 \times \text{drzewa}$); ruch po przekątnej mnoży koszt przez $\sqrt{2}$. Woda, klify, skała i ruchome piaski są nieprzechodnie (rzeka tylko brodem lub lodem). Opcja `opts.blocked(pole)` dodaje blokady (np. zajęte pola budynków); pole startu jest zawsze dozwolone.

Funkcje modułu:

- `Path.grid` — trasa jako lista indeksów pól (z kosztem w `.cost`),
- `Path.find` — A* z **wygładzeniem**: odcinki proste zastępują zakręty siatki, jeśli nie są droższe niż trasa siatkowa ($\times 1,04$),
- `Path.field` — wieloźródłowy Dijkstra: koszt dojścia z najbliższego źródła do każdego pola (te same koszty co A*); używa go `Logistics`,
- `Path.nearest` i `Path.closest` — najbliższe przechodnie pole oraz pole osiągalne najbliższe celowi za wodą lub klifem.

Test `tools/path.js` porównuje A* z niezależnym Dijkstrą (optymalność), sprawdza poprawność tras (tylko pola przechodnie, kroki sąsiednie, bez ścinania rogów, rzeka tylko brodem lub lodem), brak tras do wysp, brak RNG, determinizm i szybkość.

12.2 Widoczni piesi: moduł Walkers

Walkers rysuje postacie idące po ścieżkach **Path**: obywateli, nosicieli i budowniczych. Logika gry (losowania w `World.tick`) zostaje „cieniem symulacji” (rozdz. [§2.6](#)); widoczna postać dąży do celu cienia, a sama niczego nie losuje. Stan **Walkers** żyje poza stanem gry i nie trafia do snapshotu.

- **Drzwi budynku** (`Walkers.doorOf`): punkt tuż przed wejściem z katalogu sprite’ów (strona S lub E i punkt na ścianie), przesunięty na najbliższe wolne pole.
- **Prędkość na polu**: droga $\times 1,6$, wolniej po bagnie, brodzie i wydmach ($1/\sqrt{\text{koszt}}$).
- **Ewakuacja**: gdy plac lub budynek stanie pod postacią, wychodzi prosto na najbliższe wolne pole, a potem wraca do celu.
- **Budowniczowie** noszą materiały z Dworu na plac (etap noszenia), potem budują przy obrysie; bez przydziału lub po ukończeniu wracają do Dworu i znikają.
- **Nosiciele** wychodzą z drzwi producenta i kończą marsz przy drzwiach najbliższego Składu lub Dworu (cel wybiera logika).
- **Myśliwi** nie są częścią **Walkers**: ich pozycja to czysta funkcja czasu gry (rozdz. [§13](#)).

Tabela 47. Prędkości widocznych postaci dziś (pola na minutę gry)

Postać	Prędkość	Uwaga
Obywatel (cień)	0,25	spacer wokół budynków, cele losuje logika
Nosiciel (cień)	0,35	od drzwi producenta do drzwi Składu / Dworu; kosmetyczny (max 12 naraz)

Postać	Prędkość	Uwaga
Budowniczy	1,4	Dwór → plac; noszenie i budowa
Myśliwy	2,2 (powrót z łupem × 0,9)	pozycja jako funkcja czasu (<code>Data.FAUNA_RULES.hunterSpeed</code>)

UWAGA

Te prędkości są **kosmetyczne i niespójne z czasem logiki**: logika zakłada, że budowniczy nosi sztukę materiału w 0,2 min niezależnie od odległości, a producent „odkłada” towar w Składzie bez czasu; widoczny budowniczy idzie do placu kilka minut gry. To jedyny powód, dla którego ludzie dziś tylko *wyglądają*, jakby pracowali — dlatego powstała Faza 9B (aneks [SE](#)).

12.3 Drogi: moduł Roads

Droga to pole z `tile.road = 1` (zapisywane w kaflu, więc trafia do snapshotu). Gracz wytycza ją narzędziem **Drogi** w pasku budowy: pierwszy klik to początek, drugi koniec; trasę wyznacza `Path` (istniejące drogi są tańsze, więc są wykorzystywane), a kolejny klik kontynuuje od końca poprzedniego odcinka. Zasady:

- koszt: **1 deska za każde nowe pole** (pola z drogą nic nie kosztują); podgląd pokazuje liczbę pól i koszt przed potwierdzeniem,
- droga nie powstanie na wodzie, bagnie, wydmy, klifie, drzewach, złożach i zajętych polach; początek i koniec muszą leżeć na wolnym łądzie,
- budowa budynku lub placu zdejmuję drogę spod obrysu; narzędzie **Rozbierz drogę** usuwa pojedyncze pola (bez zwrotu),
- po drodze piesi i nosiciele idą **ok. 1,6× szybciej**, a A^* ją preferuje (koszt pola 0,6 — odpowiada prędkości ×1,6).

Test `tools/roads.js` sprawdza m.in. koszt tylko za nowe pola, odmowy, zdejmowanie drogi przez budowę, preferowanie drogi przez A^* , brak RNG i zapis przez JSON.

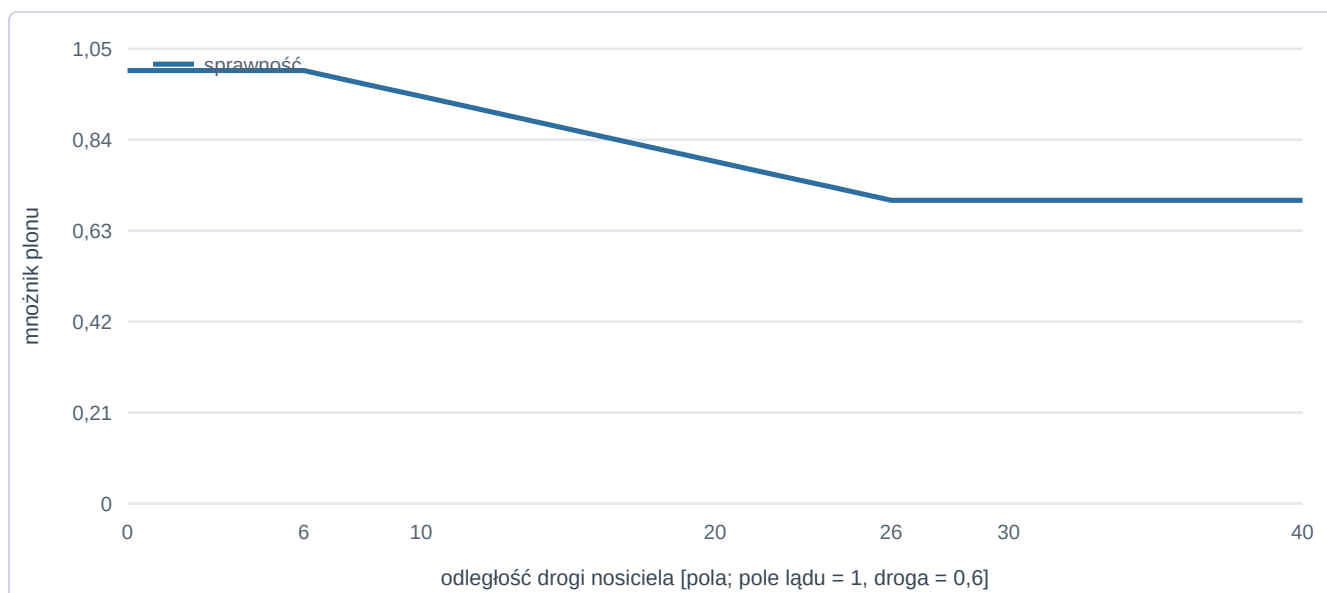
12.4 Sprawność logistyczna: moduł Logistics

Producent daleko od Składu lub Dworu daje mniej. `Logistics.eff0f` zwraca mnożnik plonu (przerobu) `eff`, który wchodzi do `Economy.tick` i `potentialFood` obok mnożnika lasu i zwierzyny:

```
dist = koszt drogi (Path.field) od drzwi producenta do drzwi najbliższego Składu lub Dworu
eff   = 1 - (1 - min) × clamp( (dist - free) / span, 0, 1 )
```

Parametry (`Data.RULES.logistics`): `free` = 6 pól (bez kary), `span` = 20 pól (zakres spadku), `min` = 0,7 (najniższa sprawność).

Odległość liczona jest po polach z kosztami terenu: pole łądu kosztuje 1, **droga 0,6**, a budynki, bagna i woda wydłużają trasę (zajęte pola są blokadą). Producentem jest każdy budynek z wyjściem będącym towarem z listy limitów. Wynik jest pamiętany do zmiany układu (liczba budynków i placów, wersja dróg); budynek dostaje `b.logi = { eff, dist }` do pokazania w interfejsie („transport do Składu ≈ N pól (–X%): zbliż budynki lub połącz drogę”). Opcja `settings.logistics` jest wyłączona w testach specyfikacji (żaden scenariusz się nie zmienia) i włączona w grze (przycisk , `?logistics=0`).



Rysunek 12. Sprawność logistyczna producenta w zależności od odległości drogi do Składu lub Dworu (Data.RULES.logistics)

12.4.1 Jak wygląda to w osadach botów

Poniższy pomiar (skrypt `tools/probe_logistyki.js`, liczony przy każdej budowie dokumentu) pokazuje, że w osadzie układanej przez bota **bez dróg** mediana odległości od Składu / Dworu wynosi ok. 12–13 pól, 90. percentyl ok. 17–21, a pojedyncze budynki (kopalnia darniowa Wikingów) leżą 40 pól od magazynu. Średnia sprawność modułu to ok. 0,88–0,90 — czyli osada „bez organizacji” traci ok. 10% produkcji, a drogi i Składy ten stan poprawiają. To budżet, którego nie wolno przekroczyć przy dalszych zmianach (spec. przyjmuje, że narzut transportu poniżej 10% jest akceptowalny, 15% już obniża ludność Franków o 18%).

Tabela 48. Osady botów R w 90. minucie (bez dróg, logistyka włączona; pomiar `tools/probe_logistyki.js`): odległość drogi drzwi producenta do najbliższego Składu lub Dworu, sprawność modułu Logistics, przepływy towarów, obciążenie transportu (Σ przepływ $\times$ odległość) i liczba nosicieli potrzebnych przy 18 polach/min

Nacja	Ludność	Producenci	Mediana [pola]	P90	Maks.	Śr. sprawność	Wyjścia [szt./min]	Wejścia [szt./min]	Obciążenie [szt.·pole/min]	Wolni ludzie	Nosiciele ($v = 18$)
Frankowie	37	22	12,5	20,1	25,3	0,879	32,3	16,1	741	8	13,7
Saraceni	34	22	12,2	16,5	28,3	0,897	30,5	9,7	538	2	10
Wikingowie	34	21	13,8	21,2	40,4	0,884	27,5	13,7	597	6	11,1
Słowianie	36	24	11,8	19,1	23	0,899	27,1	9,7	476	4	8,8

12.5 Co z tego wynika dla gracza

- **Składy przy producentach i drogi do nich** podnoszą wydajność (do –10% bez nich); Skład ma więc dwie role: pojemność (rozdz. §3) i skrócenie transportu.
- Dwór jest **magazynem centralnym** każdej osady: do niego idzie transport, gdy nie ma Składu; im bliżej Dworu stoi producent, tym mniejsza kara — kosztem zagęszczenia (przerwa i kara za ciasną zabudowę, rozdz. §11).
- Budynki daleko od wszystkiego (kopalnie na złożach, rybacy na brzegu) można „uleczyć” drogą lub własnym Składem.

13. Fauna i polowanie

UWAGA

Rozdział w opracowaniu — zostanie uzupełniony w kolejnym kroku prac nad dokumentacją (plan: [docs/PLAN.md](#)).

14. Interfejs, menu i Doradca

Interfejs to cienka warstwa nad logiką: czyta `World.state()`, wywołuje publiczne funkcje modułów (`Build.enqueue`, `Roads.lay`, `Tribute.setLevel` ...) i niczego nie liczy sam. Poniżej: wejście do gry (menu główne i Piaskownica), pasek stanu, pasek budowy, panele informacyjne i Doradca.

14.1 Wejście do gry: menu główne i Piaskownica

Od Fazy 9 gra otwiera się **menu głównym** w stylu RTS lat 90./2000. (moduł `Menu`, ekrany `#mainMenu`, `#sandbox` i `#pauseMenu`). Jedyną ścieżką startu rozgrywki pozostaje `UI.startGame(fid, {climate, type, seed})` — używając jej Piaskownica, parametr `?play=` i dawny ekran wyboru nacji, więc żaden test nie zależy od menu.

Tabela 49. Ekrany przed grą i w grze

Ekran	Kiedy	Zawartość
Menu główne	domyślnie po załadowaniu strony	logo, panorama (SVG w tle), lista: Kampania, Potyczka, Piaskownica , Wieloosobowa, Samouczek, Edytor map, Ustawienia, Twórcy, Wyjście — poza Piaskownicą pozycje są szare z podpowiedzią „Wkrótce”
Piaskownica	Menu główne → Piaskownica	nacja (cztery karty), klimat (4 kafle), typ mapy (6 kafli), podgląd mapy, opis, ziarno, „Losuj mapę”, Wstecz, Start
Wybór nacji kartami	<code>?menu=0</code>	dawny ekran z czterema kartami; klik karty = start gry na domyślnej mapie nacji (używany przez testy <code>browser_*</code>)
≡ Menu w grze	przycisk ≡ Menu (pierwszy w pasku stanu) lub Esc	Wznów, Nowa mapa, Opcje (szare), Menu główne

Sterowanie menu: ↑/↓ i Enter, mysz i dotyk; Esc w Piaskownicy wraca do menu głównego. Nieaktywne pozycje nie przejmują fokusu klawiatury, nawet gdy kursor znajduje się nad nimi. Na telefonie (szerokość ≤ 720 px lub wysokość ≤ 640 px) układ jest pionowy: podgląd mapy nad kartami, kafle w dwóch kolumnach, przyciski o większej powierzchni dotyku.

14.1.1 Piaskownica

Wybór ma trzy osie, a **podgląd odświeża się po każdej zmianie** (po 60 ms zwłoki):

- Nacja** — karty te same, co na dawnym ekranie wyboru (`UI.buildFactionCards(wrap, onPick, compact)` — wersja `compact` jest mniejsza i nie startuje gry, tylko woła `onPick`).
- Klimat** — 4 klimaty (rozdz. §10). Domyślnie klimat własny nacji (oznaczony „klimat nacji”); po ręcznej zmianie klimatu wybór zostaje przy zmianie nacji.
- Typ mapy** — 6 typów, nazwy zależne od klimatu (np. „Wydmy i oazy” zamiast „Bagnista” na pustyni). **Wikingowie są zablokowani na typie „Nadmorska”**, bo ich budynki (Przystań, Okręt) wymagają morza; przy wyborze Wikingów pole typu przełącza się na nadmorską i wyświetla się odpowiednia uwaga.

Podgląd (`Minimap`, kanwa 192×192 px, 1 pole = 4 px) rysuje mapę z tych samych danych, z których powstanie gra: `Minimap.generate` woła `MapGen.generate` z identycznymi argumentami, więc po Starcie świat jest **identyczny z podglądem** (jedyna różnica to rezerwacja pól Dworu — test `tools/minimap.js` porównuje pole po polu). Kolory: woda wg rodzaju (morze, jezioro, rzeka, bród, lód), skały, cechy terenu (bagno, wydma, ruchome piaski, oaza, zaspas, klif), złoża (kamień, żelazo, węgiel, glina, ruda darniowa), zagęszczenie drzew; biała ramka 4×4 pośrodku oznacza Dwór. Pod podglądem widać opis („Dolina rzeki · Pustynny · ziarno 597318”) i podsumowanie: udział wody i wzgórz, liczba drzew, rzeki z brodami / jeziora / morze, cechy terenu, złoża.

Ziarno. Pole liczbowe i przycisk „🎲 Losuj mapę” ustalają ziarno; to samo ziarno z tym samym klimatem i typem daje tę samą mapę. Jeśli generator uzna mapę za niepoprawną (`meta.valid = false`: np. zbyt mało drzew lub brak złóż

w zasięgu — rozdz. §10), Piaskownica **automatycznie próbuje kolejnych ziaren** (do 8 prób) i wpisuje w pole ziarno, które zadziałało.

Pamięć wyboru. Ostatnia nacja, klimat, typ i ziarno są zapisywane w `localStorage` (klucz modułu `Menu`), zawsze w `try/catch` — w trybie prywatnym lub z zablokowanymi danymi witryny gra działa bez pamięci.

14.1.2 ≡ Menu w grze

Przycisk `≡ Menu` jest **pierwszym elementem paska stanu** (także na telefonie) i otwiera nakładkę o wyższym z-index niż reszta interfejsu. Otwarcie menu **zatrzymuje czas** (`speed = 0`, etykieta przycisku prędkości zmienia się na „Pauza”); „Wznów” przywraca poprzednią prędkość. Klawisze kamery są podczas pauzy zerowane, żeby gra po wznowieniu nie „uciekała” z przytrzymanym klawiszem.

- **Nowa mapa** — ta sama nacja, klimat i typ, **nowe ziarno**.
- **Menu główne** — wraca do ekranu startowego.
- **Opcje** — pozycja nieaktywna („Wkrótce”).

Obie ostatnie pozycje działają przez **nawigację strony** (`location.href` z parametrami `?play=...&climate=...&map=...&seed=...`, a dla menu głównego bez `play`), a nie przez ponowną inicjalizację świata. To świadoma decyzja (rozdz. §17): stan modułów UI (`gameStarted`, ostatni panel, aktywna kategoria, prędkość, otwarty Doradca) i kilkaset obiektów zostaje czysto zresetowane bez ryzyka przecieków i rozjazdu zegarów.

Priorytet Esc w grze: (1) zamyka otwarte menu; (2) jeśli wybrano budynek do postawienia albo przypięto kartę budynku — anuluje wybór (stare zachowanie); (3) dopiero wtedy otwiera menu. W HUD obok pogrubionej nazwy nacji pojawia się dodatkowo opis mapy (np. „Dolina rzeki · Pustynny · ziarno 597318”; na telefonie ukryty), a Doradca zaczyna od wiersza „🗺️ Mapa: ...”.

14.1.3 Parametry adresu

`?menu=0` (stary wybór nacji), `?play=nacja&climate=...&map=...&seed=...` (bezpośredni start — pomija menu), `?logistics=0`, `?classic=1` i inne opisuje rozdział §1. Testy przeglądarkowe wchodzą do gry przez `?menu=0` (`browser_ui`, `browser_play`, `browser_mobile`, `browser_sprites`, `browser_contact`) lub `?play=`; samo menu sprawdza `tools/browser_menu.js` (rozdz. §16).

14.2 Pasek stanu (HUD)

Pasek u góry ekranu zawiera: `≡ Menu`, nazwę gry, nację i opis mapy, ⌚ **czas** (minuty gry), 👥 **ludność**, ❤️ **popularność**, 💰 **złoto**, 🛠️ **narzędzia**, 🍖 **jedzenie** (zapas w minutach zużycia), 👷 **praca** (zajęte / dostępne miejsca) i 🏠 **mieszkania**. Po nich — kontrolki:

Tabela 50. Kontrolki paska stanu

Kontrolka	Działanie	Rozdział
Racje: ×N	racje żywności ×0,5 / ×1 / ×1,5 / ×2 (zużycie i popularność), przycisk przełącza cyklicznie	§4
Podatek: N	poziom 0–3 (cyklicznie): złoto i popularność	§4
👷 Budowniczowie: N	liczba budowniczych 2–5 (po 5 wraca do 2)	§5
Haracz (lista)	„Haracz: wyłączony”, preset (Łatwy / Normalny / Trudny / Mistrz — z poziomem dla nacji) albo „Własny poziom 1–11”	§8
🚢 Wyprawy i Zapas załogi	tylko Wikingowie: tryb wypraw „Zwykłe” / „Mieszane” (wielka wyprawa z 2 okrętów) i reguła wysyłki (zapas załogi 90% / brak)	§8
🏴 Napady	tylko Saraceni: napady na karawanę włączone / wyłączone	§9
🔥 Kryzysy	harmonogram kryzysów (zaraza, pożary, krach cen, wylesienie — rozdz. §9.2) włączony / wyłączony; domyślnie wyłączony	§9
🚚 Logistyka	sprawność logistyczna producentów włączona / wyłączona	§12.4
Prędkość: ×N	czas gry ×1 / ×2 / ×4 (cyklicznie)	§E.3

Kontrolka	Działanie	Rozdział
 Doradca	otwiera panel Doradcy	niżej
 Więcej	tylko na małych ekranach: rozwija rzadko używane kontrolki (oznaczone <code>sec</code>)	niżej

Na telefonie pasek jest zwinięty: widać stan i najważniejsze przyciski, a pozostałe ukrywa  **Więcej** (przycisk widoczny wyłącznie przy małej szerokości). Test `browser_mobile.js` sprawdza, że żaden element nie wychodzi poza ekran i że dotyk działa jak kliknięcie.

14.3 Pasek budowy

U dołu ekranu leży pasek z **ośmioma kategoriami** (Mieszkania, Jedzenie, Drewno, Surowce, Rzemiosło, Handel, Militaria, Społeczne) oraz kategorią **Drogi** (narzędzia: wytycz drogę, rozbierz drogę). Każdy przycisk budynku pokazuje miniaturę `sprite'a` (z katalogu `SPRITE_META`), nazwę (zależną od nacji — `Data.nameOf`) i koszt. Wybór budynku włącza **tryb budowy** (`BuildMode`):

- **duch budynku** podąża za kursorem w obrysie `w×h` (rozd. §11); zielony oznacza poprawne miejsce, czerwony — odmowę z powodem w podpowiedzi (np. „za blisko innego budynku”, „na wodzie”, „wymaga lasu”);
- pola w strefie wydłużonego czasu budowy (odległość 2–3 od sąsiada) są podświetlone **przed** kliknięciem, a podpowiedź podaje karę czasu;
- na pochyłym terenie podpowiedź pokazuje koszt **wyrównania** (rozd. §10);
- klik = `Build.enqueue`; prawy przycisk i **Esc** anulują wybór; kliknięcie placu poza trybem budowy anuluje plac (rozd. §5).

Kamera: przeciąganie prawym przyciskiem lub WASD, kółko myszy — zoom; na telefonie przeciąganie jednym palcem i szczypanie. Dotknięcie ekranu działa jak lewy klik (`BuildMode.click`).

14.4 Panele informacyjne

- **Magazyn** (`#resPanel`, prawa strona) składa się z sekcji: **MAGAZYN** (towary z ilością i pojemnością `ilość / cap`; pełny magazyn jest wyróżniony), **ŻYWNOŚĆ** (produkcja, zużycie ludzi, podatek górniczy, maksymalna ludność), „Głód za”, ostrzeżenia (narzędzie, popularność < 45, podatek 3), link do Doradcy z liczbą ostrzeżeń, budowniczymi (czynniki / ustawienia), **BUDOWA** (do 4 placów z postępem), **HARACZ** (następna rata, dostawa / wymagane, terminy; koszyk daniny Słowian), załoga na morzu (Wikingowie) i **NAPADY** (Saraceni, gdy był już napad). Panel odświeża się co 0,2 min gry.
- **Karta budynku** (`#bInfo`): nazwa, **status z Doradcy** (kolor: czerwony / bursztynowy / szary / zielony — razem z wydajnością, transportem do Składu `b.logi` i zwierzyną w łowisku), wejście i wyjście na minutę (nazwy towarów zależne od nacji) oraz informacja o narzędziu. Najechanie kursorem pokazuje kartę chwilowo, **kliknięcie ją przypina** (Esc odpina); przy otwartym Doradcy karta się nie pokazuje.
- **Znaczniki nad budynkami:** litera w kółku informuje o stanie budynku bez otwierania karty — **!** brak wejścia (czerwony), **T** czeka na narzędzie, **P** brak pracownika, **Z** magazyn pełny, **J** stoi (jedzenie tylko z nadwyżki), **X** wyczerpane złożo.

14.5 Doradca

`Advisor` (Etap 6) to moduł tylko do odczytu, który zamienia stan gry w czytelne zdania. Otwiera go przycisk  **Doradca** lub link w panelu Magazyn; panel odświeża się razem z interfejsem. Zaczyna od wiersza „ Mapa: ...” (opis mapy z `Minimap.describe`), a dalej składa się z części:

1. Status budynku (`Advisor.statusOf`) — jedno zdanie na budynek, w kolejności reguł: Dwór i budynki bez pracownika (opis), brak obsady („za mało wolnej ludności — żywność obsadzana pierwsza”), czekanie na narzędzie („Targ kupi je za 20 zł, kuźnia zamkowa dorobi ok. 1 szt. na 8 min”), **brak wejścia** (z listą budynków, które wytwarzają brakujący towar — np. „zbuduj: Tartak / Skład drewna”), pełny magazyn, stanie z powodu zasady „jedzenie tylko z

nadwyżki”, wyczerpane złoża, a gdy wszystko jest dobrze — „Pracuje — N% wydajności” z dopiskiem o transporcie (–X% : „zbliż budynki lub połącz drogę”) i o zwierzynie w łowisku.

2. Ostrzeżenia ogólne (`Advisor.warnings`), od najpilniejszych: głód (z odliczaniem do pierwszych zgonów), popularność poniżej progu imigracji, podatek 3 jako pułapka, **podatek górniczy** (ile porcji jedzenia zjadają kopalnie), limit ludności z żywności (P_max — rozdz. §4), pełne magazyny, skarbiec do ograbienia (Saraceni, > 300 zł), przypomnienie o Koszarach 30 minut przed pierwszym terminem haraczu (Frankowie).

3. Żywność — produkcja (potencjał), zużycie (ludzie, budowniczowie, kopalnie) i brama imigracji z maksymalną ludnością (rozdz. §4).

4. Haracz — plan na wybrany poziom (`Advisor.plan`) — lista budynków potrzebnych do wskazanego poziomu haraczu (✓ / ✗ mamy / potrzeba) i uwagi nacji (żołd Franków, nasycenie rynku Saracenów, załoga Wikingów, koszyk Słowian; rozdz. §8, „Doradca a haracz”). Przy wyłączonym haraczu Doradca podpowiada wybór presetu.

5. Znaczniki na budynkach — legenda liter z poprzedniej sekcji.

Doradca nie podejmuje żadnych decyzji ani nie zmienia stanu; testy sprawdzają go przez `tools/browser_ui.js` (wiersze statusów, ostrzeżenia, panel).

14.6 Co jeszcze zaplanowano

Faza 10: **legenda** terenu, wody, dróg i zwierzyny (panel w grze), spójność Doradcy z nowymi mechanikami (wyrównanie, „brak dojścia”, przetrzebiona zwierzyna). Faza 9B: nowe **tempo bazowe czasu gry** i prędkości $\times 1/\times 2/ \times 3/\times 6$ (aneks §E, rozdz. §E.3) oraz sekcja „Transport” w karcie budynku.

15. Grafika i silnik sprite'ów

UWAGA

Rozdział w opracowaniu — zostanie uzupełniony w kolejnym kroku prac nad dokumentacją (plan: [docs/PLAN.md](#)).

16. Testy i jakość

Gra jest silnikiem ekonomicznym z dużą liczbą współzależnych liczb, więc **główną ochroną jakości są testy liczbowe**, a nie ręczne klikanie. Cały zestaw jest w katalogu `tools/` (rozd. §C.2) i dzieli się na trzy warstwy.

16.1 Trzy warstwy testów

Tabela 51. Warstwy testów

Warstwa	Narzędzie	Co chroni	Czas
Silnik (headless)	<code>tools/headless.js</code> ładuje cały skrypt gry w <code>node:vm</code> z atrapami DOM i eksportuje moduły	ekonomię, haracz, zdarzenia, teren, ścieżki, fauna, drogi, logistykę, podgląd mapy	sekundy do ok. 5 min (<code>normal.js</code>)
Przeglądarka (Playwright + Chromium)	<code>tools/browser_*.js</code>	UI, dotyk, wypiek sprite'ów, start każdej kombinacji klimat × typ, menu i Piaskownicę, brak błędów JS, FPS	ok. 0,5–3 min każdy
Regresja (porównanie z wzorcem)	wzorce <code>baselineN_*.txt</code> (w Fazie 10 trafiają do <code>tools/baseline/</code> wraz z <code>tools/regress.sh</code>)	identyczność wyników liczbowych po zmianach niezwiązanych z ekonomią	porównanie <code>diff</code>

Zasada nadrzędna: zmiana, która nie ma zmieniać ekonomii, musi dać **bajt w bajt ten sam wynik** `scenarios.js` (58 scenariuszy), `normal.js`, `mapstats.js`, `raids.js` i `store.js`. Dlatego nowe mechaniki (logistyka, fauna, drogi, a docelowo fizyczna logistyka) są za przełącznikami `settings.*`, wyłączonymi w testach specyfikacji (rozd. §2.7), a każda z nich ma własny test parytetu „wyłączona = jak dawniej”.

16.2 Boty testowe

`TestBots` rozgrywa grę bez człowieka, w trzech wariantach (rozd. §B.1):

- **R** — odtwarza przepis budowy nacji (`RECIPE`) z szumem ϵ (nieuwaga gracza: pominięte lub spóźnione kroki),
- **D** — ten sam przepis z opóźnieniem decyzji (co 2 min),
- **N** — reaktywny, bez przepisu: patrzy na brakujące wejścia i stan żywności.

Boty ustawiają budynki przez `TestBots.findPlace` (miejsce spełniające zasady zabudowy i wyrównywania terenu, rozdz. §11); Chata myśliwego jest stawiana przy lesie, choć gra tego nie wymaga — dzięki temu wyniki wzorca zostają nietknięte po zmianie reguły „myśliwy nie wymaga lasu”.

16.3 Scenariusze i niezmienniki

`tools/scenarios.js` uruchamia **58 scenariuszy akceptacyjnych** specyfikacji: normalną grę, warunki brzegowe (np. brak narzędzi, pełne magazyny, zero ludzi do pracy) i kryzysy (rozd. §9.2). W **każdym ticku** każdego scenariusza sprawdzane są niezmienniki:

- zapasy nie są ujemne i nie przekraczają limitu towaru (wyjątek: start 14 narzędzi przy limicie 10);
- ludność ≥ 10 w każdej chwili, liczba narzędzi ≥ 0 ;
- co najmniej 2 czynnych budowniczych;
- załoga na wyprawie + obsadzeni pracownicy $\leq$ ludność (+ 2 zapasu na zaokrąglenia);
- ceny Saracenów leżą na krzywej rynku (kadzidło 0,9–4,2, tkanina 0,9–3,6, ceramika 0,6–2,4; $\times 1,2$ z Karawanseraju — rozdz. §3).

Wyniki scenariuszy i botów publikuje aneks §B (odświeżane przez `docs/collect.js`).

16.4 Testy modułów

Tabela 52. Co sprawdza test każdego modułu świata

Test	Moduł	Kluczowe sprawdzenia
terrain.js	MapGen , Terrain	4 nacje × 4 klimaty × 6 typów × ziarna: niezmienniki pól, łączność lądu, liczba drzew i złóż, miejsce pod zabudowę, brody, brzeg Wikingów
mapstats.js	MapGen	cele liczby drzew (300/150/450/300), złoża, brzeg
path.js	Path	koszt = niezależny Dijkstra, trasa tylko po polach przechodnich, brak ścinania rogów, brak RNG, determinizm, szybkość
walkers.js	Walkers	postacie tylko na wolnych polach, nosiciele dochodzą do drzwi, logika identyczna z Walkers i bez (cień symulacji)
roads.js	Roads	koszt tylko za nowe pola, odmowy, zdejmowanie drogi przez budowę, preferowanie drogi przez A*, JSON
spacing.js , leveling.js	World.canPlace , Build	przerwa 1 pole, strefa kary 2–3, wyrównywanie stoków, wzgórze decyzyjne
logistics.js	Logistics	blisko Składu bez kary, daleko spadek do minimum, drogi i Skład pomagają, ekonomia produkuje mniej
fauna.js	Fauna	zaludnienie map, pozycje tylko na wolnych polach, zero RNG globalnego, determinizm, snapshot, parytet z fauną
footprints.js	obrysy w×h	zgodność z katalogiem sprite'ów, zajętość i zwolnienie pól, złoża pod kopalnią, place budowy
minimap.js	Minimap	opis i podsumowanie 24 kombinacji klimat × typ, rysunek na atrapie kanwy (każde pole = 1 prostokąt, ramka Dworu), podgląd = mapa gry (to samo ziarno → te same pola), mapa niepoprawna i mapa bez meta
raids.js , store.js	Events , Economy	strata średnia po 24 ziarnach ze Strażnikami i bez; wpływ Składu na pojemność koszyka Słowian

16.5 Testy w przeglądarce

Test	Zakres
browser_ui.js	przeprawy UI dla 4 nacji: budowa kliknięciem, karta budynku, haracz (preset i poziom 1–11), przełączniki, Doradca, żywa pętla gry
browser_play.js	bot buduje osadę, zrzuty z Doradcą i kartą budynku
browser_mobile.js	widok telefonu (dotyk): HUD, ⚙ Więcej, Doradca
browser_sprites.js	wypiek sprite'ów każdej nacji: wszystkie budynki, obrysy = katalog, niepuste warstwy, czas
browser_terrain.js	każda kombinacja klimat × typ startuje przez ?play=... : brak błędów JS, wypieczone chunki, FPS
browser_fauna.js	zaludnienie, myśliwy przechodzi fazy wyprawy, strzała w locie, FPS z fauną vs bez
browser_menu.js	menu główne (tylko Piaskownica aktywna, klawiatura, hover), Piaskownica (przepływ nacja → klimat → typ → Start dla każdej nacji; mapa gry = meta podglądu; blokada Wikingów na nadmorskiej; pamięć wyboru w localStorage), ≡ Menu i priorytet Esc, „Nowa mapa” i „Menu główne”, ?menu=0 , ?play= , telefon 390 × 844 (bez poziomego przewijania), 0 błędów JS

Testy przeglądarkowe czekają na `window.__gameReady === true` (gra startuje po wypieku sprite'ów) i — poza `browser_menu.js` — wchodzi do gry przez `?menu=0` lub `?play=`, żeby nie zależeć od menu (rozd. [§14.1.3](#)).

16.6 Bramka każdej fazy

Każda faza prac (rozd. [§17](#)) kończy się tym samym zestawem: (1) testy modułu nowej fazy i regresja headless — `scenarios` identycznie z wzorcem, `normal`, `mapstats`, `raids`, `store`, `fauna`, `terrain`, `path`, `walkers`, `roads`, `spacing`, `leveling`, `logistics`, `footprints`, `minimap`; (2) testy przeglądarkowe; (3) zrzuty

zmienianych ekranów; (4) **aktualizacja tej dokumentacji** (aneks [§D](#)); (5) nowy wzorzec regresji; (6) commit i push; (7) wersja WIP do przeklikania przez użytkownika.

16.7 Wydajność

Budżety, których pilnują testy: wypiek sprite'ów nacji (`browser_sprites.js`), FPS widoku (`browser_terrain.js` -- `fps`), `Fauna.tick` $\leq 0,25$ ms, zwierzęta $\leq \sim 100$ naraz, FPS z fauną $\geq 85\%$ FPS bez niej, $A1^*$ na 48×48 w milisekundach (`path.js`). Plan Fazy 9B dodaje budżet FPS $\geq 85\%$ sprzed zmian dla fizycznej logistyki (aneks [§E](#)).

16.8 Trudność i krzywa nauki

Skala nacji. Gra ma kampanię „w gwiazdkach”: **Słowianie** ★☆☆☆ (nacja wprowadzająca), **Frankowie** ★★☆☆ (najłatwiejsza z dotychczasowych), **Wikingowie** ★★★☆ (etap średni: miód, broń, wyprawy), **Saraceni** ★★★★★ (etap późny: handel, woda, napady). Kolejność odpowiada liczbie zasad, które gracz musi opanować naraz: Słowianie to zbieractwo i koszyk daniny, Frankowie dokładają łańcuch metalurgii i żołąd, Wikingowie wyprawy z załogą, a Saraceni rynek z nasycaniem cen i napady na karawanę.

Tempo i presja. Pierwszy termin haraczu pada po 60 / 50 / 110 / 60 minutach (Frankowie / Saraceni / Wikingowie / Słowianie), potem co 15 min. Presety (Łatwy poz. 3, Normalny 5–6, Trudny 6–8, Mistrz 7–12) skalują presję tak, by w każdej nacji podobny odsetek casualowych graczy przechodził dany poziom (rozdz. [§8.2](#)).

Krzywa nauki — co gracz odkrywa i w jakiej kolejności:

1. *0–10 min*: wybór miejsca pod pierwsze budynki (przerwa 1 pole, strefa kary, wyrównywanie stoku), deski jako wąskie gardło; pierwsza Chata drwala jest darmowa.
2. *10–30 min*: żywność (łańcuch, $P_{\max}$ i brama imigracji 110%), obsada z priorytetem jedzenia, narzędzia (Targ lub kuźnia), Skład i droga jako lek na odległości.
3. *30–60 min*: haracz — pierwszy termin, Koszary 30 min wcześniej (Frankowie), pierwsza wyprawa lub rynek; pułapka podatku 3.
4. *Później*: popularność i dobrobyt, napady, zagospodarowanie złóż, rozbudowa ludności ponad $P_{\max}$.

Pułapki, które gra wykrywa i opisuje w Doradcy: podatek 3 (popularność < 45 zamraża imigrację), brak Koszar przed terminem, brak narzędzia w kolejce, magazyn pełny przy braku Składu, producent daleko od Dworu, przetrzebiona zwierzyna w łowisku Chaty myśliwego. Ich zestaw służy też jako lista kontrolna przy ocenie, czy nowy mechanizm nie podnosi trudności ponad miarę.

Zmiany, które trudność podnoszą, trzeba mierzyć. Dlatego fizyczna logistyka (Faza 9B) ma jako bramkę bilans botów R/D/N: co najmniej 90% wyniku bazowego i zasadę „tylko na niekorzyść”, a wszystkie wyniki trafiają do rozdz. [§B.1](#).

17. Decyzje projektowe i historia

Rozdział zbiera **decyzje, które nie wynikają z kodu ani ze specyfikacji ekonomii**: zamówienia użytkownika, wybory projektowe i ich uzasadnienia. Dzięki niemu kolejne zmiany nie cofają przez przypadek czegoś, co było świadomym wyborem. Do każdej decyzji podano rozdział, który ją opisuje.

17.1 Historia prac

Tabela 53. Fazy prac i ich efekty (commity w gałęzi `claude/festive-meitner-ejo4l1`)

Faza	Efekt	Commit
Etapy 1–6 (autor)	ekonomia przepływowa, haracz, zdarzenia, Słowianie, Doradca, boty testowe i scenariusze wg spec. v3.6	361cd3d , 51227c7 , a2ad568
Prototyp grafiki	proceduralne sprite'y w stylu Settlers / Twierdza: 88 budynków 4 nacji, ludzie, przyroda	d04fcc4 ... f653211
0–4	wzorzec testów, silnik sprite'ów w grze, obrysy <code>w×h</code> , Dwór 4×4	fd38ee8
5	teren v2: 4 klimaty × 6 typów map, podłoże w chunkach	7e6d575 , c4beb2a
6, 6b, 6c, 7	A* i piesi, drogi, przerwa i kara za ciasną zabudowę, logistyka, wyrównywanie terenu	b8b4cb7 , 8466112 , b5ea3af
8	fauna (18 gatunków) i polowanie z łukiem	fd9aefb , d09c70a , ee1271d
Dokumentacja	generator PDF, rozdziały, analiza fizycznej logistyki	ee4eaea , 231dc77 , 8484777
9	menu główne RTS, Piaskownica, ≡ Menu, podgląd mapy	cf37876

Stan i plan dalszych prac: `docs/PLAN.md` (kopia w repozytorium).

17.2 Decyzje użytkownika (wiązące)

Tabela 54. Decyzje użytkownika

Decyzja	Skutek w grze	Rozdz.
Przerwa między budynkami = 1 pole (twardy zakaz stawiania „ściana w ścianę”)	<code>World.canPlace</code> odmawia „Za blisko innego budynku — zostaw wolne pole (ulicę)”	§11
Strefa wydłużonego czasu budowy = pola w odległości 2 i 3 od budynku (+10% za sąsiada, maks. +40%), widoczna przed wyborem miejsca	kara <code>crowd</code> w placu budowy; podświetlenie w trybie budowy	§11, §5
Drogi przyspieszają ruch, A* je preferuje	koszt pola 0,6, prędkość ×1,6, 1 deska za pole	§12.3
Wzgórza do wyrównania mają własną grafikę i nie utrudniają startu	obszar startowy ≤ 13,5 pola od Dworu jest płaski; wyrównywanie wydłuża budowę	§10
Chata myśliwego nie wymaga lasu („myśliwy i tak poluje na zwierzęta”)	<code>plon</code> = <code>×</code> <code>min</code> (1, <code>zwierzyna</code> w łowisku / próg); boty nadal stawiają ją przy lesie, więc wzorce regresji bez zmian	§13
Menu główne w stylu RTS + Piaskownica (nacja, klimat, typ mapy)	moduł <code>Menu</code> , <code>Minimap</code> , ≡ Menu w grze	§14.1
Ożywienie ludzi (po Fazie 9, przed 10 i 11): drwal fizycznie ścina, cieśla niesie deskę, budowniczy dochodzi na plac, karawana idzie na koniec mapy i wraca; drogi ważniejsze, centralny magazyn jeśli pomoże; bez zepsucia balansu	Faza 9B: analiza i projekt w aneksie §E	§12
Jedna wspólna dokumentacja PDF z kodu, dwóch dokumentów ekonomii i decyzji; w repozytorium; aktualizowana po każdym etapie	<code>docs/</code> (<code>node docs/build.js</code>)	§D

Decyzja	Skutek w grze	Rozdz.
Tryb pracy „auto” — działamy bez dopytywania, po każdej fazie commit, push i wersja WIP do przeklikania	procedura bramki	§16.6

17.3 Decyzje projektowe (z uzasadnieniem)

Jeden plik HTML bez zależności. Cała gra, silnik sprite'ów i dane żyją w `Nova_Roma.html`; wyniki testów i dokumentacja są liczone z tego samego pliku przez `tools/headless.js`. Dzięki temu nie ma rozjazdu „kod a dokumentacja” i można grę uruchomić z dysku bez serwera.

Cień symulacji (wizualne warstwy bez wpływu na logikę). Piesi, zwierzęta i myśliwi są rysowane po ścieżkach `Path`, ale logika gry (`World.tick`) niczego od nich nie czyta i niczego z nich nie losuje; własne PRNG (np. `state.fauna.rs`) i globalny RNG nietknięty. Efekt: **58 scenariuszy specyfikacji daje te same liczby z fauną, drogami i pieszymi i bez** (rozdz. §2.6). To najważniejsza decyzja architektoniczna tej wersji.

Nowe mechaniki za przełącznikami `settings.*`. Napady, zapas załogi, miejsca pod rybaków, logistyka i fauna są wyłączone w testach specyfikacji i włączane przez `UI.startGame`; test parytetu każdej z nich dowodzi, że wyłączona zachowuje się jak dawniej (rozdz. §2.7). Tak samo ma działać fizyczna logistyka (`settings.physical`, Faza 9B).

Zasada „tylko na niekorzyść” dla transportu. Sprawność logistyczna jest ≤ 1 : producent blisko Składu lub Dworu nie dostaje premii ponad specyfikację, a daleki traci do 30%. Osada bota bez dróg traci średnio ok. 10% (rozdz. §12.4.1) — to budżet, którego nie wolno przekroczyć.

Dwór jest magazynem centralnym. Zamiast dodawać nowy budynek „Skład centralny” (inspiracja *Knights and Merchants*), przyjęto, że funkcję tę pełni Dwór; Skład zwiększa pojemność i skraca transport lokalnie. „Duży Skład” wróci do rozważenia tylko wtedy, gdy bramki bilansu Fazy 9B tego zażądataj (aneks §E).

Menu: nawigacja zamiast ponownej inicjalizacji. „Nowa mapa” i „Menu główne” przeładowują stronę z parametrami adresu zamiast resetować stan w miejscu. Powód: `UI.startGame` jest jednorazowe, a stan modułów UI (ostatni panel, aktywna kategoria, prędkość, otwarty Doradca) i obiekty świata łatwo przeciekają przy ponownym wejściu; przeładowanie jest tańsze i pewniejsze, a gracz traci tylko to, co chce porzucić (rozdz. §14.1.2).

Wikingowie zablokowani na „Nadmorskiej” w Piaskownicy. Ich budynki (Przystań, Okręt) wymagają morza, a generator mapy dla nich zawsze zwraca typ nadmorski (`World.init`); gdyby Piaskownica pozwalała wybrać inny, podgląd kłamałby względem gry (rozdz. §14.1.1).

Podgląd = mapa gry. `Minimap.generate` woła ten sam `MapGen.generate` z tymi samymi argumentami co `World.init`; test `tools/minimap.js` porównuje pole po polu. Dla map niepoprawnych Piaskownica sama dobiera następne ziarno.

Bot układa Chatę myśliwego przy lesie. Po zniesieniu wymogu lasu gra pozwala ją stawiać gdziekolwiek, ale `TestBots.findPlace (search(true) przed search(false))` nadal szuka najpierw miejsca z lasem. Gdyby bot zmienił miejsca, zmieniłby się strumień zdarzeń, a razem z nim wszystkie wzorce regresji — a nie jest to zmiana ekonomii.

Budownicowie 2–5, domyślnie 3. Jeden budowniczy wywraca grę (ludność @20 spada do 10–15), trzech odtwarza tempo dawnego modelu; od trzech wzwyż wąskim gardłem są deski, nie ręce (rozdz. §5.2).

Dokumentacja liczona z kodu. Liczby w tabelach i wykresach (`{{v:...}}`, `{{tabela:...}}`) pochodzą z gry w chwili budowy PDF; tekst opisuje zasady. Dzięki temu zmiana danych wymaga tylko przebudowy dokumentu, a zmiana zasad — poprawy tekstu (aneks §D).

17.4 Decyzje otwarte

Poniższe wartości mają **domyślne rozstrzygnięcie**, które użytkownik może zmienić jedną stałą (aneks §E.9):

- tempo bazowe czasu gry 12 s na minutę gry (Faza 9B-0; dziś 4 s);

- napady na karawanę pozostają rozliczane ze skarbca (karawana ilustruje i opóźnia sprzedaż, ale nie jest „celem” napadu);
- magazyn centralny = Dwór;
- po Fazie 10: czy zostawić styl „klasyczny” (`?classic=1`) jako zapas.

18. Specyfikacja a implementacja

UWAGA

Rozdział w opracowaniu — zostanie uzupełniony w kolejnym kroku prac nad dokumentacją (plan: `docs/PLAN.md`).

A Wyniki symulatora ze specyfikacji

UWAGA

Rozdział w opracowaniu — zostanie uzupełniony w kolejnym kroku prac nad dokumentacją (plan: docs/PLAN.md).

B Wyniki botów i scenariuszy w grze

Wyniki poniżej pochodzą z plików `docs/data/wyniki_testow.json`, które zapisuje `node docs/collect.js` (uruchamia `tools/normal.js` i `tools/scenarios.js` albo parsuje ich gotowe wyjścia). Odświeżamy je po każdej fazie, w której zmieniła się logika gry; stan z dnia i commitu podano w podpisach tabel.

B.1 Normalna gra botów

Boty: **R** — przepis nacji z szumem ϵ (nieuwaga), **D** — przepis z opóźnieniem decyzji (co 2 min), **N** — reaktywny, bez przepisu (rozdz. §16). Haracz „Łatwy” oznacza poziom 3 od pierwszego terminu nacji.

Tabela 55. Normalna gra botów (tools/normal.js, 2026-10-06, commit cf37876): ludność @20/40/60/90/150, koniec, minuta 36 osób, minuty głodu, terminy haraczu, dostawy, złoto, deski

Nacja	Bot	Haracz	Ludność	Koniec	36 os.	Głód	Terminy	Dost.	Złoto	Deski
Frankowie	R	wył.	25 / 32 / 43 / 43 / 43	43	48	0	—	0	299	154
Frankowie	R	Łatwy	25 / 31 / 43 / 43 / 43	43	49	0	6/6	32	200	154
Frankowie	R (ϵ 0,15)	Łatwy	24 / 32 / 43 / 43 / 43	43	50	0	6/6	32	200	154
Frankowie	D (ϵ 0,15)	Łatwy	14 / 32 / 32 / 43 / 43	43	71	3	4/6	32	175	154
Frankowie	N (ϵ 0,15)	Łatwy	13 / 10 / 10 / 10 / 10	10	—	136	0/6	0	159	30
Saraceni	R	wył.	20 / 38 / 38 / 38 / 38	38	35	0	—	0	926	132
Saraceni	R	Łatwy	20 / 38 / 38 / 38 / 38	38	35	0	6/6	486	702	132
Saraceni	R (ϵ 0,15)	Łatwy	20 / 38 / 38 / 38 / 38	38	35	0	6/6	486	702	132
Saraceni	D (ϵ 0,15)	Łatwy	14 / 20 / 38 / 38 / 38	38	58	0	4/6	486	595	132
Saraceni	N (ϵ 0,15)	Łatwy	13 / 10 / 10 / 10 / 10	10	—	125	1/6	167	7	30
Wikingowie	R	wył.	18 / 32 / 38 / 38 / 38	38	54	0	—	0	1004	318
Wikingowie	R	Łatwy	18 / 32 / 38 / 38 / 38	38	54	0	6/6	22	1004	318
Wikingowie	R (ϵ 0,15)	Łatwy	17 / 32 / 38 / 38 / 38	38	55	0	6/6	15	793	278
Wikingowie	D (ϵ 0,15)	Łatwy	14 / 16 / 33 / 36 / 38	38	88	8	6/6	17	830	318
Wikingowie	N (ϵ 0,15)	Łatwy	20 / 28 / 36 / 44 / 52	52	53	0	0/6	0	492	156
Słowianie	R	wył.	32 / 40 / 40 / 40 / 40	40	25	0	—	0	236	168
Słowianie	R	Łatwy	32 / 40 / 40 / 40 / 40	40	25	0	6/6	540	236	168
Słowianie	R (ϵ 0,15)	Łatwy	32 / 40 / 40 / 40 / 40	40	26	0	6/6	540	235	168
Słowianie	D (ϵ 0,15)	Łatwy	14 / 26 / 34 / 39 / 40	40	62	9	4/6	540	203	168
Słowianie	N (ϵ 0,15)	Łatwy	32 / 44 / 49 / 49 / 49	49	32	0	0/6	0	405	125

B.2 Scenariusze akceptacyjne

58 scenariuszy ze specyfikacji (normalna gra, warunki brzegowe, kryzysy) dla czterech nacji; w każdym ticku sprawdzane są niezmienniki (brak ujemnych zapasów, ludność ≥ 10 , ceny w krzywej, magazyny w limitach...). Kolumna „Haracz” to terminy spełnione / ocenione.

Tabela 56. Scenariusze akceptacyjne (tools/scenarios.js, 2026-10-06): 58 zgodnych, 0 odbiegających; przebiegi z naruszeniem niezmienników: 0

	Id	Scenariusz	Haracz	Ludność @20/40/60/90	Koniec	Głód	Złoto
✓	F	normalna gra	6/6	25/31/43/43	43	0	200
✓	S	normalna gra (bez napadów)	6/6	20/38/38/38	38	0	971
✓	V	normalna gra	6/6	18/32/38/38	38	0	794
✓	W	normalna gra	6/6	32/40/40/40	40	0	236
✓	F1	kopalnie bez żywności	0/6	12/10/10/10	10	131	99
✓	F2	Zbrojownia bez Huty	0/6	25/32/43/43	43	0	301

	Id	Scenariusz	Haracz	Ludność @20/40/60/90	Koniec	Głód	Złoto
✓	F3	1 łańcuch i 8 kopalni	0/6	14/19/28/32	33	0	220
✓	F4	1 miejsce pod Kopalnię węgla	6/6	26/30/44/44	44	0	206
✓	F5	0 miejsc pod Kamieniołom	3/6	27/32/32/24	43	0	162
✓	F6	podatek 3	6/6	14/24/41/43	43	0	682
✓	S1	eksport bez Gajów	0/6	14/15/15/15	15	0	12
✓	S2	5 Wytwórni na 1 Gaju	5/6	18/25/25/25	25	0	66
✓	S3	bez wody	0/6	14/16/16/16	16	0	12
✓	S4	wyczerpana glina (60)	4/6	20/38/38/38	38	0	66
✓	S5	0 miejsc pod Glinianki	0/6	20/20/24/24	38	0	85
✓	S6	sam eksport kaczidła	5/6	20/38/38/38	38	0	273
✓	S7	1 Targ	6/6	20/38/38/38	38	0	854
✓	V1	Okręt bez Miodosytni	0/6	18/32/38/38	38	0	345
✓	V1	Okręt bez Zbrojowni	0/6	18/32/38/38	38	0	345
✓	V1	Okręt bez kopalni rudy	0/6	18/33/40/40	40	0	361
✓	V2	4 Wypalarki na 2 drwalach	0/6	23/33/40/40	40	0	366
✓	V3	0 miejsc pod Kopalnię darniową	0/6	18/33/40/40	40	0	360
✓	V4	3 miejsca na brzegu	6/6	18/32/32/32	32	0	750
✓	V5	3 miejsca + dodatkowy Myśliwy/Mleczarnia/Sad	6/6	18/32/48/48	48	0	865
✓	V6	poziom 8, przepis nacji, zwykle	1/6	18/32/38/38	38	0	794
✓	W1	Normalny (poziom 6)	6/6	32/40/40/40	40	0	236
✓	W2	Trudny (poziom 8), przepis nacji	5/6	32/40/40/40	40	0	236
✓	W3	Mistrz (poziom 11) + 2 pary	6/6	32/40/40/40	40	0	156
✓	W4	brak Chaty nosiwody	0/6	32/40/40/40	40	0	238
✓	W5	Barć bez Woskarni	0/6	32/40/40/40	40	0	237
✓	W6	Chaty łowców bez Garbarni	0/6	32/40/40/40	40	0	237
✓	W7	tylko futra	0/6	34/40/40/40	40	0	238
✓	W8a	5 drwali bez leśników, poziom 3	6/6	32/40/40/40	40	0	238
✓	W8b	5 drwali bez leśników, poziom 5	4/6	32/40/40/40	40	0	238
✓	W9a	las 150 drzew na starcie	6/6	28/34/34/34	39	0	201
✓	W9b	las 100 drzew na starcie	6/6	25/30/31/32	38	0	183
✓	W10	poziom 6 bez Składu	6/6	32/40/40/40	40	0	236
✓	W11	Kapliczka bez Woskarni	0/6	32/40/40/40	40	0	237
✓	W12	bez żywności leśnej	6/6	18/24/24/24	24	0	130
✓	W13	zaraza –12 os. na 60. min	6/6	32/40/28/40	40	0	235
✓	W14a	pożar 3 Barci i 3 Woskarni na 70. min	6/6	32/40/40/40	40	0	176
✓	W14b	pożar 3 Garbarni i 3 Chat łowców na 70. min	6/6	32/40/40/40	40	0	176
✓	W15	pożar lasu (60% drzew) na 70. min	6/6	32/40/40/40	40	0	236
✓	W16	las spada do 80 drzew na 60. min	6/6	32/40/40/40	40	0	236
✓	W17	brak żywności do 40. min, potem odbudowa	6/6	13/10/38/56	56	28	230
✓	FK1	brak żywności do 40. min	5/6	12/10/25/25	25	26	76
✓	FK2	zaraza –12 na 60. min	6/6	25/31/31/43	43	0	197
✓	FK3	pożar łańcucha chleba i Sadu na 60. min	6/6	25/31/43/43	43	0	199
✓	SK1	brak żywności do 40. min	3/6	13/10/35/35	35	25	531
✓	SK2	zaraza –12 na 60. min	6/6	20/38/26/36	36	0	943
✓	SK3	pożar 2 Gajów i Qanatu na 60. min	6/6	20/38/38/38	38	0	903
✓	SK4	krach cen na 70. min	6/6	20/38/38/38	38	0	613
✓	VK1	brak żywności do 40. min	6/6	13/10/21/32	32	36	618
✓	VK2	zaraza –12 na 60. min	6/6	18/32/26/38	38	0	792
✓	VK3	pożar Przystani i Okrętu na 90. min	6/6	18/32/38/38	38	0	764

	Id	Scenariusz	Haracz	Ludność @20/40/60/90	Koniec	Głód	Złoto
✓	VK4	pożar Miodosytni na 90. min	6/6	18/32/38/38	38	0	794
✓	VK5	wylesienie 150 drzew	6/6	18/32/38/38	38	0	794
✓	VK6	pożar 4 Chat rybaka na 70. min	6/6	18/32/38/38	38	0	794

C Repozytorium, polecenia i budowa

C.1 Polecenia

Tabela 57. Najczęstsze polecenia (z katalogu głównego repozytorium; Node 18+, testy przeglądarkowe wymagają Playwright z Chromium)

Polecenie	Działanie
<code>node prototyp/build.js</code>	skleja <code>prototyp/src/*.js</code> w <code>prototyp/grafika_prototyp.html</code>
<code>node prototyp/build_game.js</code>	osadza silnik sprite'ów i katalog obrysów w <code>Nova_Roma.html</code> ; --check sprawdza aktualność bloków
<code>sh prototyp/podglady.sh</code>	odświeża podglądy <code>prototyp/podglad/*.jpg</code>
<code>node tools/scenarios.js</code>	58 scenariuszy akceptacyjnych + niezmienniki
<code>node tools/normal.js [plik] [nacja]</code>	normalna gra botów R/D/N (ok. 5 min dla wszystkich nacji)
<code>node tools/fauna.js, terrain.js, path.js, walkers.js, roads.js, spacing.js, leveling.js, logistics.js, footprints.js, minimap.js</code>	testy modułów (headless)
<code>node tools/browser_ui.js, browser_play.js, browser_terrain.js, browser_fauna.js, browser_sprites.js, browser_mobile.js, browser_menu.js</code>	testy w przeglądarce (Playwright)
<code>node docs/build.js</code>	buduje <code>docs/Nova_Roma_dokumentacja.pdf</code> (--html — tylko HTML, --check — sama walidacja)
<code>node docs/collect.js</code>	odświeża <code>docs/data/wyniki_testow.json</code> (uruchamia <code>normal.js</code> i <code>scenarios.js</code> ; --parse <code>a.txt b.txt</code> — z gotowych wyjść)

C.2 Skrypty testowe

Tabela 58. Skrypty w tools/ (opis z nagłówków plików)

Skrypt	Co sprawdza
<code>browser_contact.js</code>	Kontaktówka: wszystkie sprite'y budynków nacji obok siebie (z obrysem w polach i podpisem).
<code>browser_fauna.js</code>	Test fauny w przeglądarce (Faza 8): 4 klimaty × nacje startują przez <code>?play=...</code> , zwierzęta są zaludnione i rysowane, Chata myśliwego wysyła myśliwego z łukiem (celowanie → strzała w locie → upolowanie), brak błędów JS, FPS z fauną ≥ 85% FPS bez niej (<code>?fauna=0</code>); rzuty do oględzin.
<code>browser_menu.js</code>	Test menu głównego, Piaskownicy i menu w grze (Faza 9): tylko Piaskownica aktywna, klawiatura, przepływ nacja → klimat → typ → ziarno → Start dla każdej nacji (mapa gry = mapa podglądu), blokada typów map dla Wikingów, pamięć wyboru (localStorage), przycisk ☰ i Esc (pauza, priorytet nad anulowaniem budowy), „Nowa mapa” i „Menu główne” (nawigacja), widok telefonu, brak błędów JS.
<code>browser_mobile.js</code>	Widok mobilny (390×844, dotyk): ekran wyboru nacji i HUD po starcie gry.
<code>browser_play.js</code>	Rozgrywka w przeglądarce: bot buduje osadę, rzuty z panelami UI, sprawdzenie błędów JS dla wszystkich nacji.
<code>browser_sprites.js</code>	Test silnika sprite'ów w przeglądarce: wypiek każdej nacji (Sprites.bake), zgodność obrysów z katalogiem, niepuste warstwy, czas i pamięć.
<code>browser_terrain.js</code>	Test terenu v2 w przeglądarce: każda kombinacja klimat × typ mapy startuje przez <code>?play=...&climate=...&map=...&seed=...</code> (ta sama ścieżka co menu Piaskownicy), czeka na wypiek sprite'ów i podłóża (chunki), sprawdza brak błędów JS, niepusty obraz, liczbę chunków i pamięć, mierzy FPS i składa arkusz kontaktowy. nacja — domyślnie franks (Wikingowie zawsze dostają mapę nadmorską); --fps — pomiar klatek na kombinację; --sheet — arkusz 4 klimaty × 6 typów
<code>browser_ui.js</code>	Test przepływów UI w przeglądarce: budowa kliknięciem, karta budynku, haracz, przełączniki, doradca.
<code>fauna.js</code>	Test fauny (moduł Fauna, Faza 8): zaludnienie mapy wg klimatu, poprawność pozycji, własne PRNG (zero wywołań globalnego RNG), determinizm, snapshot/restore, parytet ekonomii z fauną i bez niej, wyprawy myśliwych (przebieg faz, strzała, ubytek i odradzanie zwierzyny), przetrzebieenie przy wielu Chatach, koszt ticku.

Skrypt	Co sprawdza
<code>footprints.js</code>	Test obrysów budynków (w×h pól): zgodność z katalogiem sprite'ów, zajętość i zwolnienie pól, canPlace z pierścieniem sąsiedztwa, złoża pod kopalnią na dowolnym polu obrysu, place budowy (siteAt / cancel), wyburzenie, start Dworu 4×4 i miejsca dla botów.
<code>leveling.js</code>	Test wyrównywania terenu (Faza 7): levelOf (płasko = 0, stok = pola do przesunięcia), etap 0 placu (budowniczymi kopią bez materiałów, potem teren jest płaski i zgłoszony do odświeżenia), czas budowy na płaskim bez zmian, na stoku dłuższy o minuty wyrównania, zakaz rozpiętości ≥ 2, boty wybierają płaskie miejsca, obszar startowy bez stoków, wzgórze decyzyjne dalej od Dworu.
<code>logistics.js</code>	Test sprawności logistycznej (Logistics): producent blisko Składu / Dworu ma pełną sprawność, daleki — niższą (≥ min), drogi ją poprawiają, wyłączona opcja nie zmienia nic, ekonomia produkuje mniej przy słabej logistyce i tyle samo przy dobrej, moduł nie dotyka RNG. Użycie: node tools/logistics.js [plik.html]
<code>mapstats.js</code>	—
<code>minimap.js</code>	Test podglądu mapy (moduł Minimap, Faza 9): opis, podsumowanie, rysowanie na atrapie kanwy, zgodność podglądu z mapą gry (to samo ziarno = ta sama mapa), obsługa mapy niepoprawnej i brak wpływu na stan gry. Użycie: node tools/minimap.js [plik.html]
<code>normal.js</code>	Normalna gra (150/140/200/150 min): boty R/D/N dla 4 nacji; porównanie z tabelą kryteriów akceptacji spec.
<code>path.js</code>	Test Path (A* po polach mapy): optymalność względem niezależnego Dijkstry, poprawność trasy (tylko pola przechodnie, kroki sąsiednie, bez ścinania rogów), przejście rzeki wyłącznie brodem lub lodem, brak trasy do wysp i zalanych pól, wygładzanie nie drożeje, brak RNG, determinizm, szybkość.
<code>probe_logistyki.js</code>	Pomiar logistyki w osadach botów (headless): odległości drogi drzwi producenta → najbliższy Skład / Dwór, średnia sprawność modułu Logistics i „obciążenie transportu” (Σ przepływ towaru [szt./min] × odległość [pola] — tyle „sztuko-pól na minutę” trzeba przenieść) wraz z liczbą nosicieli, których wymagałyby fizyczny transport. Wyniki są podstawą analizy w aneksie E dokumentacji (fizyczna logistyka, Faza 9B). Użycie: node tools/probe_logistyki.js [plik.html] [nacja]
<code>raids.js</code>	Napady na karawanę (Saraceni): rozkład strat po wielu ziarnach RNG; ze Strażnicami i bez.
<code>roads.js</code>	Test dróg: wytyczanie (koszt tylko za nowe pola), odmowy (woda, brak desek, zajęte pola), budowa budynku zdejmującego drogę, A* i piesi preferują drogę, narzędzie nie dotyka RNG, a pole drogi przechodzi przez JSON (snapshot/restore). Użycie: node tools/roads.js [plik.html]
<code>scenarios.js</code>	Scenariusze akceptacyjne ze specyfikacji (rozdz. „Kryteria akceptacji i testy regresji”):
<code>spacing.js</code>	Test zasad zabudowy (Data.RULES): twarda przerwa między obrysami (canPlace odrzuca budynek stykający lub w pierścieniu `gap`), miękka kara czasu budowy za sąsiadów tuż za przerwą (crowdOf → workNeed placu), brak wpływu przerwy na drogi, boty układają całą recepturę.
<code>store.js</code>	efekt Składu na wysokich poziomach: bez Składu vs ze Składem
<code>terrain.js</code>	Test generatora terenu v2: macierz 4 nacje × 4 klimaty × 6 typów map (Wikingowie: tylko nadmorska) × kilka ziaren. Sprawdza: poprawność pól kafil, walidację generatora (łączność, drzewa, złoża, miejsce pod zabudowę, brody, brzeg Wikingów), płaski plac wokół Dworu, liczby drzew i złóż względem danych nacji, reguły canPlace dla cech terenu, determinizm (to samo ziarno → ta sama mapa) i izolację globalnego RNG (Terrain.apply nie wywołuje RNG gry — przeplot losowań logiki zostaje taki jak dotąd).
<code>walkers.js</code>	Test widocznych pieszych (moduł Walkers): symulacja boty-budowniczego na kilku mapach, w każdym kroku sprawdzamy, że postacie stoją tylko na polach przechodnich i wolnych, nosiciele kończą marsz przy drzwiach celu, budowniczymi pojawiają się przy placach i znikają po budowie, a „cień symulacji” się nie zmienia: stan logiki (zasoby, ludność, położenie obywateli i nosicieli) jest identyczny z Walkers i bez, a Walkers nie wywołuje RNG.

C.3 Zasady pracy z kodem

- **Nie edytować ręcznie** bloków `SPRITE_META` i `Sprites` w `Nova_Roma.html` — zmieniamy `prototyp/src/*.js`, a potem `node prototyp/build.js && node prototyp/build_game.js`.
- Kod ładowany przy starcie musi działać z atrapami DOM z `tools/headless.js` (bez `window.location` poza `typeof` i bez literalnego `</script>` w łańcuchach).
- Logika gry nie woła `RNG` poza modułami, które już to robią (rozdz. §2.5); nowe moduły używają własnego PRNG ze stanem w JSON.
- Każda faza kończy się: testami modułu, regresją (`scenarios`, `normal`, `mapstats`, `raids`, `store`, `fauna`), commitem i **aktualizacją dokumentacji** (rozdz. §D).

D Procedura aktualizacji dokumentacji

Dokumentacja jest częścią repozytorium i **jest aktualizowana po każdym etapie prac**, jeśli etap zmienia logikę, dane, interfejs, testy lub decyzje projektowe. Tabele i wartości liczbowe pochodzą z kodu, więc większość zmian danych widać po samej przebudowie PDF; zmiany *zasad* wymagają poprawienia tekstu rozdziału.

D.1 Lista kontrolna po etapie

1. Zbuduj PDF: `node docs/build.js` (lub `--check`, jeśli nie potrzeba PDF). Komunikat „PROBLEMY” oznacza błąd dyrektywy, nieopisane pole stanu gry albo odwołanie do nieistniejącego rozdziału.
2. Jeśli zmieniono logikę ekonomii, teren, zabudowę, ruch, faunę lub interfejs — popraw odpowiedni rozdział (tabela poniżej).
3. Jeśli zmieniono liczby, które testy sprawdzają — odśwież wyniki: `node docs/collect.js`.
4. Dodaj wpis w rozdz. §17 (nowa decyzja projektowa lub fakt historyczny) i, jeśli trzeba, w rozdz. §18 (różnica względem specyfikacji).
5. Zaktualizuj `docs/meta.json` (wersja dokumentu, opis stanu projektu na okładce) i tabelę faz w rozdz. §1.3.
6. Obejrzyj PDF (strony → PNG, np. `pdftoppm -r 60 -png docs/Nova_Roma_dokumentacja.pdf /tmp/doc`), popraw układ.
7. Zacommituj razem z kodem etapu: źródła rozdziałów i `docs/Nova_Roma_dokumentacja.pdf`.

D.2 Który rozdział zmienia która zmiana

Tabela 59. Mapa zmian i rozdziałów

Zmiana w grze	Rozdziały do przeglądu
nowy budynek, zmiana kosztu lub tempa (<code>Data.BUILDINGS</code>)	6 (tabele z kodu), 7 (łańcuchy i przepisy), 8 (lista <code>NEED_LIST</code>), 16
ekonomia: produkcja, obsada, jedzenie, imigracja, popularność	3, 4, 18
haracz, terminy, presety, rynek, wyprawy	7, 8, 9, aneks A
teren, klimaty, typy map, generator	10, 11
zasady zabudowy, wyrównywanie, strefy	5, 11
ścieżki, drogi, piesi, logistyka	12
fauna, polowanie	13
HUD, Doradca, menu, parametry adresu	1, 14
sprite'y, render, wydajność	15
testy, skrypty, regresja	16, aneksy B i C
nowy moduł lub zmiana kolejności ticku	2 (diagramy ręczne w <code>docs/diagrams.js</code>)

E Fizyczna logistyka i żywienie ludzi — analiza i projekt (Faza 9B)

Ten aneks opisuje **planowaną** zmianę: ludzie w grze mają naprawdę wykonywać swoje zadania, a czas i odległość mają wpływ na ekonomię (drogi stają się ważniejsze), **bez zepsucia dotychczasowego bilansu**. Powstał jako odpowiedź na prośbę projektową i zawiera analizę wykonalności z pomiarami, model, zasady ochrony bilansu, plan etapów i bramki. Faza 9B następuje po Fazie 9 (menu i Piaskownica), a przed Fazą 10 (integracja) i 11 (audyt wizualny).

E.1 Zamówienie

DECYZJA PROJEKTOWA

Prośba użytkownika (cytat w skrócie): „Trzeba ożywić ludzi — drwal musi fizycznie ściąć drewno, cieśla zabrać [pień], dostarczyć do siebie i przeciąć, budowniczy musi dojść na plac budowy i zacząć budowę, karawana wyruszyć, dojść na koniec mapy, a potem wrócić i tak dalej. Każda postać musi wykonywać swoje zadanie. Nie zepsuj przy tym bilansu rozgrywki. Tym bardziej ważne będą drogi. Możesz zainspirować się grą Kings and Merchants [Knights and Merchants]; był w niej centralny magazyn — jeśli może pomóc w bilansie i niczego nie zepsuje, można go dodać. Sprawdź, czy te zmiany nie zepsują bilansu; mimo wszystko chciałbym je wprowadzić, więc postaraj się.”

Wymagania wynikające z prośby: (1) czynności postaci są **widoczne i kauzalne** (budynek nie ruszy, dopóki budowniczy nie dojdzie; towar nie trafi do magazynu, dopóki ktoś go nie przyniesie), (2) bilans ekonomii **nie ulega pogorszeniu** na typowej osadzie, (3) **drogi i magazyny** zyskują znaczenie, (4) rozważamy centralny magazyn.

E.2 Stan obecny

Dziś ruch jest „cieniem symulacji” (rozdz. §12): ekonomia liczy nominalne tempa, a ludzie na ekranie chodzą w tle. Prędkości widocznych postaci (0,25 / 0,35 / 1,4 / 2,2 pola na minutę) nie mają związku z czasem logiki (budowniczy nosi sztukę materiału „w 0,2 min” bez względu na odległość; producent odkłada towar w Składzie bez czasu). Jedyną zależnością od odległości jest sprawność **Logistics** (producent daleko od Składu daje do –30% mniej) — liniowa i niezależna od rodzaju budynku. Postaci nie wykonują więc swoich zadań: drwal nie ścina konkretnego drzewa, a „nosiciel” jest ozdobą.

E.3 Wniosek 1: tempo gry decyduje o tym, jak szybko mogą chodzić ludzie

Czas gry biegnie dziś **4 s na minutę gry** (przy prędkości $\times 1$). Tempa ekonomii są „na minutę” (drwal: 1 pień/min, Tartak: 2 deski/min), więc w czasie rzeczywistym drwal powinien produkować pień co 4 sekundy. Jeśli ludzie mają fizycznie chodzić po drzewa i do magazynu (odległości 5–25 pól), muszą się poruszać **szybko albo gra musi biec wolniej**: widoczna prędkość = $\text{pola/min} \div \text{sekundy na minutę}$.

Tabela 60. Prędkość chodu ludzi (pola na minutę gry) a sprawność transportu przy odległości 12 pól (ładunek 6 szt., obsługa 0,1 min) i prędkość widoczna na ekranie [pola/s] przy tempie gry 4, 12 i 20 s na minutę

v [pola/min]	Sprawność bez drogi (12 pól)	Sprawność po drodze (7,2 pola)	Ekran: 4 s/min [pola/s]	12 s/min	20 s/min
6	0,802	0,953	1,5	0,5	0,3
12	0,877	0,973	3	1	0,6
18	0,91	0,981	4,5	1,5	0,9
24	0,93	0,985	6	2	1,2

v [pola/min]	Sprawność bez drogi (12 pól)	Sprawność po drodze (7,2 pola)	Ekran: 4 s/min [pola/s]	12 s/min	20 s/min
30	0,942	0,988	7,5	2,5	1,5

Żeby fizyczny transport miał narzut rzędu 9% na typowej odległości 12 pól (tyle ma dziś model liniowy), ludzie muszą pokonywać ok. **18 pól na minutę gry**. Przy obecnym tempie 4 s/min to **4,5 pola na sekundę** (postać „biegnie” po ekranie 4× szybciej niż spacerujący), przy tempie 12 s/min — **1,5 pola/s**, czyli zwykły marsz (pole ma 128 px szerokości). Wniosek: aby postacie wyglądały wiarygodnie, **bazowe tempo gry należy spowolnić do ok. 12 s na minutę gry** (przyciski prędkości $\times 1 / \times 2 / \times 3 / \times 6$, gdzie $\times 3$ odpowiada dzisiejszemu $\times 1$). Zmiana nie dotyczy bilansu: ekonomia, testy i terminy haraczu liczone są w minutach gry; zmienia się tylko, ile sekund trwa minuta.

E.4 Wniosek 2: skala transportu

Pomiar w osadach botów (tabela poniżej; skrypt `tools/probe_logistyki.js`) pokazuje, ile towaru trzeba przenieść: **27–32 szt./min wyjść i 10–16 szt./min wejść** (w 90. minucie), średnio ok. 12 pól od magazynu, czyli obciążenie transportu rzędu **400–780 sztuko-pól na minutę** (w 40. minucie ok. 370–540).

Tabela 61. Osady botów R w 90. minucie (bez dróg, logistyka włączona; pomiar `tools/probe_logistyki.js`): odległość drogi drzwi producenta do najbliższego Składu lub Dworu, sprawność modułu Logistics, przepływy towarów, obciążenie transportu (Σ przepływ $\times$ odległość) i liczba nosicieli potrzebnych przy 18 polach/min

Nacja	Ludność	Producenci	Mediana [pola]	P90	Maks.	Śr. sprawność	Wyjścia [szt./min]	Wejścia [szt./min]	Obciążenie [szt.·pole/min]	Wolni ludzie	Nosiciele (v = 18)
Frankowie	37	22	12,5	20,1	25,3	0,879	32,3	16,1	741	8	13,7
Saraceni	34	22	12,2	16,5	28,3	0,897	30,5	9,7	538	2	10
Wikingowie	34	21	13,8	21,2	40,4	0,884	27,5	13,7	597	6	11,1
Słowianie	36	24	11,8	19,1	23	0,899	27,1	9,7	476	4	8,8

Konsekwencje:

- Nosiciele z puli ludności nie wchodzą w grę.** Przy ładunku 6 szt. i prędkości 18 pól/min jeden nosiciel przenosi 54 szt.·pole/min, czyli potrzeba 7–14 nosicieli, a wolnych ludzi (ludność – pracownicy – budowniczy) jest 2–11. Gdyby nosiciele byli ludnością, osada byłaby ograniczona transportem o 30–70% i ekonomia zostałaby przestrojona od nowa (nominalne tempa specyfikacji są „netto po marszu robotnika” — praca przy noszeniu jest już w nich zawarta).
- Producent sam noszący towar** (model ze specyfikacji) działa tylko dla producentów o małych przepływach: Tartak (2 deski/min wyjścia i 1 pień wejścia) zająłby przy 12 polach ok. 2/3 czasu pracownika, Piekarnia (3,3 chleba/min) więcej niż cały czas. Czas noszenia **musi więc być interpretowany jako sprawność (narzut), a nie dosłowna wędrówka jedyne robotnika.**
- Magazyn centralny** istnieje już w grze: Dwór (komora 20 szt. każdego towaru) jest celem transportu, gdy nie ma Składu, a Skład (+80 szt.) jest magazynem okręgowym; odległość do najbliższego z nich wyznacza sprawność. Osobny budynek „Magazyn centralny” nie jest potrzebny do bilansu (rozważamy go dopiero, jeśli bramki bilansu wykażą, że gracz nie ma jak naprawić odległych producentów — „Duży Skład” z większym promieniem bez kary).

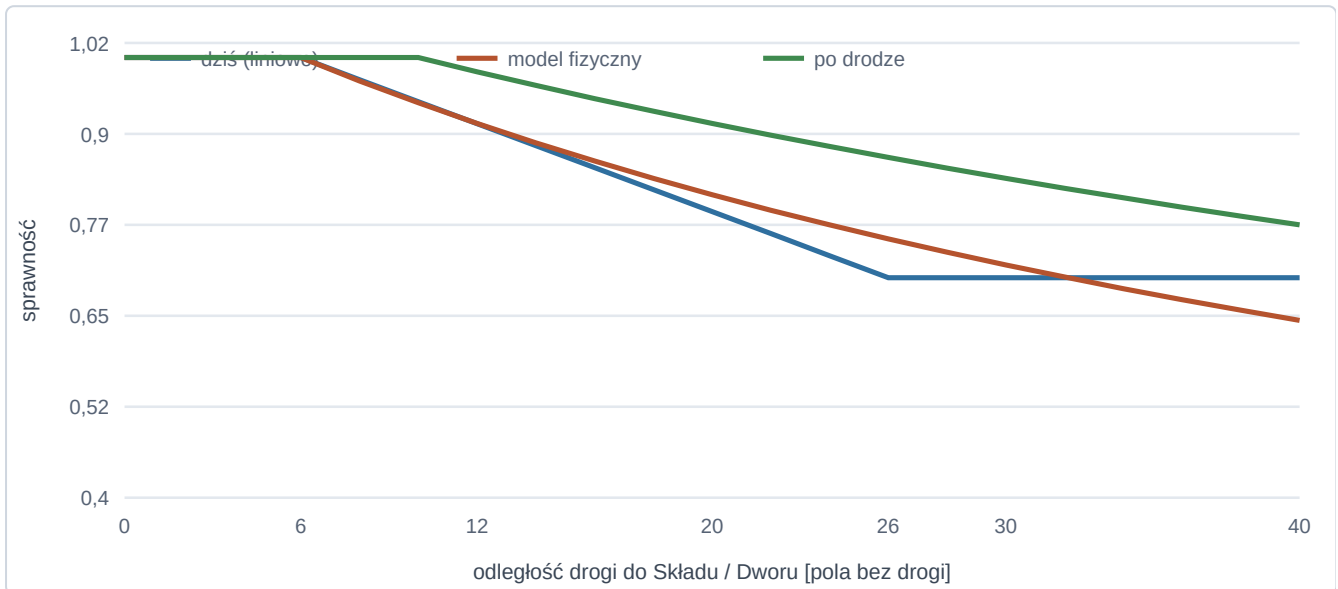
E.5 Model: sprawność z cyklu pracy

Fizyczny model ma być **kalibrowany do tego, co już zatwierdzono** (reguła specyfikacji: „kalibruj prędkość chodu i rozmiar ładunku, nie liczby z tabeli”). Każda czynność dostaje cykl `czas pracy + czas dojścia`, a ekonomia dostaje sprawność:

$$\text{eff} = \min(1, (T_{\text{praca}} + T_{\text{dojścia}}(L_{\text{ref}})) / (T_{\text{praca}} + T_{\text{dojścia}}(L))) \quad T_{\text{dojścia}}(L) = 2 \cdot L / v + t_{\text{obsługi}}$$

Cechy modelu:

- **Tylko na niekorzyść:** $\text{eff} \leq 1$ — mała odległość nie daje nadwyżki ponad tempo nominalne (producent odczeka, to widać jako postój), więc **nic nie może przyspieszyć gospodarki ponad specyfikację**; odległość powyżej odległości odniesienia L_{ref} (dziś 6 pól) obniża plan.
- **Kalibracja do dzisiejszej krzywej:** ładunek 6 szt. (pojemność lokalnego składziku producenta), obsługa 0,1 min i prędkość **18 pól/min** odtwarzają obecną krzywą `Data.RULES.logistics` z błędem $\leq 0,06$ w całym zakresie 0–40 pól (przy 12,5 pola: 0,90 wobec 0,90; przy 20: 0,81 wobec 0,79; przy 26: 0,75 wobec 0,70).
- **Drogi** zmniejszają koszt pola do 0,6 (a prędkość rośnie $\times 1,6$): ta sama odległość po drodze daje sprawność ok. 0,98 zamiast 0,90 — **droga naprawdę zwraca się** w osadach, w których budynki stoją daleko od magazynu.



Rysunek 13. Krzywa sprawności dziś (liniowa, `Data.RULES.logistics`) i jej fizyczna interpretacja: cykl pracy producenta niosącego 6 szt., $v = 18$ pól/min, obsługa 0,1 min; trzecia linia — ta sama odległość po drodze (koszt pola 0,6)

E.5.1 Role i ich cykle

Poszczególne klasy czynności mają własne składniki cyklu; wszystkie liczone są z tej samej mapy kosztów

`Path.field`:

Tabela 62. Fizyczne składniki sprawności (plan)

Rola	Cykl widoczny na ekranie	Co zależy od odległości (sprawność)
Drwal	idzie do konkretnego drzewa (pole z <code>trees > 0</code> najbliższej chaty), ścina (animacja, drzewo znika z pola), niesie pień do drzwi / Składu	odległość do najbliższego drzewa (próg 4 pola, spadek na 12 polach do min 0,5) — las musi rosnąć blisko chaty (Leśniczówka ma sens); kolejność ścinania deterministyczna (bez globalnego <code>RNG</code>)
Leśnik	idzie na wolne pole obok, sadi sadzonkę, wraca	odległość do wolnego pola pod sadzonkę
Cieśla (Tartak)	idzie po pień (do Składu / drwala), niesie do tartaku, piłuje (animacja), niesie deski do magazynu	odległość wejść (pnie) i wyjść (deski)
Górnicy, kamieniarze, gliniarze	schodzą w głąb obrysu, wracają z taczka do drzwi, wynoszą do magazynu	odległość wyjścia do magazynu
Rolnicy, zbieracze, sadownicy, rybacy	pracują w polu / sadi / lesie / na brzegu (animacje narzędzi), niosą kosz / sieć	odległość wyjścia; rybak musi mieć dojście do brzegu
Młynarz, piekarz, kasza, huta, kuźnia, zbrojownia...	po drodze w obie strony: wejścia z magazynu, wyjścia do magazynu	odległość wejść i wyjść (dojazd łączony: ładunek w obie strony)
Budowniczy	z Dworu / Składu z materiałem idzie na plac ; budowa startuje dopiero po dojściu; noszenie partiami po 4 szt.	czas dojścia przed pracą; czas noszenia = odległość Dwór/Skład → plac

Rola	Cykl widoczny na ekranie	Co zależy od odległości (sprawność)
Myśliwy	(jest) idzie, celuje, strzela, niesie	odległość do zwierzyny (jest)
Karawana (Saraceni)	wyrusza z Targu z partią towarów, idzie do krawędzi mapy , sprzedaje, wraca z kasą; złoto trafia do skarbca po powrocie	opóźnienie $2 \cdot L/v$; liczba karawan = Targi i Karawansera
Statek (Wikingowie)	odpływa z Przystani na czas wyprawy i wraca z łupem (jest w logice)	bez zmian logiki; tylko ruch na ekranie
Wolni obywatele	siedzą, spacerują, idą na Targ / do Kościoła	brak (nie pracują)

E.5.2 Budowniczy

Dziś budowniczy nosi sztukę w 0,2 min (5 szt./min) i buduje od razu. W modelu fizycznym: dojście z Dworu na plac trwa L / v (czas bez pracy), noszenie idzie partiami po $c = 4$ szt.: $\text{czas partii} = 2 \cdot L_{\text{skład} \rightarrow \text{plac}} / v + 0,1$ min, czyli 0,2 min/szt. pozostaje przy odległości $\leq$ ok. 6 pól, a rośnie wraz z nią (na 14 polach ok. 0,4 min/szt.). Ponieważ w specyfikacji budowniczy mają 60–73% bezczynności (wąskim gardłem są deski), wydłużenie noszenia o kilkadziesiąt procent na dalekich placach powinno mieścić się w tym luzie; sprawdza to bramka bilansu.

E.5.3 Karawana

Targ sprzedaje dziś natychmiast z całego zapasu (rozdz. 58). W modelu fizycznym towar handlowy (kadzidło, tkanina, ceramika) zbiera się przy Targu do partii (6 szt. lub po 3 min), a karawana (wielbłądy) idzie do **najbliższej krawędzi mapy** (ok. 24 pola od Dworu, $\approx 1,3$ min przy 18 polach/min), tam następuje sprzedaż po aktualnej krzywej cen, a złoto wraca w drugim 1,3 min. Przepływ jest ten sam, zmienia się opóźnienie ≈ 3 min (gotówka w pierwszej sprzedaży pojawia się później), co przy pierwszym terminie w 50. minucie nie ma znaczenia. Napady na karawanę zostają liczone ze skarbca (wg specyfikacji), a na ekranie ilustrowane zdarzeniem na trasie karawany.

E.6 Ochrona bilansu

Zasady, które mają zagwarantować „bez zepsucia bilansu”:

- Przełącznik** `settings.physical` (jak `logistics` i `fauna`): w testach specyfikacji `false` — **58 scenariuszy daje dokładnie te same wyniki**; w grze włączony (`?physical=0` wyłącza).
- Nigdy ponad nominal:** $\text{eff} \leq 1$ dla każdej składowej; nowe składowe (wejścia, drzewa, budowa) mają własne odległości odniesienia dobrane tak, by **średnia sprawność w osadach botów nie spadła poniżej dzisiejszych 0,88** (budżet narzutu $< 10\text{--}12\%$ dla osady „bez organizacji”, $\approx 0\text{--}3\%$ dla osady z drogami i Składami; spec.: narzutu 15% obniża ludność Franków o 18%).
- Bramka bilansu w testach** (`tools/physical.js`): boty R, D i N czterech nacji przy `physical = true` osiągają $\geq 90\%$ wyniku bazowego (ludność @60/90/150, terminy haraczu) i nie wchodzą w głód dłuższy niż 10 min; scenariusze kryzysowe (brak jedzenia, pożar, zaraza) dają się uratować; dodanie drogi lub Składu **podnosi** wynik (czułość: $\geq +3$ pp sprawności na producenta oddalonego o ≥ 15 pól).
- Determinizm i JSON:** stan fizyczny w `state.phys` (JSON), zero wywołań globalnego RNG (wybór drzew deterministyczny: najbliższe pole z drzewami), snapshot / restore jak dotąd.
- Brak zakleszczeń:** brak dojścia (zabudowa odcięła drogę) daje sprawność minimalną (nie zero) i komunikat „brak dojścia” w Doradcy.
- Wydańność:** pola kosztów `Path.field` liczone raz na zmianę układu (jak dziś w `Logistics`), cykle w logice to wzory, a nie symulacja zdarzeń; ruch postaci to wizualizacja zsynchronizowana z tymi wzorami (`Walkers`).

E.6.1 Rozważone i odrzucone warianty

- Pełna symulacja zdarzeniowa towarów** (każda sztuka jako obiekt niesiony przez postać, produkcja dopiero po dojściu): najwierniejsza, ale przy dzisiejszym tempie i skali (30 szt./min na 12 pól) wymaga $\sim 10\text{--}14$ nosicieli, których gra nie ma (patrz wniosek 2), a każda niedokładność zmienia ekonomię specyfikacji — **odrzucona jako zbyt ryzykowna dla bilansu**.

- **Wspólna pula nosicieli z ludności (styl Knights and Merchants):** pojemność transportu byłaby ograniczona wolną ludnością (2–11), a wczesna gra nie ma wolnych ludzi; wymagałaby przestrojenia ekonomii. Może wrócić jako **opcja zaawansowana** („Nosiciele” w Składach), jeśli bramki pokażą, że lokalna sprawność jest zbyt łagodna.
- **Producent jako jedyny nosiciel:** niewykonalne dla budynków o dużych przepływach (Piekarnia, Tartak, Huta) — patrz wniosek 2.

E.7 Ożywienie wizualne

Postaci w grze zostają **jedyną widoczną reprezentacją** pracy; ich liczba odpowiada ludności: pracownicy obsadzonych budynków + budowniczy + wolni obywatele (załoga na wyprawie znika z ekranu). Każdy obsadzony budynek ma swojego mieszkańca z pętlą czynności o okresie $1 / (\text{tempo} \times \text{eff})$, a stan budynku jest widoczny: praca, **postój** (brak wejścia, pełny magazyn — pracownik siedzi lub stoi w drzwiach), brak pracownika (pusty budynek). W razie przeszkód (zabudowanie drogi, zatopienie przejścia) czynność się zatrzymuje i Doradca pokazuje powód. Wymagane elementy graficzne to rekwizyty w ręku (siekiera, piła, kilof, motyka, sierp, młot, kosz, wiadro, sieć, worek, taczka) i niesione towary (pień, deska, kamień, worek mąki, chleb, ryba, sztabka...) w `prototyp/src/40_nature.js` (`bakeFigure`), a dla karawany i statków — `sprite'y` wielbłąda z towarem i łodzi.

E.8 Plan etapów i bramki

Tabela 63. Etapy Fazy 9B

Etap	Zakres	Bramka
9B-0	tempo gry (sekundy na minutę , domyślnie 12; przyciski $\times 1/\times 2/\times 3/\times 6$, <code>?tempo=</code>), kalibracja prędkości postaci (≈ 18 pól/min), przegląd wizualny	testy bez zmian (dt-based); szybki przegląd ekranu
9B-1	role i czynności (wizualne, zsynchronizowane ze stanem logiki): mieszkaniac per budynek, pętla, rekwizyty, wolni obywatele; zero zmian ekonomii	scenariusze i <code>normal</code> identyczne; <code>browser_workers.js</code> (liczba postaci = ludność, każdy obsadzony budynek ma pracownika w zasięgu)
9B-2	<code>settings.physical</code> : sprawność z cyklu pracy (wyjścia, wejścia), panel „Transport”, status „brak dojścia”, droga i Skład zwracają się	<code>tools/physical.js</code> : parytet przy <code>false</code> , kalibracja krzywej, czułość na drogę i Skład, determinizm, snapshot
9B-3	drwal i leśnik fizycznie (najbliższe drzewo, deterministyczny wybór), budowniczy (dojście, partie po 4 szt.)	bramka bilansu: boty R/D/N $\geq 90\%$ wyniku bazowego; scenariusze kryzysowe do uratowania
9B-4	karawany (Saraceni) i statki (Wikingowie) na ekranie i w opóźnieniu logiki; napady zilustrowane na trasie	scenariusze Saracenów (S1–S7, SK1–SK4) w zakresie specyfikacji; <code>raids.js</code> bez zmian średniej straty
9B-5	strojenie, ew. „Duży Skład”/nosiciele opcjonalni, dokumentacja (rozdz. 12, aneks E → „Stan”), wersja WIP, nowy wzorzec regresji	komplet testów, FPS $\geq 85\%$ sprzed zmian

E.9 Decyzje do potwierdzenia

Poniższe wartości przyjęto domyślnie; można je zmienić bez przebudowy architektury (są stałymi w danych):

1. **Tempo bazowe 12 s na minutę gry** (zamiast 4) i przyciski $\times 1/\times 2/\times 3/\times 6$ — warunek wiarygodnego wyglądu postaci; $\times 3$ odpowiada dzisiejszemu tempu.
2. **Napady na karawanę nadal ze skarbca** (specyfikacja), karawana tylko ilustruje i opóźnia sprzedaż.
3. **Magazyn centralny = Dwór**; nowy budynek („Duży Skład”) tylko jeśli bramki bilansu tego wymagają.